Multi-Channel Web-Integration für föderiertes Adressmanagement: Design und Implementierung einer Self-Sovereign Address Lösung

Masterarbeit zur Erlangung des akademischen Grades
Master of Science
im Studiengang Digital Sciences
an der Fakultät für Informatik und Ingenieurwissenschaften
der Technischen Hochschule Köln
Campus Gummersbach

vorgelegt von:	Christian Holländer
Matrikel-Nr.:	112 628 60
Adresse:	Oelschlägerstraße 57 41844 Wegberg christian.hollaender@smail.th-koeln.de
eingereicht bei:	Prof. Christian Noss
Zweitgutachter*in:	Prof. Dr. Stefan Bente

Wegberg, 27. April 2026

Kurzfassung

In Deutschland ziehen jährlich rund acht Millionen Menschen um – und jeder Umzug zieht eine Kaskade manueller Adressänderungen bei dutzenden Diensten nach sich. Bestehende Lösungsansätze auf Basis dezentraler Identitäten (Self-Sovereign Identity) scheiterten nicht an technischer Unreife, sondern an einem Henne-Ei-Problem: Ohne teilnehmende Dienste kein Nutzwert, ohne Nutzwert kein Anreiz zur Teilnahme.

Der wissenschaftliche Beitrag dieser Arbeit sind drei generalisierbare Architekturprinzipien: **Partnerunabhängigkeit durch abgestufte Kanalstrategie** (die unterste Stufe liefert Mehrwert ohne externe Kooperation), **zentraler Vertrauens-Provider** (Zentralisierung ermöglicht stellvertretende Authentifizierung, die ein dezentrales Modell ausschließt) und **Graceful Degradation** (explizites Sichtbarmachen von Handlungsbedarf statt stummem Versagen). Diese Prinzipien gelten über den Adressmanagement-Kontext hinaus für jede föderierte Stammdaten-Architektur.

Zur Validierung entwirft und implementiert diese Arbeit *SMove* – eine Multi-Channel-Integrationsarchitektur mit dreistufiger Kanalstrategie: **Level 1** (E-Mail-Vorlagen) deckt alle Dienste ohne jede externe Kooperation ab, **Level 2** (Browser-Extension) ermöglicht semi-automatische Formular-Befüllung als Brückentechnologie, **Level 3** (API-Integration) demonstriert vollautomatische Erweiterbarkeit ohne Architekturänderungen. Die szenarienbasierte Evaluation zeigt, dass der entscheidende Effekt weniger in der Zeitersparnis liegt als in der *strukturellen Unterstützung*: Das System macht offenen Handlungsbedarf explizit sichtbar und reduziert das Risiko, einzelne Dienste zu vergessen.

Schlüsselwörter: Adressmanagement, Multi-Channel-Integration, Browser-Extension, föderierte Datenweitergabe, Self-Sovereign Identity, Datensouveränität, inkrementelle Adoption, Graceful Degradation

Inhaltsverzeichnis

Tabellenverzeichnis VII

Abbildungsverzeichnis VIII

Abkürzungsverzeichnis X

1. Einleitung 1
1.1. Motivation . . . 1
1.2. Problemstellung und Forschungsfrage . . . 2
1.3. Vorgehen und Aufbau . . . 3

2. Grundlagen 5
2.1. Digitale Identitäten und das Konzept der Datensouveränität . . . 5
2.2. Anforderungen an sichere Datenverwaltung . . . 5
2.3. Bestehende Ansätze – Überblick und zentrale Learnings . . . 6
2.3.1. Praktische Lösungen am Markt . . . 6
2.3.2. Self-Sovereign Identity in der Praxis: Sovrin, Trinsic und Co. . . . 7
2.3.3. KI-Browser-Agenten als aufkommende Kategorie . . . 8
2.4. Multi-Channel-Integration als Architekturmuster . . . 9

3. Anforderungsanalyse 11
3.1. Problemraum im Detail . . . 11
3.1.1. Nutzergruppen und Problemszenarien . . . 11
3.1.2. Pain Points aus Nutzersicht . . . 11
3.1.3. Problemdimension und Relevanz . . . 12
3.2. Realitätscheck: Was externe Dienste wirklich können . . . 12
3.3. Funktionale Anforderungen . . . 13
3.3.1. Nutzer- und Dienstverwaltung . . . 13
3.3.2. Auswahl und Einwilligung . . . 13
3.3.3. Ausführung, Status und Fehlerbehandlung . . . 14
3.4. Nicht-funktionale Anforderungen . . . 14
3.5. Scope und Priorisierung . . . 14
3.5.1. Feature-Reichhaltigkeit vs. Fokus . . . 15
3.5.2. Priorisierung für MVP . . . 15

4. Architekturdesign **17**
4.1. Designprinzipien . . . 17
4.1.1. Partnerunabhängigkeit . . . 17
4.1.2. Zentralisierung vs. Dezentralisierung . . . 17
4.1.3. Privacy-by-Design . . . 18
4.1.4. Selektiver API-Einsatz . . . 18
4.1.5. Pragmatismus über Idealismus . . . 18
4.2. Sicherheitskonzept . . . 18
4.2.1. Bedrohungsmodell . . . 19
4.2.2. Datenhaltung und Verschlüsselung . . . 19
4.2.3. Authentifizierung . . . 19
4.2.4. Ressourcen-basierte Autorisierung . . . 20
4.2.5. Audit-Logging . . . 20
4.3. Systemkomponenten . . . 20
4.3.1. Address & Relocation Management . . . 21
4.3.2. Service Registry . . . 22
4.3.3. Notification Engine . . . 22
4.3.4. Datenmodell . . . 22
4.3.5. Prozessablauf einer Adressänderung . . . 23
4.4. Multi-Channel Integrationsstrategie . . . 26
4.4.1. Level 1: Manuelle Benachrichtigungen . . . 26
4.4.2. Level 2: Semi-automatische Integration via Browser-Extension . 26
4.4.3. Level 3: Vollautomatische API-Integration . . . 27
4.4.4. Vergleich mit bestehenden Ansätzen . . . 27

5. Implementierung **31**
5.1. Technologie-Stack . . . 31
5.2. Kernkomponenten . . . 31
5.2.1. Backend-Architektur . . . 32
5.2.2. Frontend und Web-Oberfläche . . . 32
5.2.3. Browser-Extension . . . 33
5.3. Multi-Channel-Implementierung . . . 36
5.3.1. Level 1: E-Mail-basierte Adressänderung . . . 36
5.3.2. Level 2: Semi-automatische Integration via Browser-Extension . 37
5.3.3. Level 3: API-Integration . . . 38
5.4. Herausforderungen und Lösungsansätze . . . 40
5.4.1. Schema-Evolution und Datenmodell-Refactoring . . . 40
5.4.2. Statusmaschine und Flow-Hierarchie . . . 40
5.4.3. Bestätigungs-Workflow . . . 40
5.4.4. Formular-Heterogenität und Manifest-V3-Einschränkungen . . . 41

6. Evaluation **43**
6.1. Technische Evaluation durch automatisierte Tests 43
6.1.1. Testaufbau und -umfang 43
6.1.2. Testabdeckung 44
6.1.3. Repräsentative Testfälle 44
6.2. Szenarienbasierte Evaluation 45
6.2.1. Szenario 1: Einzelner Umzug mit wenigen Diensten (Level-1-Integration) 46
6.2.2. Szenario 2: Wiederholte Umzüge mit wachsender Dienste-Landschaft (Level-2-Integration) 48
6.2.3. Szenario 3: Nutzung durch weniger technikaffine Personen (Zero-Setup-Ansatz) 49
6.3. Diskussion der Evaluationsergebnisse 54

7. Diskussion und Ausblick **57**
7.1. Generalisierbarkeit und architektonische Implikationen 57
7.1.1. Übertragbarkeit auf andere Datentypen 57
7.1.2. Architektonische Implikationen für ähnliche Systeme 57
7.1.3. Datensouveränität ohne SSI 58
7.2. Limitierungen und Herausforderungen 58
7.3. Offene Forschungsfragen 60

8. Zusammenfassung **63**

Literaturverzeichnis **65**

Weiterführende Literatur **69**

A. Anhang **72**
A.1. Repositories und Demo-Deployment 72
A.2. Evaluierte Dienste – Realitätscheck 72
A.3. Screenshots 73
A.3.1. Webanwendung 73
A.3.2. Browser-Extension 77
A.4. OAuth-2.0-Flow der Browser-Extension 77
A.5. Build-vs.-Buy-Entscheidungen 78
A.6. Ausgewählte Code-Listings 79
A.6.1. BaseExecutionStrategy 79
A.6.2. ExecutionStrategy-Enum 80
A.6.3. ServiceWebsiteConfig – Struktur der Feldmappings 81
A.6.4. Generische Felderkennung im Content Script 82

A.6.5. ProcessAddressChangeAttemptJob – Retry-Mechanismus . . . 83
A.6.6. AddressChangeFlowObserver – Status-Propagierung 85
A.7. Datenbankschema – Entity-Relationship-Diagramme 86

Tabellenverzeichnis

3.1. Nicht-funktionale Anforderungen 14

4.1. Vergleich: SMove vs. bestehende Ansätze 27

6.1. Code-Coverage nach Modul (Zeilenabdeckung) 44
6.2. Ausgewählte Testfälle und ihr Ergebnis 45
6.3. Heuristische Einordnung der Usability-Befunde (Szenario 3) 53

A.1. Repositories der Implementierung 72
A.2. Evaluierte Dienste und gefundene Adressänderungskanäle 73
A.3. Build-vs.-Buy-Entscheidungen im Technologie-Stack 78

Abbildungsverzeichnis

4.1. Komponentendiagramm: Gesamtarchitektur von SMove 21
4.2. Prozessablauf einer Adressänderung (kanalunabhängig) 24
4.3. Neunstufige Statusmaschine: *AddressChange*-Status und *AddressChangeFlow*-Status mit Observer-gesteuerten Übergängen (gestrichelt) . . . 25
4.4. Sequenzdiagramm: Level-2-Integration über die Browser-Extension . . 29

5.1. Dashboard-Übersicht mit aktuellem Umzug und Statusanzeige 33
5.2. Umzugs-Detailseite: Fortschrittsübersicht, verknüpfte Dienste und Adressinformationen 34
5.3. Extension-Popup: Tab „Aktueller Tab“ – ohne Treffer (links) und mit erkanntem Dienst (rechts) 36
5.4. SMove-Prozessstruktur: Einmalige Setup-Schritte, pro-Umzug-Schritte und pro-Dienst-Ausführung nach Integrationsstufe 39

6.1. Prozessvergleich: Status Quo (links) vs. SMove (rechts). Rot: pro Dienst wiederholte Schritte; blau: einmaliger Setup. 47
6.2. Nach Öffnen der Webseite des Cloud-Anbieters Hetzner: Bestätigung ausstehend, 1 Versuch, Nutzer wird zur manuellen Rückmeldung aufgefordert 51
6.3. Abgeschlossene Adressänderung: Status *Abgeschlossen*, Versuch erfolgreich, manuelle Bestätigung protokolliert 52

A.1. Formular zum Erstellen eines Umzugs mit Feldern für bisherige und neue Adresse sowie Umzugsdatum 74
A.2. Dienst einem Umzug hinzufügen 75
A.3. Dashboard nach Registrierung: Leerer Startzustand mit dreischrittiger Onboarding-Anleitung 76
A.4. Extension-Popup: Tab „Aktiver Umzug“ mit Fortschrittsübersicht und offenen Änderungen 77
A.5. Vollständiges ER-Diagramm (Übersicht) 87
A.6. ER-Diagramm: Nutzer, Adressen, Registrierungen und Audit-Log . . . 88
A.7. ER-Diagramm: Service Registry mit typisierten Kanalkonfigurationen . 90
A.8. ER-Diagramm: Relocation, AddressChange und Flow 91

A.9. ER-Diagramm: Flow-Ausführung – Attempts, Confirmations und Challenges 92

Abkürzungsverzeichnis

AES Advanced Encryption Standard. 19

API Application Programming Interface. 2, 3, 6, 8, 9, 12–15, 18–20, 22, 23, 26, 27, 31–33, 37, 38, 46, 50, 54, 55, 63, 73, 78, 80, I

CSRF Cross-Site Request Forgery. 19

CSS Cascading Style Sheets. 9, 34, 41, 54, 59, 81

DID Decentralized Identifier. 5

DOM Document Object Model. 8, 33, 37

DSGVO Datenschutz-Grundverordnung. 2, 7–9, 11, 14, 18, 20, 26

DSR Design Science Research. 3, 40

EAI Enterprise Application Integration. 9, 10, 60

eIDAS electronic Identification, Authentication and trust Services. 7

ER Entity-Relationship. 22

GraphQL Graph Query Language. 18, 20, 26, 31, 37, 43–45

HTML Hypertext Markup Language. 34, 35, 82

HTTP Hypertext Transfer Protocol. 32, 43, 45

HTTPS Hypertext Transfer Protocol Secure. 14

JSON JavaScript Object Notation. 23, 31, 40, 81

KI Künstliche Intelligenz. 8, 9, 93–95

MVP Minimum Viable Product. 15

OAuth Open Authorization. 19, 20, 26, 31, 33, 34, 44, 45, 77, 78, 94

ORM Object-Relational Mapping. 31

PKCE Proof Key for Code Exchange. 19, 26, 31, 33, 77, 78

PSD2 Payment Services Directive 2. 12

REST Representational State Transfer. 18, 20, 31

SaaS Software as a Service. 8, 27

SPA Single Page Application. 31

SSI Self-Sovereign Identity. 5, 7, 8, 15, 17, 27, 58–60, 63, 94

TLS Transport Layer Security. 14

UI User Interface. 32, 37, 78, 80

URL Uniform Resource Locator. 22, 26, 35, 43, 45, 54, 59

UUID Universally Unique Identifier. 37, 40, 41

WAI Web Accessibility Initiative. 35

WHATWG Web Hypertext Application Technology Working Group. 35

1. Einleitung

1.1. Motivation

Ein Umzug gehört zu den stressintensivsten Lebensereignissen – nicht nur wegen des physischen Aufwands, sondern vor allem aufgrund der administrativen Hürden. In Deutschland ziehen jährlich rund acht Millionen Menschen um (Deutsche Post Adress GmbH & Co. KG, 2024, S. 6), und jeder dieser Umzüge löst eine Kaskade an Adressänderungen aus, die bei einer Vielzahl von Institutionen, Unternehmen und Dienstleistern vorgenommen werden müssen. Nach der Ummeldung beim Einwohnermeldeamt beginnt erst die eigentliche Arbeit: Finanzamt, Krankenkasse, Banken, Versicherungen, Arbeitgeber, Energieversorger, Abonnements, Online-Dienste und viele weitere müssen einzeln kontaktiert werden. Eine zentrale Stelle, die diese Änderungen automatisch weiterleitet, existiert nicht.

Die Folgen dieser fehlenden Automatisierung sind vielfältig. Privatpersonen verpassen wichtige Post und Fristen und behalten kaum den Überblick darüber, welche Stellen bereits informiert wurden. Unternehmen büßen durch veraltete Adressen im Schnitt rund 13 Prozent ihrer Kundendatenbankeinträge an Zustellbarkeit ein; zugleich verursachen Postretouren direkte Zusatzkosten (Deutsche Post Direkt, 2025a).

Zwar existiert mit der kostenlosen Umzugsmitteilung der Deutschen Post (umziehen.de) ein Dienst, der über 1.000 Unternehmen ermöglicht, Adressen aus einer Umzugsdatenbank abzurufen und zu aktualisieren („Umzugsmitteilung | umziehen.de“, 2025). Doch der Prozess ist intransparent und passiv: Nutzer erfahren nicht, welche Unternehmen teilnehmen oder ob die Daten tatsächlich abgerufen wurden. Eine detaillierte Analyse dieser Schwächen folgt in Abschnitt 2.3.1.

Diese Masterarbeit setzt an genau diesem Spannungsfeld an: einem klaren Bedarf nach transparenter, zuverlässiger Adressverwaltung beim Umzug und erheblichen Lücken in der aktuellen Versorgung. Die Idee eines Postadresse-Änderungsdienstes ermöglicht es Umziehenden, ihre neue Adresse aktiv und nachvollziehbar an alle relevanten Vertragspartner weiterzuleiten.

1.2. Problemstellung und Forschungsfrage

Das beschriebene Problem der Adressänderung ist Symptom eines grundlegenderen Defizits: Persönliche Daten liegen verstreut in hunderten verschiedenen Systemen, über die Nutzer kaum Kontrolle haben. Dieser Zustand widerspricht den Prinzipien der Datensouveränität, die auch in der Datenschutz-Grundverordnung (DSGVO) verankert sind. Theoretisch wäre eine föderierte Lösung ideal – Nutzer verwalten ihre Daten zentral und autorisieren einzelne Dienste gezielt zum Zugriff.

Warum existiert eine solche Lösung nicht längst? Projekte wie das Sovrin Network (Sovrin.org, n. d.) oder cheqd (Edwards, 2026) haben genau solche Systeme entwickelt – und sind dennoch gescheitert oder kämpfen bis heute mit der Adoption. Die Analyse dieser Projekte in Kapitel 2.3 zeigt ein wiederkehrendes Muster: Technische Exzellenz allein reicht nicht aus. Diese Systeme waren auf universelle Partnerkooperation angewiesen (Hughes, 2024a), um überhaupt einen Nutzen zu bieten. Ohne teilnehmende Dienste kein Mehrwert für Nutzer – ohne Nutzer kein Anreiz für Dienste.

Hinzu kommt eine technische Realität: Die relevanten Dienste bieten fast ausnahmslos keine APIs für Adressänderungen an. Ein praxistauglicher Ansatz muss daher mit dieser Heterogenität umgehen können (vgl. Abschnitt 2.3).

Aus dieser Problemanalyse ergibt sich die zentrale Forschungsfrage:

> **Wie lässt sich eine Multi-Channel-Integrationsarchitektur für föderiertes Adressmanagement gestalten, die unabhängig vom Digitalisierungsgrad externer Dienste funktioniert?**

Zur Beantwortung werden drei Teilfragen untersucht:

Teilfrage 1 – Multi-Channel Web-Integration: Welche webbasierten Integrationsmuster eignen sich, um Adressaktualisierungen über verschiedene Kanäle (manuell, semi-automatisch, vollautomatisch) zu ermöglichen?

Teilfrage 2 – Browser-Extensions als Brückentechnologie: Wie können Browser-Extensions als semi-automatische Integrationsschicht dienen, um die Lücke zwischen manuellen Prozessen und API-basierten Lösungen zu schließen?

Teilfrage 3 – Inkrementelle Adoption: Wie muss ein solches System gestaltet sein, damit Nutzer sofort profitieren – auch wenn zunächst nur ein Bruchteil ihrer Dienste automatisiert integriert ist?

Diese Arbeit verfolgt einen pragmatischen Ansatz: Statt das gesamte Problem der digitalen Identität lösen zu wollen, dient das Adressmanagement als konkreter Anwendungsfall, an dem sich architektonische Muster demonstrieren und generalisieren

lassen. Der wissenschaftliche Beitrag sind drei Architekturprinzipien – Partnerunabhängigkeit durch abgestufte Kanalstrategie, zentraler Vertrauens-Provider und Graceful Degradation –, die über diesen Anwendungsfall hinaus auf föderierte Stammdaten-Architekturen übertragbar sind. Der Prototyp *SMove* dient als Validierungsinstanz; Partnerkooperation ist dabei *hinreichend*, aber *nicht notwendig* für unmittelbaren Nutzen.

1.3. Vorgehen und Aufbau

Zur Beantwortung der Forschungsfrage folgt diese Arbeit dem Design Science Research Framework nach Hevner et al. (2004) in Verbindung mit dem Prozessmodell von Peffers et al. (2007). Design Science Research (DSR) eignet sich hier, weil das Forschungsziel nicht die Erklärung eines Phänomens, sondern die Gestaltung eines nützlichen Artefakts ist – einer Architektur, die ein identifiziertes Praxisproblem löst und anschließend evaluiert wird.

Der Forschungsprozess verlief in drei iterativen Zyklen. Im **ersten Zyklus** wurden Problemraum und Anforderungen erschlossen: eine Analyse bestehender Ansätze sowie der rechtlichen und technischen Grundlagen (Kapitel 2), ein Realitätscheck zur API-Verfügbarkeit realer Dienste und die Ableitung funktionaler sowie nicht-funktionaler Anforderungen (Kapitel 3). Im **zweiten Zyklus** entstand das Architekturdesign mit den drei Integrationsstufen und dem Sicherheitskonzept (Kapitel 4); die prototypische Umsetzung sowie Rückkopplungen aus der Implementierung führten zu Anpassungen des Designs (Kapitel 5). Im **dritten Zyklus** wurde der Prototyp technisch durch automatisierte Tests und inhaltlich durch szenarienbasierte Analyse evaluiert (Kapitel 6). Kapitel 7 diskutiert Generalisierbarkeit, Limitierungen und offene Forschungsfragen; Kapitel 8 fasst die wesentlichen Erkenntnisse zusammen.

2. Grundlagen

2.1. Digitale Identitäten und das Konzept der Datensouveränität

Digitale Identitätssysteme lassen sich in drei Entwicklungsstufen einteilen:

1. *siloisierte Systeme*, bei denen jede Plattform eigene, voneinander unabhängige Nutzerkonten führt
2. *föderierte Ansätze* wie „Login mit Google“, die die Authentifizierung zentralisieren, das grundlegende Datengefälle zwischen Anbieter und Nutzer aber nicht auflösen
3. *Self-Sovereign Identity* (Self-Sovereign Identity (SSI)), bei der Nutzer die Kontrolle über ihre eigenen Identitätsdaten zurückbekommen, unabhängig von einer zentralen Institution (Christopher Allen, 2016; Satybaldy et al., 2024)

SSI basiert auf drei Kernelementen: dezentralen Identifikatoren (Decentralized Identifiers (DIDs)), verifizierbaren Nachweisen (Verifiable Credentials) und verteilten Registern als Vertrauensanker. Das Modell kennt drei Akteure: *Issuer* (stellt einen Nachweis aus), *Holder* (verwahrt ihn in einer Wallet) und *Verifier* (prüft ihn ohne Rückfrage beim Aussteller). Im Kontext der Adressverwaltung verspricht SSI damit genau das Wünschenswerte: Nutzer könnten eine verifizierte Adresse einmal speichern und gezielt an Dienste weitergeben. Warum dieser Ansatz in der Praxis dennoch nicht zum Durchbruch gekommen ist, zeigt das folgende Kapitel.

2.2. Anforderungen an sichere Datenverwaltung

Wenn Adressdaten zentral verwaltet und an Dritte weitergeleitet werden, ergeben sich unmittelbare Anforderungen an Sicherheit und Datenschutz. Das Prinzip der *Datensparsamkeit* (Data Minimization) verlangt, nur die für den jeweiligen Zweck notwendigen Daten zu erheben und zu speichern. *Zweckbindung* schließt eine Verwendung für andere Zwecke (etwa Marketing oder Profilbildung) aus. Nutzer müssen die volle Kontrolle darüber behalten, welcher Dienst welche Adresse erhält und wann. Jede Weitergabe erfordert eine explizite, bewusste Handlung des Nutzers.

Technisch sind gespeicherte Daten zu verschlüsseln und API-Endpunkte durch Authentifizierung und Autorisierung abzusichern. Die Browser-Extension als halbautomatische Integrationsschicht, die mit aktiven Nutzersessions fremder Webdienste interagiert, stellt besondere Sicherheitsanforderungen, die in Kapitel 4 detailliert behandelt werden.

2.3. Bestehende Ansätze – Überblick und zentrale Learnings

Bevor eigene Lösungsansätze entwickelt werden, lohnt ein Blick auf existierende Angebote. Dabei zeigen sich zwei grundlegend verschiedene Kategorien: zum einen technisch ambitionierte Projekte aus dem Bereich Self-Sovereign Identity, die das Problem der Datensouveränität grundlegend lösen wollten, aber größtenteils gescheitert sind oder mit massiven Adoptionsproblemen kämpfen (vgl. Abschnit Self-Sovereign Identity in der Praxis: Sovrin, Trinsic und Co). Zum anderen existieren praktische Lösungen, die sich bereits am Markt etabliert haben, aber nur Teilaspekte des Problems adressieren.

2.3.1. Praktische Lösungen am Markt

Nachsendeauftrag und Umzugsmitteilung der Deutschen Post

Der bekannteste Dienst für Umziehende ist der Nachsendeauftrag der Deutschen Post: Post an die alte Adresse wird bis zu 24 Monate weitergeleitet (Deutsche Post AG, n. d.). Er ist jedoch eine Übergangslösung, da die Adresse beim absendenden Unternehmen unverändert bleibt; das Problem wird aufgeschoben, nicht gelöst.

Ergänzend existiert die *Umzugsmitteilung*: Teilnehmende Unternehmen werden informiert, wenn ein Brief umgeleitet wird oder sie aktiv mit der alten Adresse anfragen. Der entscheidende Mangel dieses Ansatzes ist fehlende Transparenz und Kontrolle: Nutzer erfahren nicht, welche Unternehmen teilnehmen, ob ein Abruf stattfand oder welche Daten übermittelt wurden. Selektive Steuerung (bestimmte Dienstleister gezielt ein- oder auszuschließen) ist nicht möglich. Wer nicht teilnimmt oder nicht aktiv anfragt, erhält schlicht keine Benachrichtigung.

Umzugsportale und Checklisten

Eine zweite Kategorie praktischer Hilfestellungen bilden Umzugsportale und deren Checklisten. Webseiten wie umzug.de, myumzug.de, homeday.de oder wohnglueck.de bieten umfangreiche Informationen rund um den Umzug – inklusive Listen, wer über die Adressänderung informiert werden sollte (Homeday.de, n. d.; ImmobilienScout24, n. d.; Lisa Gutknecht, n. d.).

Die Analyse dieser Angebote zeigt ein durchgängiges Muster: Es handelt sich ausschließlich um statische Checklisten. Die Listen sind generisch und nicht auf individuelle Situationen zugeschnitten. Ein typischer Eintrag lautet etwa: „Arbeitgeber, Behörden (z.B. Kindergeldstelle), Versicherungen, Banken etc. über die Adressänderung informieren.“(Homeday.de, n. d.)

Bemerkenswert ist die Heterogenität dieser Listen: Jede Plattform nennt andere Stellen (Fitnessstudio, Streaming-Dienste, ADAC), weil kein allgemeingültiger Standard existiert, welche Dienstleister bei einem Umzug zu informieren sind. Die individuelle Dienste-Landschaft variiert stark je nach Lebenssituation. Vor allem aber bieten die Portale keine Unterstützung bei der tatsächlichen Durchführung: Der Nutzer steht trotz der allgemeinen Checklisten vor derselben Aufgabe wie zuvor.

2.3.2. Self-Sovereign Identity in der Praxis: Sovrin, Trinsic und Co.

Das bekannteste Beispiel gescheiterter SSI-Implementierung ist das Sovrin Network, das ab 2017 eine globale Infrastruktur für selbstsouveräne Identitäten aufzubauen versuchte. Die Foundation des Sovrin Network wurde aber im Januar 2026 aufgelöst, während das MainNet technisch weiterläuft (Curran, 2025, 2026). Trotz technischer Reife und internationaler Unterstützung blieb die Adoption weit hinter den Erwartungen zurück. Das Grundproblem ist ein klassisches Henne-Ei-Dilemma: SSI-Systeme benötigen gleichzeitig Aussteller, Nutzer und Verifier. Keine dieser Gruppen hat einen Anreiz, als Erste zu investieren (Kubach et al., 2020). Erschwerend kommen Usability-Defizite hinzu: Schlüsselverwaltung und Recovery-Mechanismen wie Seed-Phrasen erweisen sich im Massenmarkt als praktisch unbrauchbar (Hughes, 2024a). Rechtliche Unsicherheiten verschärfen das Problem: Die Unveränderlichkeit von Blockchain-Einträgen kollidiert mit dem DSGVO-Recht auf Löschung (Kubach et al., 2020).

Dieses Adoptionsdilemma beschränkt sich nicht auf Sovrin. Trinsic – ein führendes SSI-Startup – zog 2024 öffentlich Bilanz: „SSI didn’t work. Trinsic is pivoting.“ (Hughes, 2024a) und verkaufte seine SSI-Plattform. Auf europäischer Ebene verpflichtet electronic Identification, Authentication and trust Services (eIDAS) 2.0 die Mitgliedstaaten bis 2026 zur Bereitstellung einer Digital Identity Wallet (Europäisches Parlament,

2024), ein staatlich mandatierter Ansatz, dessen Breitenwirkung zum Zeitpunkt dieser Arbeit noch aussteht.

Hovi ist ein jüngeres SSI-Startup, das eine Identity-Infrastructure-as-a-Service-Plattform auf Basis verifizierbarer Credentials anbietet („Hovi - Identity Infrastructure For The Internet“, n. d.). Im Unterschied zu Sovrin setzt Hovi auf einen Software as a Service (SaaS)-Ansatz mit vereinfachter Integration für Unternehmen und vermeidet damit die Betriebskomplexität einer eigenen Blockchain-Infrastruktur. Das Adoptionsproblem besteht jedoch strukturell fort: Auch Hovi setzt voraus, dass Dienste das Credential-Ökosystem aktiv unterstützen, da Nutzer erst profitieren, wenn eine kritische Masse an Verifiern vorhanden ist („Wallets | Hovi Documentation“, n. d.).

Die zentralen Konsequenzen dieser Analyse für das Design von SMove werden in Kapitel 3.5 als konkrete Kompromisse ausgearbeitet.

2.3.3. KI-Browser-Agenten als aufkommende Kategorie

Eine jüngere Entwicklung, die erst nach den SSI-Projekten an Relevanz gewann, sind agentenbasierte Künstliche Intelligenz (KI)-Browser. OpenAI verfolgt mit seinen agentenbasierten Erweiterungen für ChatGPT einen Browser-Agenten, der eigenständig Webformulare ausfüllen, Bestellungen abschließen und mehrstufige Aufgaben ausführen kann (OpenAI, 2025). Ähnliche Ansätze verfolgen Perplexity Comet sowie Open-Source-Frameworks wie Browser Use („browser-use/browser-use“, 2026) und Skyvern („Skyvern-AI/skyvern“, 2026).

Diese Werkzeuge lösen das Formularausfüllproblem prinzipiell auf generische Weise: Ein Sprachmodell interpretiert das Document Object Model (DOM) einer beliebigen Seite und befüllt die Felder heuristisch. Für das spezifische Problem der strukturierten, nachvollziehbaren Adressaktualisierung bei einer definierten Menge eigener Dienste weisen sie jedoch charakteristische Lücken auf.

Strukturelle Unterschiede zu einem spezialisierten System

Für den Anwendungsfall der strukturierten, nachvollziehbaren Adressaktualisierung weisen KI-Agenten drei charakteristische Lücken auf. Erstens übertragen sie DOM-Inhalte und Seitenkontext (möglicherweise inklusive eingeloggter Sitzungsdaten) an externe Sprachmodell-APIs; ein spezialisiertes System überträgt ausschließlich vordefinierte Felder an konfigurierte Dienste. Zweitens fehlt ein strukturierter Audit-Trail: ob eine Änderung tatsächlich erfolgreich war, ist nicht systemseitig verifizierbar; ein explizites Consent-Modell pro Dienst, wie es die DSGVO verlangt (Europäisches

Parlament, 2016, Art. 13, 14), lässt sich mit einem autonomen Agenten nicht abbilden. Drittens operieren KI-Agenten ausschließlich im Webformular-Kontext; als stochastische Systeme produzieren sie bei identischer Eingabe nicht garantiert dasselbe Ergebnis, während dienstspezifische Mappings deterministisch sind und zusätzlich E-Mail- und API-Kanäle abdecken.

KI-Browser-Agenten und ein spezialisiertes Adressmanagementsystem sind damit weniger direkte Alternativen als komplementäre Ansätze mit unterschiedlichem Scope. Für wiederholbare, nachvollziehbare und DSGVO-konforme Adressaktualisierung als verwaltbare Infrastruktur liefern KI-Agenten keine strukturell äquivalente Lösung.

2.4. Multi-Channel-Integration als Architekturmuster

Der Kern dieser Arbeit ist kein Identitätssystem, sondern ein Integrationsproblem: Wie lassen sich strukturierte Daten zuverlässig, nachvollziehbar und kanalübergreifend an eine heterogene Menge externer Systeme propagieren? Diese Fragestellung ist in der Literatur zur Enterprise Application Integration (EAI) gut etabliert.

Hohpe und Woolf beschreiben in ihrem grundlegenden Werk zu Enterprise Integration Patterns den Kern des Problems: Heterogene Anwendungen können nicht direkt miteinander kommunizieren, weil sie jeweils andere Schnittstellen, Protokolle und Datenformate verwenden (Hohpe & Woolf, 2003). Das *Channel Adapter*-Muster adressiert genau dies: Ein Adapter greift auf die API oder die Daten einer Anwendung zu und übersetzt sie in kanalspezifische Nachrichten, ohne dass die Anwendung selbst Kenntnis vom Nachrichtensystem hat.

Zwei verwandte Muster sind für die Einordnung von SMove relevant, aber aus unterschiedlichen Gründen nicht direkt anwendbar.

Das *Pipes-and-Filters*-Muster beschreibt eine sequentielle Verarbeitungskette, in der jede Stufe eine Transformation an einer Nachricht vornimmt (Hohpe & Woolf, 2003). Es setzt voraus, dass die Nachricht in einem einheitlichen Format durch alle Stufen fließt, wobei die Ausgabe jedes Filters als Eingabe des nächsten dient. Bei SMove ist das nicht gegeben: Die Ausgabe für denselben Adressdatensatz ist je nach Kanal eine E-Mail-Textnachricht, ein Satz von CSS-Selektor-Zielwerten oder ein API-Payload. Es handelt sich um keine Transformationen desselben Formats, sondern um grundlegend verschiedene Repräsentationen für nicht-kooperative Zielsysteme, worunter hier Systeme verstanden werden, die keine maschinenlesbare Schnittstelle bereitstellen und daher per E-Mail oder Browser-Automation erreicht werden. Hinzu kommt, dass die Kanalauswahl nicht statisch ist, sondern zur Laufzeit aus der Konfiguration des jeweiligen Dienstes abgeleitet wird.

Das *Message Broker*-Muster zentralisiert die Routing-Logik in einem Intermediär, der eingehende Nachrichten an registrierte Empfänger weiterleitet (Hohpe & Woolf, 2003). Es setzt voraus, dass Zielsysteme als Subscriber aktiv am Nachrichtenkanal teilnehmen und die Nachrichten entgegennehmen. Bei SMove trifft das nur für Level-3-Dienste zu; die Mehrheit der Zielsysteme ist passiver Empfänger ohne jede Broker-Infrastruktur. Die *Notification Engine* übernimmt dennoch eine abgewandelte Broker-Funktion auf der sendenden Seite: Sie leitet Ausführungsaufträge an die jeweils passende Strategie weiter und spricht eine Kanalempfehlung aus, die endgültige Entscheidung verbleibt jedoch beim Nutzer.

SMove setzt das Channel-Adapter-Prinzip um, weicht aber in zwei Punkten von der klassischen EAI-Literatur ab. Erstens behandelt der klassische Channel Adapter Systeme ohne maschinenlesbare Schnittstelle als Sonderfall oder schließt sie aus. SMove erhebt Browser-Automation zur gleichrangigen Integrationsstufe und definiert dafür eine eigene Ausführungsstrategie. Zweitens ist im klassischen Modell der Kanal pro Adapter/Partnerdienst fix. SMove wählt den Kanal dynamisch anhand der zur Laufzeit verfügbaren Konfiguration.

Die Kombination aus dynamischer Kanalwahl und der Unterstützung heterogener, teils nicht-kooperativer Zielsysteme stellt die wesentliche Erweiterung gegenüber dem klassischen Channel-Adapter-Muster dar und bildet den architektonischen Kern des wissenschaftlichen Beitrags dieser Arbeit.

Die beschriebene Architektur muss auch unter Ausfallbedingungen robust bleiben. Tanenbaum und Van Steen formulieren als zentrales Designziel für verteilte Systeme, dass partielle Ausfälle die Gesamtfunktionalität nicht wesentlich beeinträchtigen sollen (Tanenbaum & Van Steen, 2007). SMove übernimmt dieses Prinzip auf Kanalebene: Schlägt ein Übermittlungsversuch fehl, macht das System Handlungsbedarf explizit sichtbar und bietet alternative Ausführungswege an, statt stumm abzubrechen.

3. Anforderungsanalyse

Die Grundlagen in Kapitel 2 haben gezeigt, dass bestehende Ansätze im Bereich digitaler Identitätsverwaltung entweder an Partnerabhängigkeit scheitern oder Datenschutzanforderungen unzureichend adressieren. Daraus ergibt sich die Frage, welche konkreten Anforderungen ein praxistaugliches System erfüllen muss – unabhängig von externer Kooperation und unter Wahrung der DSGVO. Dieses Kapitel leitet diese Anforderungen systematisch aus dem Problemraum ab.

3.1. Problemraum im Detail

3.1.1. Nutzergruppen und Problemszenarien

Im Rahmen einer vorgelagerten Requirements-Engineering-Analyse wurden Personas und Szenarien für die relevanten Nutzergruppen entwickelt (Holländer et al., 2025). Allen gemeinsam ist dasselbe Grundproblem: Es gibt keinen zentralen Ort, an dem Adressänderungen gesammelt veranlasst werden können. Stattdessen muss jedes Dienstportal einzeln besucht, jede E-Mail separat verfasst und jedes Formular manuell ausgefüllt werden, wobei stets das Risiko besteht, Dienste zu vergessen, bei denen die letzte Korrespondenz Jahre zurückliegt. Je nach Lebenssituation (technikaffin oder nicht, einmalig oder häufig Umziehender) variiert der Aufwand erheblich. Die beiden zentralen Treiber bleiben jedoch überall gleich: repetitiver Aufwand und fehlende Vollständigkeit.

3.1.2. Pain Points aus Nutzersicht

Aus den beschriebenen Szenarien und der Marktanalyse bestehender Lösungen (vgl. Abschnitt 2.3.1) lassen sich fünf zentrale Pain Points ableiten:

1. Unübersichtlichkeit: Kaum jemand hat einen vollständigen Überblick, welche Dienste die eigene Adresse gespeichert haben – die individuelle Dienste-Landschaft variiert stark je nach Lebenssituation.

2. Repetitive manuelle Arbeit: Dieselben Daten müssen für 15 bis 30 Dienste immer wieder neu eingegeben werden: per Telefon, E-Mail, Webformular oder Brief.

3. Fehlende selektive Kontrolle: Bestehende Lösungen bieten keine Möglichkeit, gezielt auszuwählen, welche Unternehmen die neue Adresse erhalten (vgl. Abschnitt 2.3.1).

4. Zeitdruck und Vergessen: Adressänderungen werden im Umzugsstress aufgeschoben. Konsequenzen zeigen sich erst später durch verpasste Post oder Mahnungen.

5. Kein Feedback: Es gibt keine Bestätigung, dass eine Änderung tatsächlich durchgeführt wurde; Probleme werden erst bemerkt, wenn etwas schiefgeht.

3.1.3. Problemdimension und Relevanz

Mit **7,81 Millionen** Umzügen jährlich in Deutschland (Deutsche Post Adress GmbH & Co. KG, 2024, S. 6) und einem erheblichen manuellen Aufwand pro Umzug betrifft das Problem einen signifikanten Teil der Bevölkerung. Der Nachsendeauftrag der Deutschen Post (6 Monate: 31,90 €) schiebt es lediglich auf.

Ein praxistauglicher Lösungsansatz muss vier Dimensionen adressieren: **Vollständigkeit** (kein Dienst darf vergessen werden), **Effizienz** (viele Änderungen mit wenigen Klicks statt separater Logins), **zeitliche Koordination** (unterschiedliche Gültigkeitsdaten je Dienst) sowie **Transparenz und Kontrolle** (Nutzer wissen jederzeit, welche Daten wohin übermittelt wurden).

Das folgende Kapitel untersucht, welche Integrationsmöglichkeiten externe Dienste tatsächlich anbieten, da dies eine notwendige Grundlage bildet, um einzuschätzen, wie realistisch die Vision der „wenigen Klicks“ umsetzbar ist.

3.2. Realitätscheck: Was externe Dienste wirklich können

Eine zentrale Annahme vieler Systeme im Bereich des Adress- und Identitätsmanagements ist, dass externe Dienste programmatische Schnittstellen anbieten. Der Realitätscheck zeigt, dass diese Annahme für die überwältigende Mehrheit der für Privatpersonen relevanten Dienste nicht zutrifft.

Banken stellen zwar Online-Banking-APIs bereit, etwa im Rahmen der Payment Services Directive 2 (PSD2)-Richtlinie. Diese beschränken sich jedoch auf Kontoinformationen und Zahlungsinitiierungen (Bank-Verlag GmbH, n. d.). Adressänderungen werden ausnahmslos über authentifizierte Webformulare abgewickelt. Versicherungen, Telekommunikationsanbieter und Energieversorger bieten keine öffentlichen APIs

für Stammdaten. Der Standardweg ist das Kundenportal oder E-Mail, teils ohne standardisiertes Format. Große Plattformen wie Amazon oder Netflix erlauben Adressänderungen ausschließlich über ihre eigenen Webportale. Kleinere Online-Shops auf Basis von Shopify oder WooCommerce stellen hingegen APIs bereit, über die autorisierte Partner Versand- und Rechnungsadressen programmatisch aktualisieren können (Shopify, n. d.; Woo, n. d.). Diese Schnittstellen sind jedoch für Shop-Betreiber und deren Partner gedacht, nicht für Endnutzer zur eigenständigen Adresspflege. Eine Übersicht der untersuchten Dienste mit den jeweils vorgefundenen Kanälen findet sich in Anhang A.2 (Tab. A.2).

Schlussfolgerung: Für nahezu keinen der für Privatpersonen relevanten Dienste existiert eine öffentliche API zur programmatischen Adressaktualisierung. Der realistische Integrationsweg ist in fast allen Fällen ein Webformular oder eine E-Mail. Diese Erkenntnis begründet, warum SMove den Schwerpunkt auf Level 1 (E-Mail) und Level 2 (Browser-Extension) legt und Level 3 (API) nur als konzeptionellen Erweiterungspfad behandelt.

3.3. Funktionale Anforderungen

Die funktionalen Anforderungen wurden im Rahmen einer vorgelagerten Requirements-Engineering-Analyse erhoben (Holländer et al., 2025).

3.3.1. Nutzer- und Dienstverwaltung

Der Nutzer authentifiziert sich, gibt seine neue Adresse samt Umzugsdatum ein und erhält eine Validierung auf Vollständigkeit. Dieses System stellt eine durchsuchbare, filterbare Liste verfügbarer Dienste bereit. Nutzer können eigene Dienste anlegen und die verfügbaren Übermittlungskanäle konfigurieren. Damit ist das System auch für nicht offiziell integrierte Dienste nutzbar.

3.3.2. Auswahl und Einwilligung

Der Nutzer wählt die zu informierenden Dienste aus; das System zeigt eine Vorschau mit Übermittlungskanal. Für jeden Dienst ist eine explizite Einwilligung erforderlich. Erteilte Einwilligungen sind jederzeit einsehbar und einzeln oder gesammelt widerrufbar.

3.3.3. Ausführung, Status und Fehlerbehandlung

Eine einmalige Bestätigung löst Updates an alle ausgewählten Dienste aus. Je nach Kanal erfolgt die Übermittlung automatisch (API), semi-automatisch (Browser-Extension füllt Formular, Nutzer sendet ab) oder manuell unterstützt (vorausgefüllte E-Mail-Vorlage). Ein Dashboard zeigt pro Dienst den Zustellstatus sowie Zeitstempel und Übertragungskanal. Das System unterscheidet zwischen temporären Fehlern (automatische Wiederholung) und dauerhaften Fehlern (Nutzerbenachrichtigung mit Handlungsoptionen). Fehlgeschlagene Übermittlungen können manuell neu angestoßen oder auf einen alternativen Kanal umgeleitet werden.

3.4. Nicht-funktionale Anforderungen

Tabelle 3.1 fasst die wesentlichen Qualitätsanforderungen zusammen; die zugehörigen Designentscheidungen sind in Abschnitt 3.5 ausgeführt.

Tabelle 3.1.: Nicht-funktionale Anforderungen

Kategorie	**Anforderung**
Sicherheit	Verschlüsselte Übertragung (HTTPS/Transport Layer Security (TLS)); Adressfelder serverseitig verschlüsselt; Audit-Log aller sicherheitsrelevanten Ereignisse
Datenschutz	DSGVO-konform: Einwilligung pro Dienst, Datenminimierung, Transparenz, Recht auf vollständige Löschung, Zweckbindung
Benutzerfreundlichkeit	Zero-Setup-Nutzung ohne Einarbeitung; minimale Schritte von Anmeldung bis abgeschlossener Änderung; verständliche Statusrückmeldungen; Mobilfähigkeit
Erweiterbarkeit	Neue Dienste und Integrationskanäle ohne Änderung der Kernlogik integrierbar; Strategy-Pattern für Ausführungsstrategien

3.5. Scope und Priorisierung

Die folgenden Abschnitte dokumentieren, welche Funktionen im Rahmen des Prototyps umgesetzt wurden und wie die Priorisierung begründet ist. Architektonische

Grundsatzentscheidungen wie die Wahl einer zentralen Architektur oder das Sicherheitskonzept sind in den Kapiteln 4.1.2 und 4.2 beschrieben.

3.5.1. Feature-Reichhaltigkeit vs. Fokus

SMove begrenzt den Scope bewusst auf einen einzigen Anwendungsfall: die Verwaltung und Aktualisierung von Postadressen im Kontext eines Umzugs. Dieses Fokusprinzip folgt der aus der SSI-Praxis abgeleiteten Erkenntnis, dass verwertbarer Produktnutzen vor technischer Breite stehen muss (Hughes, 2024a). Die Architektur ist dabei so gestaltet, dass eine spätere Erweiterung auf weitere Datentypen möglich ist (vgl. Abschnitt 7.1), für den Prototyp aber kein Implementierungsziel darstellt.

3.5.2. Priorisierung für MVP

Der Vorrang von Produktnutzen vor technischer Breite (Hughes, 2024a) bestimmt auch die Priorisierung des Prototyps. Das Minimum Viable Product (MVP) konzentriert sich auf den kürzesten Pfad zu einem erlebbaren Nutzen:

Must-Have (Prototyp): Adress- und Dienstverwaltung, Umzugs-Workflow mit automatischer Erkennung betroffener Dienste, E-Mail-basierte Adressänderung (automatisch und manuell), Statusverfolgung und Bestätigungs-Workflow.

Should-Have (Prototyp, wenn zeitlich möglich): Browser-Extension für Level-2-Integration.

Could-Have (konzeptionell ausgearbeitet): API-Integration (Level 3) als Proof-of-Concept, erweitertes Consent-Management.

Won't-Have (außerhalb des Prototyps): Produktionsreife API-Adapter für reale Dienste, mobile App, Verwaltung anderer Datentypen (Kontaktdaten, Zahlungsinformationen), BundID-Anbindung.

Diese Priorisierung stellt sicher, dass der Prototyp den vollständigen Level-1-Flow demonstriert und damit den zentralen wissenschaftlichen Beitrag (die partnerunabhängige Integrationsstrategie) erlebbar macht.

4. Architekturdesign

4.1. Designprinzipien

Die folgenden Prinzipien liegen dem Gesamtsystem zugrunde und sind nicht als abstrakte Leitlinien zu verstehen, sondern als konkrete Anforderungen, die sich direkt auf Architektur, Datenmodell und Integrationsstrategie auswirken.

4.1.1. Partnerunabhängigkeit

Das in Kapitel 2.3 dokumentierte Adoptionsdilemma (kein Mehrwert ohne teilnehmende Dienste, kein Anreiz ohne Mehrwert) war der entscheidende Faktor beim Scheitern von Sovrin, Trinsic und vergleichbaren Projekten (Curran, 2025; Hughes, 2024a).

SMove begegnet diesem Problem durch ein anderes Grundprinzip: Das System muss unmittelbaren Nutzen liefern, auch wenn kein einziger externer Dienst kooperiert. Alle weitergehenden Integrationsstufen bauen auf dieser Basis auf und stellen Erweiterungen dar, keine Voraussetzungen.

4.1.2. Zentralisierung vs. Dezentralisierung

SSI-Projekte wie Sovrin scheiterten nicht an mangelnder technischer Eleganz, sondern an der Komplexität, die eine voll dezentrale Architektur für alle Beteiligten erzeugt (Curran, 2025). SMove trifft daher eine bewusste Gegenwahl: eine relationale Datenbank mit serverseitiger Verschlüsselung sensibler Felder. Der entscheidende strukturelle Vorteil liegt nicht in der Einfachheit, sondern in der Rolle, die eine zentrale Architektur ermöglicht. SMove agiert gegenüber externen Diensten als *Vertrauens-Provider*. Bei einem dezentralen Modell müsste sich jeder Nutzer bei jedem Dienst separat authentifizieren und legitimieren. Die zentrale Architektur übernimmt diese Rolle stellvertretend. Nutzerkontrolle wird dabei nicht durch dezentrale Datenhaltung erreicht, sondern durch Consent-Management, einen vollständigen Audit-Log und das Recht auf vollständige Datenlöschung.

4.1.3. Privacy-by-Design

Postadressen sind im Sinne der Datenschutz-Grundverordnung personenbezogene Daten. Das bedeutet für die Architektur konkret: Datenminimierung (es werden ausschließlich die Informationen gespeichert, die für den jeweiligen Zweck notwendig sind), Zweckbindung (Adressdaten eines Nutzers werden nicht ohne explizite Freigabe an Dritte weitergegeben) sowie vollständige Löschbarkeit aller nutzerbezogenen Daten (Europäisches Parlament, 2016, Artikel 5). Diese Anforderungen beeinflussen nicht nur den Betrieb, sondern bereits das Datenmodell selbst, insbesondere die Modellierung von Freigaben und den Audit-Log.

4.1.4. Selektiver API-Einsatz

Das System exponiert zwei externe Schnittstellen: eine Representational State Transfer (REST)-API und eine Graph Query Language (GraphQL)-API. Die Webanwendung kommuniziert über eigene Inertia.js-Controller-Endpunkte; die externen APIs sind für Clients vorgesehen, die eine tokenbasierte, zustandslose Kommunikation benötigen. Die Browser-Extension nutzt dabei die GraphQL-API; das Angebot beider API-Stile senkt die Integrationsschwelle für potenzielle Partner, die eigene Präferenzen mitbringen.

4.1.5. Pragmatismus über Idealismus

Die Architektur verzichtet bewusst auf Blockchain-Technologie und ein dezentrales Speichermodell. Für den vorliegenden Anwendungsfall bringt Dezentralisierung keinen nachweisbaren Mehrwert für den Nutzer, erhöht jedoch die Komplexität erheblich. Bereits cheqd, technisch deutlich ausgereifter als das Sovrin-Network, kämpft nach eigener Aussage mit einer schleppenden Adoption (Edwards, 2026). Statt Dezentralisierung als Selbstzweck zu verfolgen, wird echte Nutzerkontrolle durch Consent-Management und einen vollständigen Audit-Log realisiert. Der Nutzer sieht jederzeit, welche Daten wann an wen übermittelt wurden, und kann einzelne Freigaben gezielt widerrufen.

4.2. Sicherheitskonzept

Da Postadressen personenbezogene Daten im Sinne der DSGVO sind (vgl. Abschnitt 4.1.3), muss das System konkret benennen, gegen welche Risiken es schützt. Ausgangspunkt ist ein Bedrohungsmodell, aus dem sich die Designentscheidungen zu Datenhaltung, Authentifizierung und Autorisierung direkt ableiten und das als Grundlage für die Systemarchitektur in Abschnitt 4.1 dient.

4.2.1. Bedrohungsmodell

Für SMove sind drei konkrete Bedrohungsszenarien relevant:

Direkter Datenbankzugriff: Datenbankzugang allein darf keinen vollständigen Zugriff auf Adressdaten im Klartext liefern.

Horizontale Privilege-Escalation über die API: Ein authentifizierter Nutzer oder kompromittierter Client darf ausschließlich auf eigene Ressourcen zugreifen. Das Manipulieren von IDs in API-Anfragen muss auf Anwendungsebene ausgeschlossen sein.

Mangelnde Transparenz über Datenverarbeitung: Nutzer müssen nachvollziehen können, welche Änderungen wann an welche Dienste übermittelt wurden. Vertrauen entsteht durch Transparenz und Nutzerkontrolle, nicht durch die technische Unmöglichkeit serverseitigen Zugriffs.

4.2.2. Datenhaltung und Verschlüsselung

Dem ersten Bedrohungsszenario wird durch serverseitige Feldverschlüsselung begegnet. Alle direkt personenbezogenen Adressfelder werden mit Advanced Encryption Standard (AES)-256-CBC verschlüsselt gespeichert, sodass Datenbankzugang allein keine verwertbaren Klartextdaten liefert. Die konkrete Umsetzung ist in Abschnitt 5.2.1 beschrieben.

Eine clientseitige Verschlüsselung (End-to-End) hätte die asynchrone Verarbeitung durch die Notification Engine unmöglich gemacht, da der Server die Adressdaten im Klartext benötigt, um E-Mails zu generieren oder API-Calls durchzuführen. Der gewählte Kompromiss schützt die Daten vor direktem Datenbankzugriff, ermöglicht aber die serverseitige Verarbeitung.

4.2.3. Authentifizierung

SMove bedient zwei grundlegend verschiedene Client-Typen mit unterschiedlichen Authentifizierungsanforderungen. Die Webanwendung nutzt Session-basierte Authentifizierung mit Cross-Site Request Forgery (CSRF)-Schutz; Zwei-Faktor-Authentifizierung ist verfügbar, aber optional, da Adressdaten keine hochsensiblen Finanzdaten sind. Die Browser-Extension ist ein eigenständiger Client ohne gemeinsamen Session-Kontext und benötigt daher eine tokenbasierte, zustandslose Kommunikation. Ein reines Passwort-Login für die Extension hätte Credentials dauerhaft im Extension-Storage exponiert; Open Authorization (OAuth) 2.0 mit Proof Key for Code Exchange (PKCE) (Aaron

Parecki et al., n. d.; Sakimura et al., 2015) vermeidet dies, indem ausschließlich kurzlebige Token gespeichert werden (Lodderstedt et al., 2025). Die konkrete Implementierung des OAuth-Flows ist in Abschnitt 5.2.3 beschrieben.

4.2.4. Ressourcen-basierte Autorisierung

Authentifizierung allein verhindert keine horizontale Privilege-Escalation. Jeder Datenzugriff wird deshalb zusätzlich auf Ressourcenebene autorisiert: Eine dedizierte Policy pro Datenmodell stellt sicher, dass Nutzer ausschließlich auf eigene Ressourcen zugreifen können, unabhängig davon, ob der Zugriff über die Webanwendung, die REST-API oder die GraphQL-API erfolgt. Die Implementierung über Laravel Policies ist in Abschnitt 5.2.1 beschrieben.

4.2.5. Audit-Logging

Transparenz über ausgeführte Datenoperationen ist sowohl für die Nutzerkontrolle als auch für die Nachvollziehbarkeit im Sinne der DSGVO (Europäisches Parlament, 2016, Artikel 5) relevant. SMove implementiert ein ereignisbasiertes Audit-Log: Adressänderungen, Token-Ausstellungen und -Widerrufe werden persistent protokolliert.

Jeder Eintrag enthält Zeitstempel, Nutzer-ID, Ereignistyp, betroffene Ressource und genutzten Integrationskanal. Bewusst nicht geloggt wird der Adressinhalt selbst. Der Log dokumentiert *dass* eine Änderung ausgelöst wurde, nicht *welche konkreten Daten* übermittelt wurden. Das vermeidet redundante Speicherung sensibler Daten und hält den Log datenschutzkonform.

Die beschriebenen Maßnahmen (z.B. Feldverschlüsselung, ressourcenbasierte Autorisierung und ereignisbasiertes Audit-Logging) erfüllen zugleich die datenschutzrechtlichen Anforderungen der DSGVO. Datenminimierung, Zweckbindung und das Recht auf vollständige Löschung sind durch Architektur und Datenmodell sichergestellt (Europäisches Parlament, 2016).

4.3. Systemkomponenten

Das System gliedert sich in drei funktionale Kernmodule, die über eine gemeinsame Datenbankschicht miteinander kommunizieren. Abbildung 4.1 veranschaulicht den Gesamtaufbau.

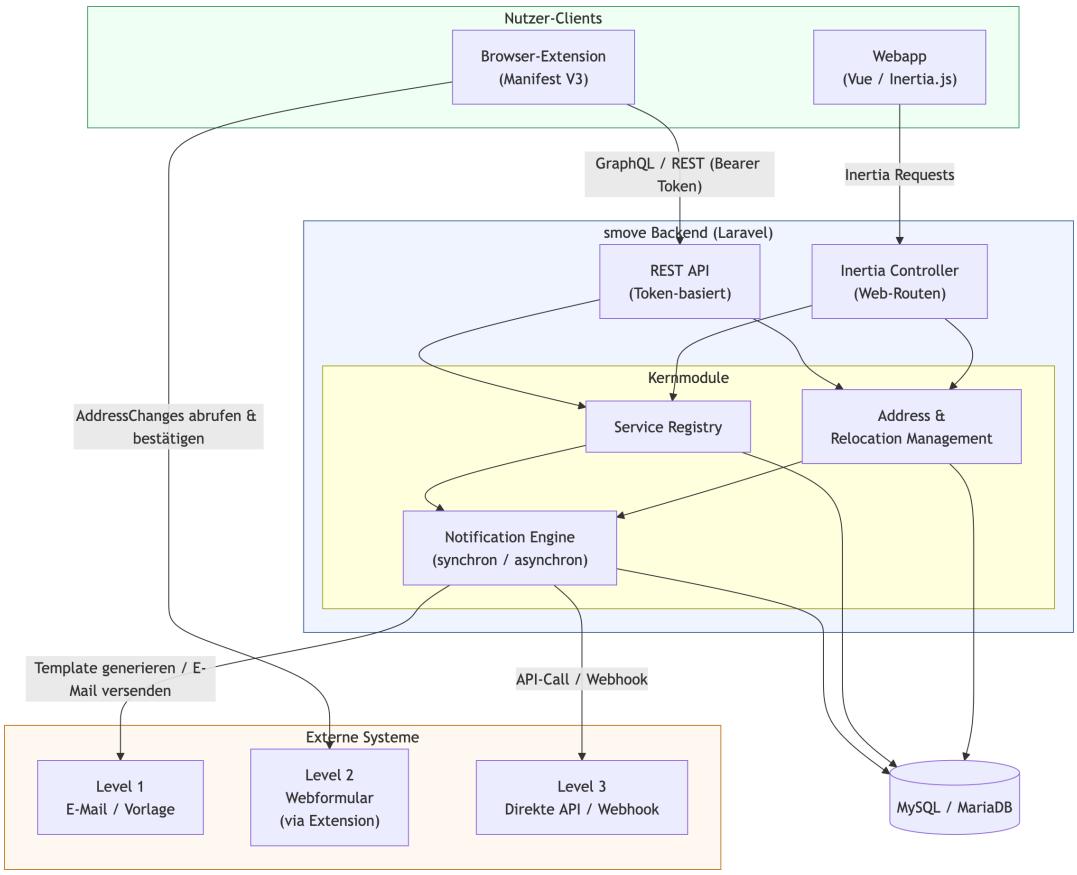


Abbildung 4.1.: Komponentendiagramm: Gesamtarchitektur von SMove

4.3.1. Address & Relocation Management

Dieses Modul bildet den Kern des Systems. Es verwaltet die Adressen eines Nutzers, wobei jede Adresse aus strukturierten Feldern besteht (Land, Bundesland, Stadt, Postleitzahl, Straße, Hausnummer) und vom Nutzer mit einem frei wählbaren Label versehen werden kann.

Neben der reinen Adressverwaltung koordiniert das Modul Umzugsvorgänge (*Relocations*) und die daraus entstehenden einzelnen Adressänderungen pro Dienst (*AddressChanges*). Ein *AddressChange* wurde dabei bewusst als eigenständige Entität modelliert: Er kann im Kontext eines Umzugs entstehen, ist aber nicht zwingend an einen solchen gebunden. Das erlaubt es Nutzern, auch außerhalb eines realen Umzugs einzelne Dienste nachzupflegen, beispielsweise wenn ein Dienst bei einem zurückliegenden Umzug vergessen wurde.

4.3.2. Service Registry

Die Service Registry ist ein kuratierter Katalog externer Dienste, mit denen Nutzer ihre Adressen verknüpfen können. Welche Adressfelder ein bestimmter Dienst benötigt, wird über sogenannte *ServiceDataRequirements* konfiguriert; pro Dienst und Feld ist hinterlegt, ob es verpflichtend oder optional ist und zu welchem Zweck es benötigt wird. Die *ServiceDataRequirements* bilden die Grundlage für den Vollständigkeitscheck, der beim Verknüpfen eines Dienstes ausgeführt wird; fehlt ein Pflichtfeld in der zugeordneten Adresse, wird der Nutzer unmittelbar darauf hingewiesen, bevor die Verknüpfung gespeichert wird. Zusätzlich enthält die Registry typisierte Kanalkonfigurationen pro Dienst – etwa eine *ServiceEmailConfig* mit der Empfänger-E-Mail-Adresse des Kundenservices oder eine *ServiceWebsiteConfig* mit URL-Mustern und Feldmappings für die Browser-Extension. Diese typisierten Konfigurationen bestimmen, welche Ausführungsstrategien für einen Dienst verfügbar sind.

4.3.3. Notification Engine

Die Notification Engine ist für die eigentliche Ausführung der Adressänderungen zuständig. Die Ausführung unterscheidet sich je nach Interaktionsmodell: Vom Nutzer direkt ausgelöste Strategien werden synchron verarbeitet; hintergrundfähige Aufträge (etwa geplante oder automatisch wiederholte Ausführungen) werden als asynchrone Jobs in eine Queue eingereiht. Pro Kanal existieren mehrere *Ausführungsstrategien*, die sich im Automatisierungsgrad unterscheiden: automatischer E-Mail-Versand (`AutoSendEmailStrategy`), manuelles Kopieren einer Vorlage (`TemplateCopyManualEmailStrategy`), Öffnen der Dienst-Webseite mit Formularunterstützung (`OpenWebsiteStrategy`) oder direkte API-Calls. Die Verfügbarkeit einer Strategie ergibt sich zur Laufzeit aus der vorhandenen Kanalkonfiguration des Dienstes.

Schlägt eine Ausführung fehl (etwa weil eine API nicht erreichbar ist), greift ein Retry-Mechanismus mit exponentiellem Backoff. Nach Erschöpfung der Versuche wird der Nutzer aktiv über die Situation informiert und erhält die Möglichkeit, die Änderung auf einem anderen Weg abzuschließen. Dieses Fallback-Verhalten ist ein wesentlicher Unterschied zu Ansätzen, die stumm fehlschlagen und den Nutzer im Unklaren lassen.

4.3.4. Datenmodell

Das relationale Datenmodell (MySQL/MariaDB) spiegelt die fachliche Struktur direkt wider; das vollständige Schema und thematische Ausschnitte finden sich in den Entity-Relationship (ER)-Diagrammen im Anhang (Abbildungen A.5–A.9).

Drei Designentscheidungen prägen das Modell besonders. Erstens sind Kanalkonfigurationen typisiert (`service_email_configs`, `service_website_configs`) statt als generisches JSON gespeichert; ob automatischer E-Mail-Versand möglich ist, ergibt sich zur Laufzeit aus dem Vorhandensein einer `ServiceEmailConfig`. Es gibt kein Boolean-Flag und keine implizite Konvention. Zweitens ist `scheduled_at` auf Ebene der *address_changes* verankert, nicht auf Ebene der *Relocation*, sodass verschiedene Dienste desselben Umzugs zu unterschiedlichen Zeitpunkten benachrichtigt werden können. Drittens wird der Aggregatstatus einer *Relocation* gecacht und über einen Eloquent Observer aktualisiert, wodurch Konsistenzgarantie mit Abfrageeffizienz verbunden wird, da der Status direkt filterbar ist, ohne die zugehörigen *AddressChanges* laden zu müssen.

4.3.5. Prozessablauf einer Adressänderung

Das Datenmodell beschreibt die beteiligten Entitäten. Der folgende Abschnitt zeigt, wie diese im Rahmen einer konkreten Adressänderung zusammenspielen. Abbildung 4.2 stellt den Ablauf abstrakt dar, unabhängig vom gewählten Integrationskanal.

Der **Flow-Status** beschreibt den Zustand eines konkreten *AddressChangeFlow*; der AddressChange-Status (**AC-Status**) wird daraus über einen Observer abgeleitet. Aus dem Anlegen einer *Relocation* entstehen für jeden verknüpften Dienst *AddressChanges* im Status `planned`. Die Strategiewahl startet einen Flow (`created` → `prepared` → `in_progress`); nach Ausführung ist immer eine Bestätigung erforderlich: entweder direkt durch den Nutzer, automatisch nach einem API-Aufruf oder über eine `ConfirmationChallenge` als externen E-Mail-Link (`awaiting_confirmation` → `confirmed`, AC: `completed`). Erschöpfte Retry-Versuche setzen den Flow auf `failed` und den *AddressChange* auf `needs_attention`; der Nutzer kann einen neuen Flow mit alternativer Strategie starten, ohne die *Relocation* zurückzusetzen.

Abbildung 4.3 zeigt die resultierende neunstufige Statusmaschine mit allen Zuständen und Transitionen beider Entitäten.

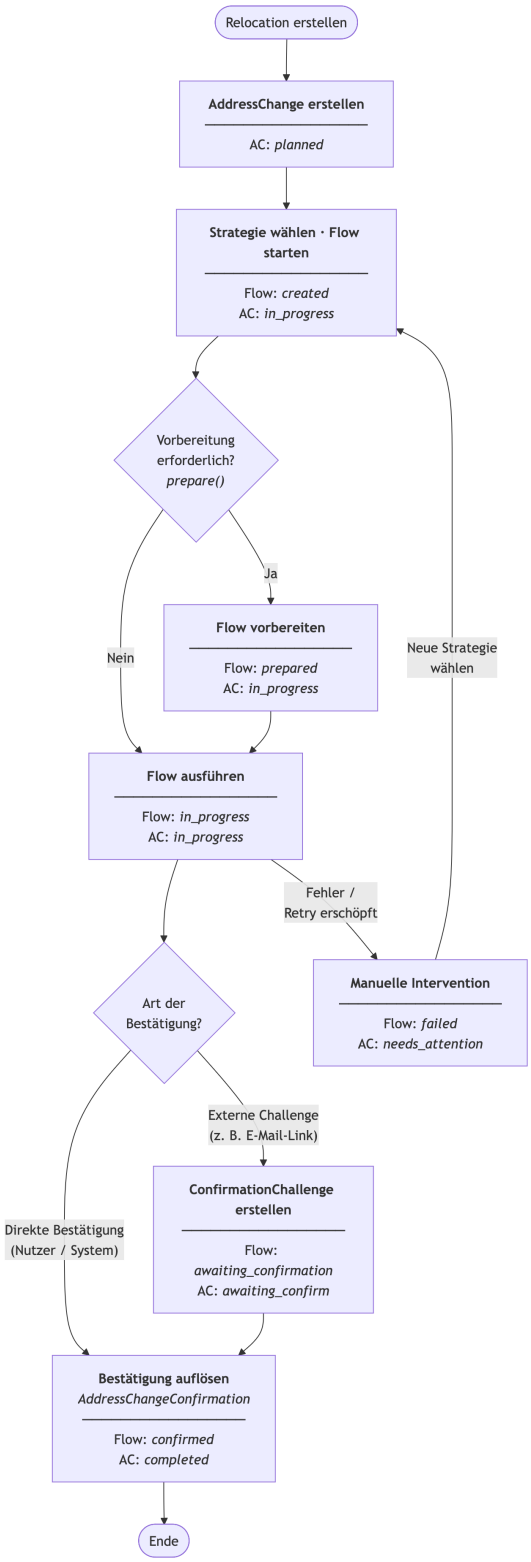


Abbildung 4.2.: Prozessablauf einer Adressänderung (kanalunabhängig)

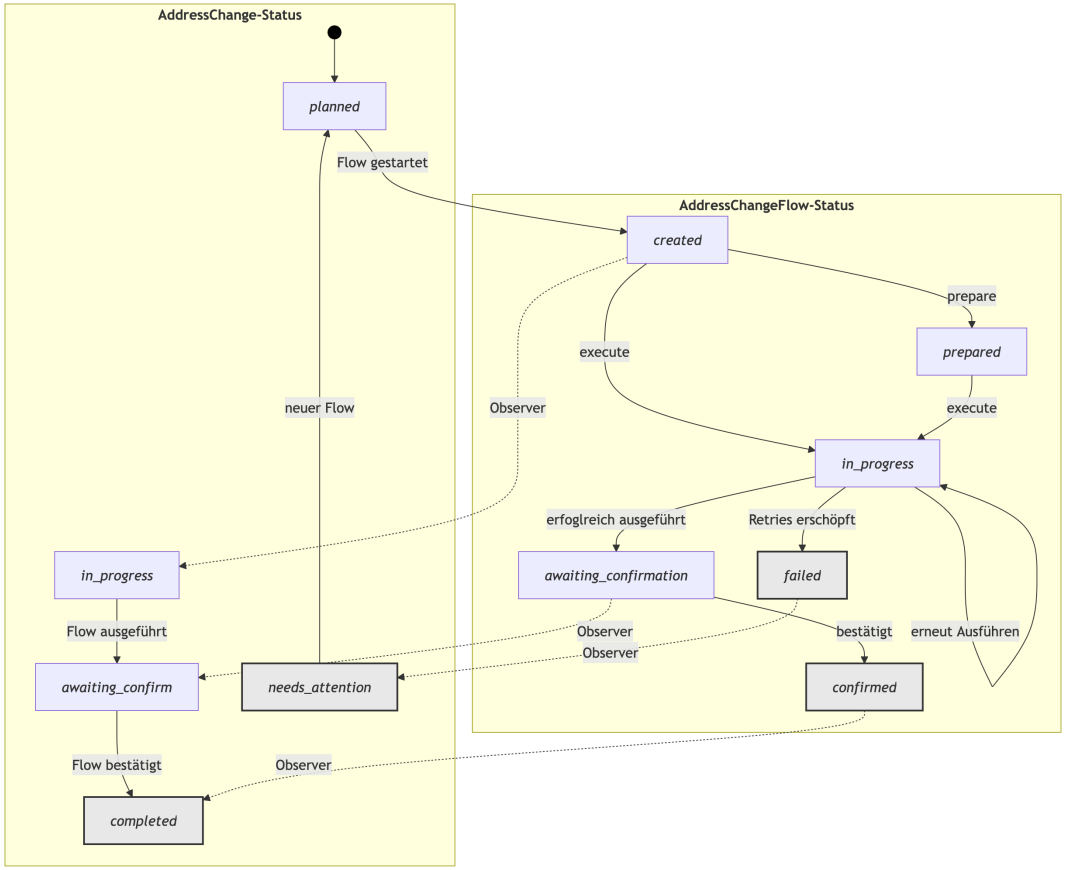


Abbildung 4.3.: Neunstufige Statusmaschine: *AddressChange*-Status und *AddressChangeFlow*-Status mit Observer-gesteuerten Übergängen (gestrichelt)

4.4. Multi-Channel Integrationsstrategie

Der entscheidende wissenschaftliche Beitrag dieser Arbeit liegt in der Konzeption einer Integrationsstrategie, die explizit auf Partnerunabhängigkeit ausgelegt ist und unmittelbaren Nutzen auf jeder Stufe liefert, unabhängig davon, welche Integrationsstufe für einen bestimmten Dienst verfügbar ist.

4.4.1. Level 1: Manuelle Benachrichtigungen

Level 1 ist die Basisstufe und funktioniert ohne jegliche Kooperation externer Dienste. Für Dienste, bei denen eine E-Mail-Konfiguration hinterlegt ist, generiert das System eine service-spezifische Vorlage: wahlweise als automatisch versendete E-Mail oder als vorformatierter Entwurf, den der Nutzer selbst versendet.

Der zentrale Mehrwert liegt weniger in der Zeitersparnis als in der *Vollständigkeit*. Das System stellt sicher, dass kein Dienst vergessen wird, da das Vergessen einzelner Dienste zu den häufigsten Ursachen für Zustellprobleme zählt (Deutsche Post Adress GmbH & Co. KG, 2024). Die Implementierung der E-Mail-Pipeline ist in Abschnitt 5.3 beschrieben.

4.4.2. Level 2: Semi-automatische Integration via Browser-Extension

Level 2 schließt die Lücke zwischen manuellen Prozessen und API-Integration. Da die Mehrheit relevanter Dienste zwar Webformulare, aber keine API zur Adressaktualisierung anbietet (vgl. Kapitel 3), erkennt eine Browser-Extension anhand von URL-Mustern das geöffnete Portal, ruft die Adressdaten über die SMove-GraphQL-API ab und befüllt die Formularfelder analog zu Passwort-Managern. Das finale Absenden verbleibt beim Nutzer, was sowohl Sicherheitsanforderungen als auch der aktiven Zustimmungshandlung nach DSGVO genügt (Europäisches Parlament, 2016).

Abbildung 4.4 zeigt den Ablauf im Detail.

Die technische Umsetzung (Manifest V3, Content Script, Background Service Worker, OAuth 2.0 mit PKCE, zweistufiges Feldmapping) ist in Abschnitt 5.3 beschrieben.

4.4.3. Level 3: Vollautomatische API-Integration

Level 3 ermöglicht die vollständig automatisierte Adressaktualisierung über API-Aufrufe oder Webhooks und setzt die aktive Kooperation des jeweiligen Dienstes voraus. Im Rahmen dieser Arbeit wird Level 3 als konzeptioneller Proof-of-Concept implementiert. Da öffentliche Adress-APIs bei der Mehrheit relevanter Dienste nicht existieren, steht der Nachweis der Erweiterbarkeit im Vordergrund, nicht die Vollständigkeit der Dienste-Abdeckung. Jeder Adapter folgt dem Strategy-Pattern (vgl. Abschnitt 5.3); neue Dienste erfordern lediglich eine neue Strategie-Klasse, keine Architekturänderung.

4.4.4. Vergleich mit bestehenden Ansätzen

Die dreistufige Strategie unterscheidet sich fundamental von den in Kapitel 2.3.1 analysierten Ansätzen. Sovrin, cheqd und die initiale Version von Trinsic bieten dem Endnutzer nur dann Mehrwert, wenn externe Dienste das jeweilige Protokoll bereits implementiert haben. Level 1 von SMove hingegen ist vollständig unabhängig von externer Kooperation und liefert damit unmittelbaren Mehrwert (*immediate value*) – einen Aspekt, den Hughes selbst im Kontext des gescheiterten SSI-Ansatzes von Trinsic als entscheidend fehlend beschrieb (Hughes, 2024a).

Tabelle 4.1 fasst die wesentlichen Unterschiede zusammen; Hovi wird als Vertreter eines neueren, SaaS-basierten SSI-Ansatzes einbezogen (vgl. Abschnitt 2.3.2).

Tabelle 4.1.: Vergleich: SMove vs. bestehende Ansätze

Kriterium	**Sovrin/cheqd**	**Trinsic (alt)**	**Hovi**	**SMove**
Partnerabhängigkeit	Hoch	Hoch	Mittel	Keine (L1)
Sofortiger Nutzen	Nein	Nein	Nein	Ja (L1)
Adoptionsrisiko	Hoch	Hoch	Mittel	Niedrig
Implementierungskomplexität	Hoch	Mittel	Mittel	Graduell
Blockchain-Abhängigkeit	Ja	Nein	Nein	Nein

Das zentrale Argument lässt sich wie folgt zusammenfassen: Ein Adress-Management-System, das erst ab einer bestimmten Partneradoption funktioniert, wird diese Adoption nie erreichen, weil Nutzer keinen Anreiz haben, ein System zu nutzen, das noch keinen Nutzen bietet. Der Multi-Channel-Ansatz durchbricht diesen Zirkel, indem Level 1 sicherstellt, dass das System von Tag 1 an einen konkreten Mehrwert liefert.

Damit adressiert die dreistufige Strategie direkt Teilfrage 1 (welche Integrationsmuster über verschiedene Kanäle geeignet sind) und Teilfrage 3 (wie sofortiger Nutzen unabhängig von externer Kooperation realisiert werden kann) aus Abschnitt 1.2.

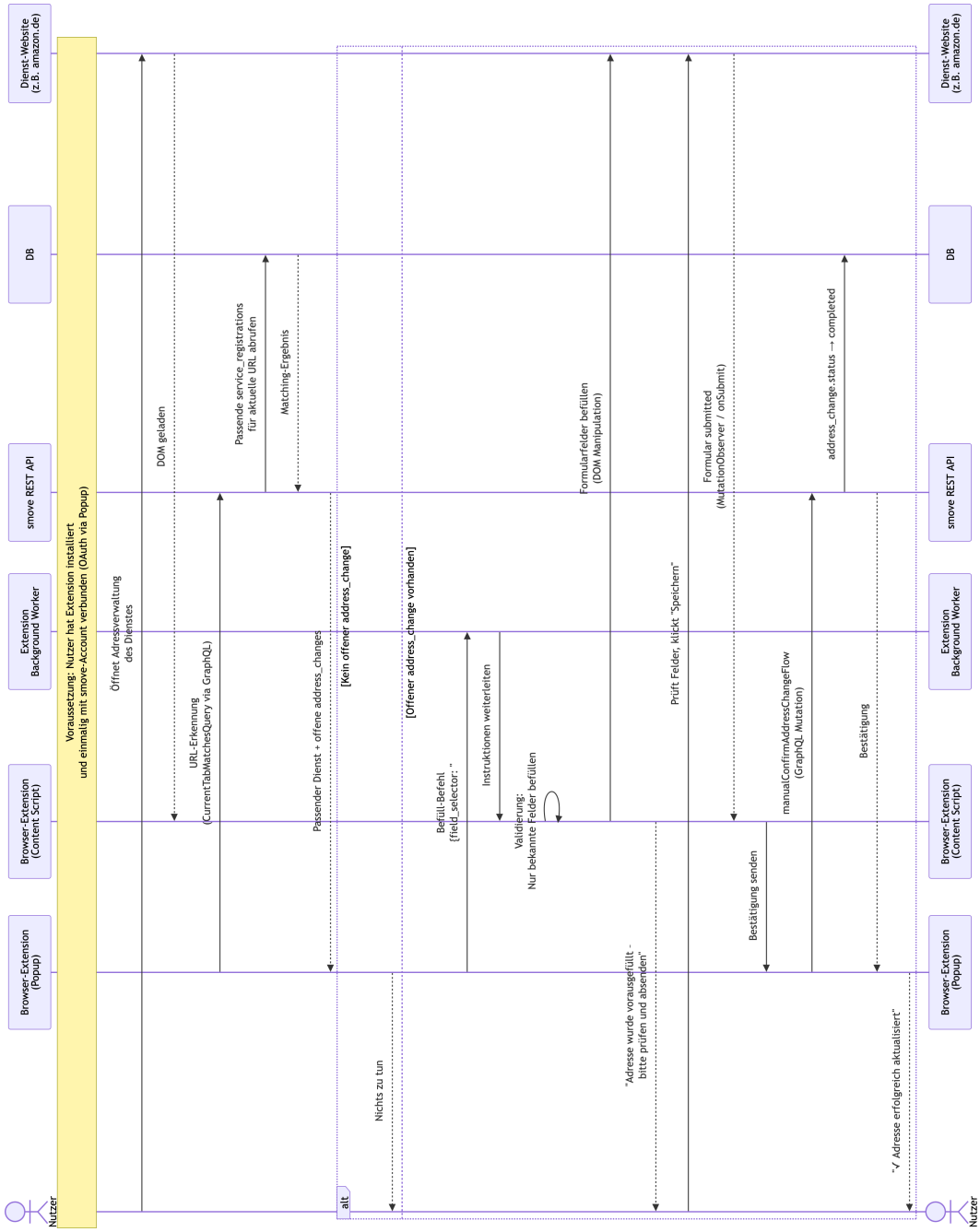


Abbildung 4.4.: Sequenzdiagramm: Level-2-Integration über die Browser-Extension

5. Implementierung

5.1. Technologie-Stack

Die Wahl des Technologie-Stacks folgt dem Pragmatismus-Prinzip aus Abschnitt 4.1.5: Da der wissenschaftliche Beitrag dieser Arbeit in der Integrationsarchitektur liegt, wurden etablierte Technologien bevorzugt, die generische Infrastrukturaufgaben mit geringem Eigenaufwand abdecken.

Als Backend-Framework kommt **Laravel** (Laravel Community, n. d. d) zum Einsatz, das serienmäßig alle benötigten Komponenten liefert: Object-Relational Mapping (ORM) mit transparenter Feldverschlüsselung (*Encrypted Casts* (Laravel Community, n. d. b)), Queue-Abstraktion, Event-System, Policy-Autorisierung sowie Laravel Passport als OAuth-2.0-Server mit PKCE-Unterstützung. Als Datenbank wird **MySQL/MariaDB** eingesetzt. Das relationale Modell mit referenzieller Integrität entspricht dem stark vernetzten Datenmodell (vgl. Abschnitt 4.3.4). Die Web-Oberfläche basiert auf **Inertia.js** („Inertia.js - The Modern Monolith“, n. d.), das Single Page Application (SPA)-Navigation ohne separate JSON-API ermöglicht. Für externe Clients stehen eine REST-API und eine GraphQL-API bereit (vgl. Abschnitt 4.1). Die Browser-Extension nutzt GraphQL und implementiert **Manifest V3** („Erweiterungen / Manifest V3“, n. d.), das dauerhafte Background-Scripts durch einen zustandslosen *Service Worker* ersetzt und eine explizit zustandslose Architektur erfordert (vgl. Abschnitt 5.2.3).

Der Grundsatz lautet: Generische Infrastrukturaufgaben werden durch bewährte Bibliotheken gelöst; domänenspezifische Logik – insbesondere die Abbildung von Diensten auf Formularfelder – erfordert Eigenentwicklung. Die wesentlichen Build-vs.-Buy-Entscheidungen sind in Tabelle A.3 im Anhang zusammengefasst.

5.2. Kernkomponenten

Die Implementierung gliedert sich in drei Schichten: das Laravel-Backend als zentrales Verarbeitungssystem, die Inertia-basierte Web-Oberfläche und die Browser-Extension als externer Client.

5.2.1. Backend-Architektur

Drei Entscheidungen im Backend haben direkte architektonische Konsequenzen für die Forschungsfrage:

Verschlüsselung auf Model-Ebene: Die Verschlüsselung sensibler Felder ist deklarativ auf Model-Ebene konfiguriert; die gesamte Anwendungslogik arbeitet mit unverschlüsselten Werten. Die Datenschutzanforderung ist damit strukturell erzwungen, nicht konventionsabhängig.

Policy-basierte Autorisierung: Für jedes relevante Model (`Address`, `ServiceRegistration`, `Relocation`, `AddressChange`) prüft eine Laravel Policy, ob der authentifizierte Nutzer Eigentümer der Ressource ist – einheitlich für Web-Oberfläche und API. Horizontale Privilege-Escalation durch manipulierte IDs wird damit zuverlässig verhindert.

Asynchrone Verarbeitung mit Queue-System: Hintergrundfähige Ausführungsstrategien werden als Queue-Jobs verarbeitet. Diese Entscheidung hat zwei Konsequenzen: Die HTTP-Response-Zeit ist von der tatsächlichen Verarbeitungsdauer entkoppelt, und der geplante Ausführungszeitpunkt ist auf Ebene der einzelnen Adressänderung gespeichert, sodass verschiedene Dienste desselben Umzugs zu unterschiedlichen Zeitpunkten benachrichtigt werden können (vgl. Abschnitt 4.3.4). Das asynchrone Modell ist die technische Voraussetzung für zwei Eigenschaften, die direkt aus den Anforderungen folgen: den Retry-Mechanismus bei fehlschlagenden Kanälen ohne Nutzeraktion (vgl. Abschnitt 5.3.3) und die zeitlich gesteuerte Ausführung pro Dienst, die unterschiedliche Benachrichtigungszeitpunkte innerhalb desselben Umzugs ermöglicht.

5.2.2. Frontend und Web-Oberfläche

Das zentrale UI-Element für den Nutzer ist die **Umzugs-Detailseite** (Abbildung 5.2), die den aggregierten Status aller `AddressChange`-Einträge visualisiert. Über die Fortschrittsübersicht sieht der Nutzer auf einen Blick, welche Dienste bereits aktualisiert wurden (`completed`), welche noch Handlung erfordern (`planned`, `needs_attention`), welche in Bearbeitung sind (`in_progress`, `awaiting_confirm`) und welche fehlgeschlagen sind (`failed`). Die Statusanzeige ist der direkte Ausdruck der neunstufigen Statusmaschine (vgl. Abschnitt 4.3.4, Abbildung 4.3). Ohne sie bliebe das Nutzerversprechen der Transparenz über einen verteilten Prozess ohne Oberfläche. Abbildung 5.1 zeigt die übergeordnete Dashboard-Ansicht, die dem Nutzer nach dem Login den aktuellen Stand des aktuellen Umzuges auf einen Blick vermittelt.

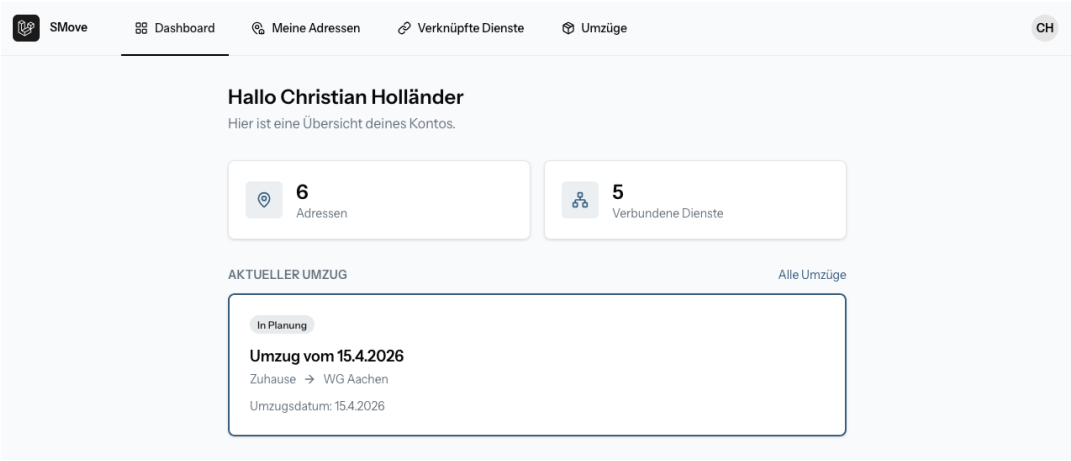


Abbildung 5.1.: Dashboard-Übersicht mit aktuellem Umzug und Statusanzeige

5.2.3. Browser-Extension

Die Browser-Extension ist der technisch differenzierteste Bestandteil von SMove, weil sie als externer Client mit eigener Laufzeitumgebung (dem Browser) funktioniert und dabei gleichzeitig mit dem SMove-Backend kommuniziert und in das DOM fremder Webseiten eingreift.

Strukturelle Aufteilung (Manifest V3)

Manifest V3 erzwingt eine strikte Kontexttrennung. Background Service Worker, Content Script und Popup laufen in isolierten Umgebungen mit jeweils unterschiedlichen Zugriffsrechten und kommunizieren ausschließlich über Message Passing. Diese Einschränkung ist zunächst eine Komplexitätsquelle, ergibt sich aber aus dem Sicherheitsmodell des Standards: Kein einzelner Kontext hat gleichzeitig DOM-Zugriff auf fremde Seiten *und* vollen Zugriff auf Extension-APIs. Die Architektur der SMove-Extension folgt dieser Trennung konsequent: API-Kommunikation und Token-Verwaltung liegen im Service Worker, DOM-Manipulation im Content Script, Nutzerinteraktion im Popup.

OAuth-Authentifizierung

Die Authentifizierung folgt dem OAuth-2.0-Flow mit PKCE (vgl. Abschnitt 4.2.3), der vollständig im Background Service Worker ausgeführt wird. Der Popup-Kontext

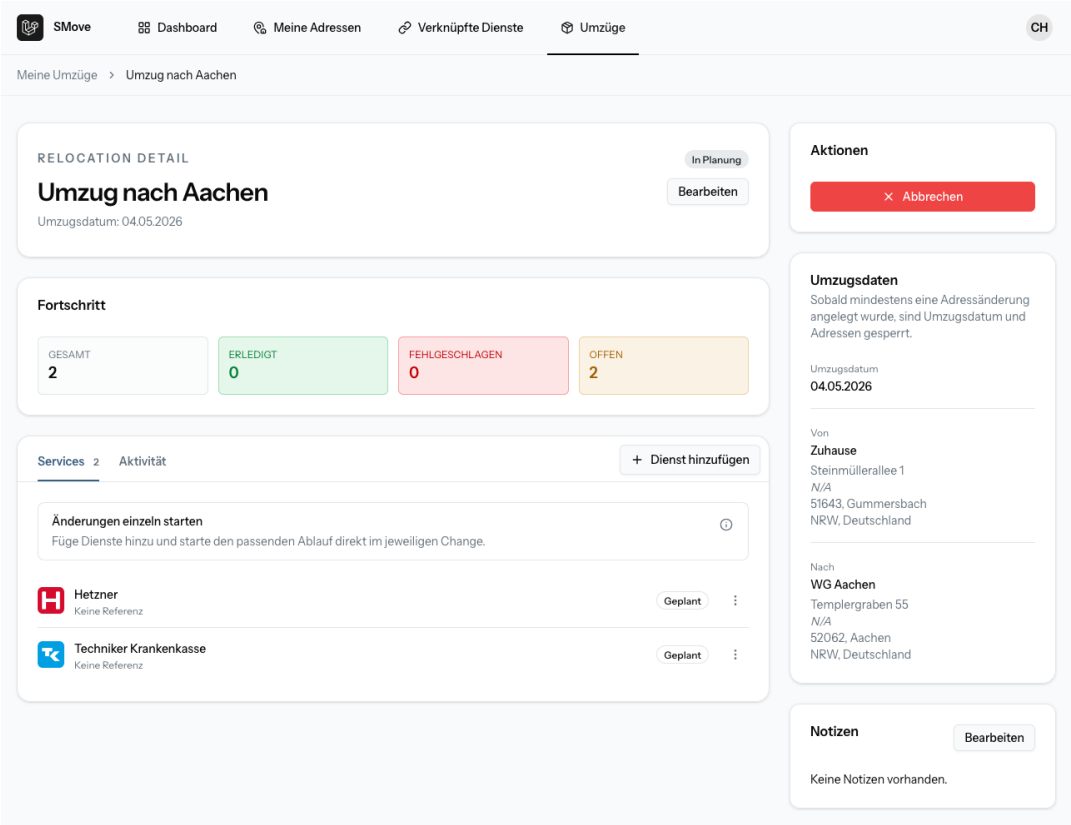


Abbildung 5.2.: Umzugs-Detailseite: Fortschrittsübersicht, verknüpfte Dienste und Adressinformationen

scheidet dafür aus. Manifest V3 schließt ihn automatisch, sobald der Nutzer zur OAuth-Autorisierungsseite wechselt, da ein dort gestarteter Flow seinen Callback nicht mehr empfangen könnte. Die Ablaufdetails sind in Anhang A.4 dokumentiert.

Formularerkennung und Feldmapping

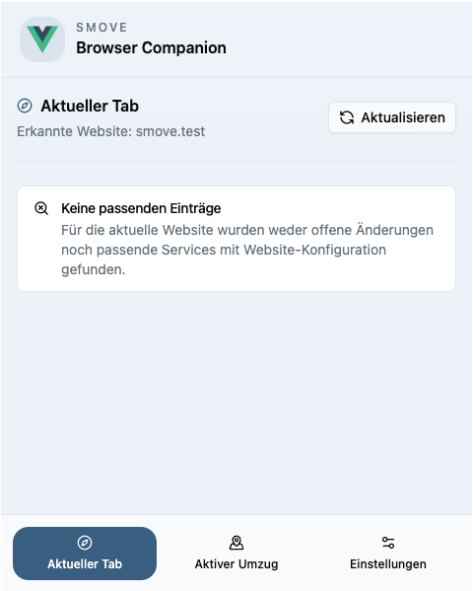
Das zentrale technische Problem von Level 2 ist die Heterogenität der Adressformulare: Jede Webseite verwendet eigene HTML-Strukturen, Attributnamen und Feldlayouts. Die Implementierung löst dieses Problem mit einem zweistufigen Ansatz.

Stufe 1 – Dienstspezifische Konfiguration (vorrangig): Ist für den erkannten Dienst ein explizites Feldmapping in der Service Registry hinterlegt, wird dieses vorrangig verwendet. Die Konfiguration bildet SMove-Adressfelder über CSS-Selektoren

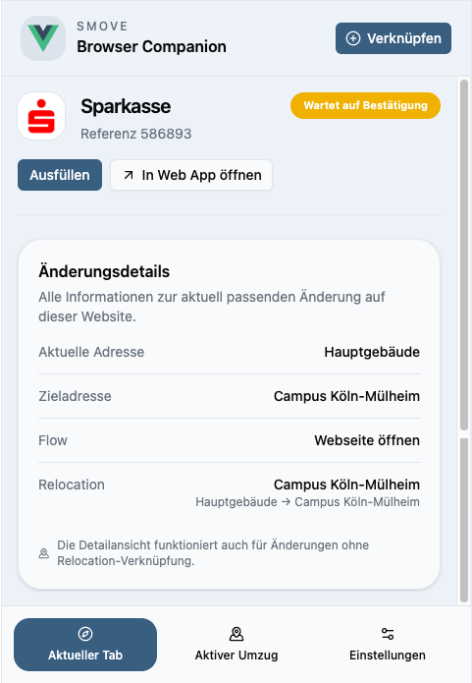
direkt auf die konkreten Formularfelder des Zieldienstes ab; ein URL-Muster bestimmt, ob die aktuelle Seite zum Dienst gehört. Dieser Ansatz garantiert Präzision, erfordert aber manuelle Pflege pro Dienst.

Stufe 2 – Generischer Fallback (dienstunabhängig): Ist kein dienstspezifisches Mapping vorhanden, greift eine heuristikbasierte Erkennung ohne manuelle Konfiguration. Priorisiert wird das `autocomplete`-Attribut: Es ist im WHATWG HTML Living Standard als Pflichtmerkmal für Adressformulare spezifiziert („HTML attribute“, 2026) und durch die WAI als Barrierefreiheitstechnik empfohlen („H98: Using HTML autocomplete attributes | WAI | W3C“, n. d.). Wer Barrierefreiheit ernst nimmt, liefert damit kostenlos die nötigen Metadaten. Fehlt dieses Attribut, wertet ein mehrsprachiges Muster-Set gängige Feldbezeichnungen aus (vgl. Anhang A.6.4). Dieser Fallback funktioniert für eine Vielzahl realer Formulare ohne Zusatzaufwand.

Der zweistufige Ansatz ist die technische Antwort auf Teilfrage 2: Er zeigt, unter welchen Bedingungen Browser-Extensions als Brückentechnologie ohne tiefe Dienstkooperation skalieren, sowie wo die Grenze dieser Skalierbarkeit liegt. Abbildung 5.3 zeigt das Extension-Popup für eine nicht erkannte und eine erkannte Seite.



(a) Kein passender Dienst für die aktuelle Webseite erkannt



(b) Erkannter Dienst (Sparkasse) mit Änderungsdetails und Befüllungs-Aktion

Abbildung 5.3.: Extension-Popup: Tab „Aktueller Tab“ – ohne Treffer (links) und mit erkanntem Dienst (rechts)

5.3. Multi-Channel-Implementierung

Die dreistufige Integrationsstrategie wurde in Kapitel 4.4 konzeptionell beschrieben. Dieser Abschnitt dokumentiert, wie die drei Stufen mit Fokus auf die technischen Entscheidungen, die über das Konzeptionelle hinausgehen, konkret implementiert sind. Zusammen decken die drei Stufen das vollständige Spektrum realistischer Kooperationsbedingungen ab und machen empirisch prüfbar, welche Integrationskanäle föderiertes Adressmanagement ermöglichen sowie wo die jeweiligen Grenzen liegen.

5.3.1. Level 1: E-Mail-basierte Adressänderung

Level 1 ist technisch die schlankeste Stufe mit dem höchsten Abdeckungsgrad: Jeder Dienst mit einer `ServiceEmailConfig` kann integriert werden. Die `AutoSendEmailStrategy` versendet die Mitteilung als Queue-Job und erzeugt dabei eine `Confirmation-`

`Challenge` mit öffentlichem Universally Unique Identifier (UUID) für den externen Bestätigungslink. Die `TemplateCopyManualEmailStrategy` generiert stattdessen einen vorformatierten Entwurf zum manuellen Kopieren; die Challenge wird analog erzeugt. Challenges verfallen nach einer konfigurierbaren Frist (Standard: drei Wochen).

Level 1 markiert den kooperationsfreien Grenzfall: Der Dienst muss nichts anpassen, keine API bereitstellen und keine Extension unterstützen. Eine bekannte E-Mail-Adresse genügt. Dies ist gleichzeitig die Stärke und die strukturelle Schwäche dieser Stufe. Der Prozess bleibt asynchron und manuell bestätigt, weil kein programmatisches Feedback des Empfängers möglich ist. Für die Forschungsfrage bedeutet das: Level 1 setzt die Untergrenze der erreichbaren Automatisierung, definiert aber auch den maximalen Abdeckungsgrad eines partnerunabhängigen Systems.

5.3.2. Level 2: Semi-automatische Integration via Browser-Extension

Level 2 ist die technisch aufwendigste und wissenschaftlich relevanteste Komponente von SMove. Der vollständige Ablauf ist in Abbildung 4.4 als Sequenzdiagramm dargestellt.

Ablauf und Datentransfer

Der Nutzer löst die Formularbefüllung über das Popup oder das Side Panel aus. Die UI-Komponente ruft die Adressdaten dabei selbst authentifiziert über GraphQL vom SMove-Backend ab und übergibt sie zusammen mit der Tab-ID an den Background Service Worker, der als Proxy für den Tab-Zugriff fungiert, da Popup und Side Panel keinen direkten Zugriff auf andere Tabs haben. Der Service Worker sendet die aufbereiteten Adressdaten anschließend via Message Passing an das Content Script, das die DOM-Manipulation durchführt. Die Extension speichert keine Adressdaten persistent. Sie hält die Felder ausschließlich für die Dauer der DOM-Manipulation im Speicher.

Die Bestätigung erfolgt manuell durch den Nutzer über das Extension-Popup oder das Dashboard, nachdem das Formular abgesendet wurde. Eine automatische Erkennung des erfolgreichen Absendens ist konzeptionell vorgesehen, im Prototyp aber nicht implementiert, da das System damit vermeidet, in unklaren Fällen fälschlicherweise `completed` zu setzen.

5.3.3. Level 3: API-Integration

Level 3 wird als konzeptioneller Proof-of-Concept implementiert und demonstriert das Integrationsmuster für Dienste mit programmatischer Adress-API. Alle Ausführungsstrategien folgen dem Strategy-Pattern (Gamma et al., 1995): gemeinsame abstrakte Basisklasse, einheitliche Schnittstelle, Verfügbarkeitsprüfung zur Laufzeit. Schlägt eine Ausführung fehl, greift ein Retry-Mechanismus mit exponentiellem Backoff (5 Minuten × Versuchsnummer, maximal drei Versuche); danach wechselt der Status auf `needs_attention`.

Level 3 ist der kooperationsintensive Gegenpol zu Level 1: Vollautomatische Ausführung ohne Nutzerinteraktion ist möglich, aber nur wenn der Dienst eine maschinenlesbare API bereitstellt und SMove die nötigen Zugangsdaten verwaltet. Das Strategy-Pattern macht diesen Unterschied für die restliche Architektur unsichtbar. SMove wählt zur Laufzeit die verfügbare Strategie (von API über Extension bis E-Mail) und degradiert dabei geordnet. Für die Forschungsfrage markiert Level 3 die Obergrenze der erreichbaren Automatisierung: Es zeigt, was bei vollständiger Dienstkooperation technisch möglich ist, und macht damit das Potenzial erkennbar, das Level 1 und Level 2 nur partiell ausschöpfen können.

Abbildung 5.4 fasst die Struktur aller drei Stufen zusammen: von den einmaligen Setup-Schritten über den pro-Umzug-Aufwand bis zur stufenspezifischen Ausführung pro Dienst.

Die Bewertung der drei Implementierungsstufen folgt in Kapitel 6.

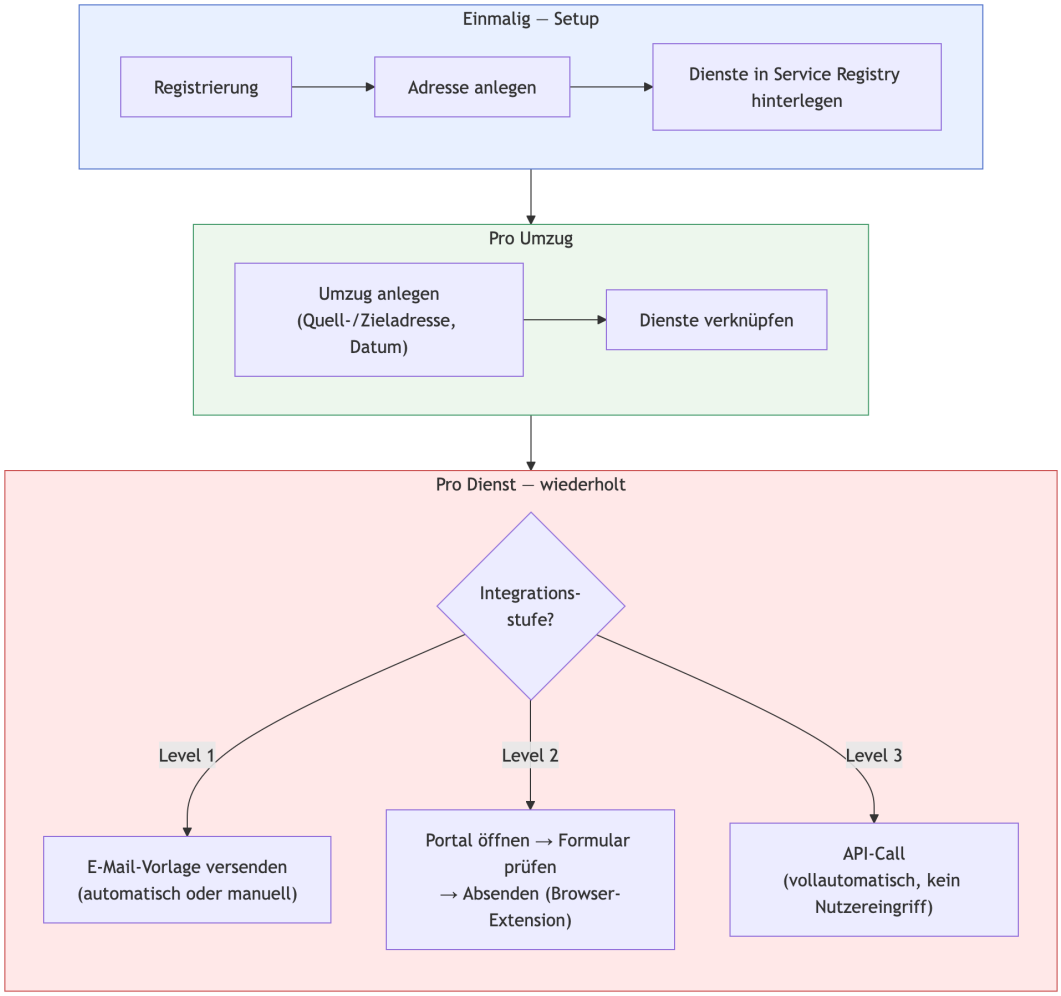


Abbildung 5.4.: SMove-Prozessstruktur: Einmalige Setup-Schritte, pro-Umzug-Schritte und pro-Dienst-Ausführung nach Integrationsstufe

5.4. Herausforderungen und Lösungsansätze

Die Architektur in Kapitel 4 beschreibt das System in seiner finalen Form. Die Implementierung verlief jedoch iterativ: Mehrere Designentscheidungen wurden erst durch konkrete Implementierungserfahrung ausgelöst oder nachträglich angepasst. Dieser Abschnitt dokumentiert die wesentlichen Abweichungen vom initialen Design und die Erkenntnisse, die zu den finalen Entscheidungen geführt haben (im Sinne der DSR-Methodik ein normaler und erwarteter Teil des Artefakt-Entwicklungsprozesses (Hevner et al., 2004)).

5.4.1. Schema-Evolution und Datenmodell-Refactoring

Zwei Iterationen prägten das finale Datenmodell: Die zunächst vorgesehene Zwischenentität `relocation_items` erwies sich als überflüssig, da ein `AddressChange` immer genau zu einer `Relocation` und einer `ServiceRegistration` gehört. Die Service-Konfiguration wurde von einem generischen JSON-Objekt zu typisierten Entitäten (`service_email_configs`) migriert, da sich Typsicherheit und Datenbankvalidierung nur so gewährleisten ließen.

5.4.2. Statusmaschine und Flow-Hierarchie

Das initiale Modell sah vier Zustände vor (`pending`, `sent`, `confirmed`, `failed`). Bereits in der ersten Iteration zeigte sich, dass das Dashboard zwischen laufendem Prozess, erwarteter Nutzeraktion und abgeschlossenem Vorgang unterscheiden muss, woraus die finale neunstufige Statusmaschine (vgl. Abschnitt 4.3.4, Abbildung 4.3) entstand. Gleichzeitig konnte ein `AddressChange` mit direkten Versuche-Referenzen einen Strategiewechsel nicht sauber abbilden; der `AddressChangeFlow` als Zwischenebene kapselt jeden Ausführungsdurchlauf, ohne die Historisierung vergangener Versuche zu beeinträchtigen.

5.4.3. Bestätigungs-Workflow

Aufforderung und Bestätigung sind zwei unabhängige Prozesse: Nicht jede Bestätigung erfordert einen externen Sachbearbeiter, nicht jeder Kanal erzeugt eine öffentliche UUID. Eine einzelne Entity vermengte beides und machte den Statusfluss unnötig komplex. Die Trennung in `ConfirmationChallenge` (Aufforderung per E-Mail mit

UUID-Link) und `AddressChangeConfirmation` (Bestätigungsnachweis, kanalunabhängig) löst dieses Problem – und vermeidet zugleich einen Login-Zwang für externe Sachbearbeiter.

5.4.4. Formular-Heterogenität und Manifest-V3-Einschränkungen

Der zweistufige Feldmapping-Ansatz (vgl. Abschnitt 5.2.3) mildert die Heterogenität realer Webformulare. Dienstspezifische CSS-Selektoren greifen vorrangig, der generische Fallback über `autocomplete`-Attribute und Regex-Muster deckt den Rest ab. Das Lifetime-Modell von Manifest V3 erfordert eine explizit zustandslose Extension-Architektur. Sämtliche Zustände werden in `chrome.storage.local` persistiert.

Die beschriebenen Herausforderungen sind nicht nur Implementierungsdetails, sondern haben architektonische Konsequenzen für die Forschungsfrage: Die Flow-Hierarchie ist die strukturelle Voraussetzung dafür, dass Strategiewechsel ohne Datenverlust möglich sind, und damit dafür, dass das System auch bei fehlschlagenden Kanälen partnerunabhängig bleibt. Die zustandslose Extension-Architektur wiederum ist die Bedingung dafür, dass Level 2 in einem sicherheitskonformen Browser-Kontext überhaupt realisierbar ist.

6. Evaluation

Die Evaluation des entwickelten Artefakts orientiert sich am Design Science Research Framework nach Hevner et al. (2004), das Evaluation als integralen Bestandteil des Forschungsprozesses versteht. Ziel ist es nicht, das System unter idealen Bedingungen zu testen, sondern nachzuweisen, dass es das identifizierte Problem – die mangelnde Effizienz und Transparenz beim Adressänderungsprozess – in realistischen Kontexten besser löst als der bisherige Status quo.

Hevner et al. (2004) unterscheiden dabei verschiedene Evaluationsmethoden, die je nach Artefakttyp und verfügbaren Ressourcen eingesetzt werden können. Für diese Arbeit wurden zwei komplementäre Methoden gewählt: funktionales Testen zur Überprüfung der technischen Korrektheit sowie eine szenarienbasierte Evaluation zum Nachweis der praktischen Nützlichkeit. Auf ein kontrolliertes Nutzerexperiment wurde bewusst verzichtet, da der dafür notwendige Aufwand – Rekrutierung, Durchführung und statistische Auswertung – den zeitlichen Rahmen dieser Arbeit überschreiten würde. Diese Lücke wird als Ansatzpunkt für weiterführende Forschung aufgegriffen (vgl. Kapitel 7.3).

6.1. Technische Evaluation durch automatisierte Tests

Den Ausgangspunkt der Evaluation bildet der Nachweis der technischen Korrektheit des Artefakts. Hevner et al. (2004) ordnen dies dem Bereich *Functional (Black Box) Testing* zu, das die grundlegende Funktionsfähigkeit eines Systems belegt und als *Proof by Construction* gilt.

6.1.1. Testaufbau und -umfang

Für die Umsetzung wurden automatisierte Tests auf zwei Ebenen eingesetzt: **Unit-Tests** für isolierte Klassen und Methoden (etwa URL-Pattern-Matching in der Service Registry) sowie **Feature-Tests**, die vollständige HTTP-Requests und GraphQL-Mutations gegen die Anwendung simulieren und sowohl Happy Path als auch Fehlerfälle – ungültige Token, abgelaufene Challenges, erschöpfte Retry-Versuche – abdecken.

Insgesamt umfasst die Test-Suite **73 Testdateien** mit **488 Testfällen** und **3.122 Assertions**, die in 58 Sekunden vollständig durchlaufen werden. Alle 488 Testfälle werden erfolgreich bestanden.

6.1.2. Testabdeckung

Die Zeilenabdeckung des gesamten Projekts beträgt **84,17 %** (5.024 von 5.969 Zeilen). Tabelle 6.1 zeigt die Abdeckung der für die Kernfunktionalität relevanten Module.

Tabelle 6.1.: Code-Coverage nach Modul (Zeilenabdeckung)

Modul	**Abdeckung (Zeilen)**	**Abdeckung (Methoden)**
Models	93,27 %	91,11 %
Actions	90,10 %	93,33 %
Concerns	93,33 %	80,65 %
Enums	87,50 %	78,26 %
Policies	80,65 %	62,50 %
Services	80,95 %	59,09 %
Jobs	76,92 %	70,00 %
GraphQL	77,27 %	57,14 %
Gesamt	**84,17 %**	**75,19 %**

Die höchste Abdeckung weisen die Kernmodule Models und Actions auf – genau jene Schichten, in denen die domänenspezifische Geschäftslogik konzentriert ist. Die niedrigere Abdeckung bei GraphQL-Resolvern und Jobs ist darauf zurückzuführen, dass Randpfade dieser Komponenten (z. B. Fehlerbehandlung bei fehlenden Relationen) im Testaufbau schwieriger zu provozieren sind.

6.1.3. Repräsentative Testfälle

Tabelle 6.2 zeigt eine Auswahl von Testfällen, die zentrale Invarianten des Systems abdecken.

Die Testfälle belegen insbesondere, dass die für die Sicherheit kritischen Invarianten – ressourcenbasierte Autorisierung, OAuth-Scope-Prüfung, Challenge-Lebenszyklus – zuverlässig durchgesetzt werden.

Tabelle 6.2.: Ausgewählte Testfälle und ihr Ergebnis

Testfall	**Komponente**	**Status**
Status eines `AddressChange` wird korrekt aus dem aktiven Flow abgeleitet	StatusObserver	✓
Statusübergang `planned` → `in_progress` → `awaiting_confirm` → `completed`	StatusObserver	✓
Erschöpfte Retry-Versuche setzen Status auf `needs_attention`	Job	✓
Veralteter Job-Lock wird nach Threshold-Überschreitung zurückgegeben	Job	✓
Bestätigung einer abgelaufenen Challenge liefert HTTP 410	Confirmation	✓
Bestätigung einer bereits aufgelösten Challenge liefert HTTP 409	Confirmation	✓
Horizontale Privilege-Escalation über manipulierte ID wird blockiert	Policy	✓
URL-Pattern-Matching erkennt korrekte und inkorrekte Dienst-URLs	Unit	✓
GraphQL-Mutation `manualConfirmAddressChangeFlow` erfordert korrekten OAuth-Scope	GraphQL	✓
Andere laufende Versuche werden bei erfolgreichem Attempt abgebrochen	AttemptObserver	✓

6.2. Szenarienbasierte Evaluation

Über die technische Korrektheit hinaus muss nachgewiesen werden, dass das System in realistischen Nutzungskontexten tatsächlich einen Mehrwert gegenüber dem Status quo bietet. Hevner et al. (2004) beschreiben hierfür die Methode der *Descriptive Evaluation through Scenarios*: Durch das Konstruieren und Durchspielen detaillierter Szenarien lässt sich argumentativ zeigen, dass ein Artefakt die Anforderungen realer Problemsituationen erfüllt.

Die Szenarien sind nicht rein konstruiert: Ergänzend wurden Beobachtungen aus einem zweistufigen Problemidentifikationstest einbezogen. Im ersten Durchlauf testete eine Testperson (23 Jahre). Die identifizierten Probleme, wie fehlende Orientierung nach der Registrierung, Notwendigkeit einzeln anzustoßender Änderungen, wurden durch erklärende Texte im Onboarding und im Umzugsablauf behoben. Im zweiten Durchlauf testeten zwei weitere Personen (60–65 Jahre) und bestätigten verbliebene

Reibungspunkte. Label-Felder für Adressen und Relocation sowie die Strategieauswahl waren nicht intuitiv. Zudem bestand Unsicherheit darüber, welcher Einstiegspunkt der richtige sei, obwohl beide infrage kommenden Wege zum Ziel geführt hätten.

Methodisch entspricht dieses Vorgehen keiner kontrollierten Studie und keiner systematischen heuristischen Evaluation. Die Beobachtungen erlauben keine quantitativen Aussagen über Fehlerrate oder Bearbeitungszeit. Dies ist die zentrale methodische Grenze dieser Arbeit: Aussagen zur Usability stützen sich auf drei Testpersonen in einem informellen Problemidentifikationstest, nicht auf eine repräsentative Stichprobe oder ein standardisiertes Verfahren. Diese Limitation wird in Kapitel 7.2 als primäre Einschränkung behandelt. Die vorliegenden Beobachtungen werden in Szenario 3 dennoch genutzt, um die identifizierten Hürden heuristisch einzuordnen.

Im Folgenden werden drei Szenarien vorgestellt, die die in Kapitel 3.1.1 identifizierten Nutzergruppen repräsentieren. Jedes Szenario prüft andere Aspekte des Multi-Channel-Integrationsansatzes.

6.2.1. Szenario 1: Einzelner Umzug mit wenigen Diensten (Level-1-Integration)

Das erste Szenario bildet den häufigsten Anwendungsfall ab: eine einzelne Person, die bei einem Umzug drei bis fünf Dienste informieren muss.

Ausgangssituation: Markus, ein digital versierter Berufstätiger, zieht um und muss seine Adresse bei der Bank, seinem Mobilfunkanbieter und einem Streaming-Dienst aktualisieren. Keiner dieser Dienste bietet eine direkte API-Schnittstelle zur Adressaktualisierung an.

Status quo: Für jeden Dienst ist ein separater Login erforderlich; jedes Portal hat ein anderes Interface. Kein einheitlicher Überblick existiert, welche Dienste bereits aktualisiert wurden.

Mit SMove: Markus legt einmalig seine neue Adresse an und verknüpft die drei Dienste (vgl. Abbildungen A.1 und A.2 im Anhang). Das System generiert vorausgefüllte E-Mail-Benachrichtigungen; Markus wählt pro Dienst zwischen automatischem Versand und manuellem Entwurf. Das Dashboard zeigt jederzeit, welche Dienste `awaiting_confirm` oder `completed` erreicht haben.

Bewertung: Die Bewertungskriterien für dieses Szenario sind Effizienz und Transparenz (vgl. Abschnitt 3.4). Abbildung 6.1 stellt die Prozessschritte beider Ansätze gegenüber: Separater Login, Portalnavigation und identische Feldeingabe wiederholen sich im Status quo pro Dienst. SMove reduziert dies auf eine zentrale Interaktion.

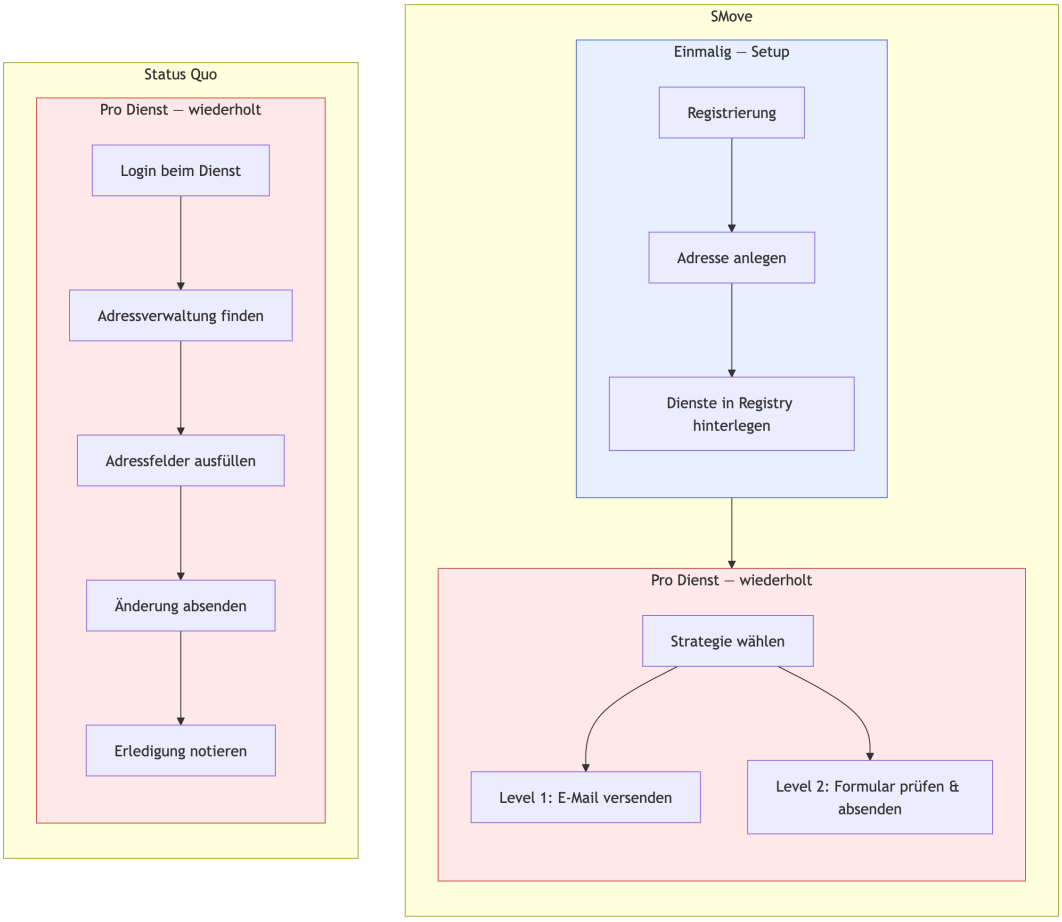


Abbildung 6.1.: Prozessvergleich: Status Quo (links) vs. SMove (rechts). Rot: pro Dienst wiederholte Schritte; blau: einmaliger Setup.

Die Effizienzanforderung ist damit erfüllt, soweit Effizienz als Reduktion redundanter Handlungen verstanden wird. Eine zeitliche Messung wurde nicht durchgeführt. Die Transparenzanforderung ist erfüllt, dadurch, dass das Dashboard den Status jedes `AddressChange` kontinuierlich abbildet: `awaiting_confirm` und `completed` sind jederzeit unterscheidbar, ohne auf das eigene Gedächtnis angewiesen zu sein.

6.2.2. Szenario 2: Wiederholte Umzüge mit wachsender Dienste-Landschaft (Level-2-Integration)

Das zweite Szenario adressiert einen Nutzer, der regelmäßig umzieht und dabei jeweils eine große Anzahl von Diensten verwalten muss.

Ausgangssituation: Jonas, 27 Jahre alt, ist seit dem Studium viermal umgezogen – zuletzt für eine neue Stelle in eine andere Stadt. In dieser Zeit hat sich seine Dienste-Landschaft erheblich ausgeweitet: zwei Bankkonten, drei Versicherungen, mehrere Streaming- und Abo-Dienste, Fitnessstudio, Strom- und Internetanbieter sowie diverse Online-Shops mit gespeicherter Lieferadresse. Beim letzten Umzug hat er trotz sorgfältiger Vorbereitung vier Dienste vergessen, was er erst bemerkte, als Post an die alte Adresse nicht weitergeleitet wurde.

Status quo: Jonas führt eine handgeschriebene Liste, vergisst aber regelmäßig Dienste, bei denen die letzte Korrespondenz Jahre zurückliegt.

Mit SMove: Jonas hat beim ersten Umzug alle relevanten Dienste einmalig in der Service Registry hinterlegt. Für den aktuellen Umzug legt er eine neue Relocation an und löst die Adressänderungen aus. Die Browser-Extension befüllt Adressformulare beim Öffnen der jeweiligen Portale automatisch; Jonas prüft und klickt auf Speichern. Kein Dienst kann vergessen werden, weil das System ihn explizit im Status `planned` hält, bis eine Aktion erfolgt. Abbildung 6.2 zeigt exemplarisch den Zustand nach einem Versuch mit ausstehender Bestätigung; Abbildung 6.3 den abgeschlossenen Zustand nach manueller Rückmeldung.

Bewertung: Dieses Szenario prüft zwei funktionale Anforderungen aus Abschnitt 3.3: erstens die vollständige Nachverfolgung aller Dienste (kein Dienst darf unbemerkt ausgelassen werden), zweitens die Wiederverwendbarkeit der Dienstekonfiguration über mehrere Umzüge hinweg. Die Vollständigkeitsanforderung ist durch die explizite Statusverfolgung jedes `AddressChange` erfüllt. Ein Dienst im Zustand `planned` bleibt sichtbar, bis eine Nutzeraktion ihn weiterbewegt – strukturell kann kein Dienst unbemerkt im Rückstand bleiben. Die Wiederverwendbarkeitsanforderung ist erfüllt, weil die Service Registry dienstübergreifend und umzugsunabhängig gepflegt wird. Beim zweiten Umzug verursacht die Dienste-Auswahl keinen erneuten Konfigurationsaufwand. Level-1 und Level-2 koexistieren im selben Umzugsvorgang – das Szenario demonstriert damit zugleich die inkrementelle Adoptionsstrategie. Jeder Dienst wird auf der Integrationsstufe ausgeführt, die für ihn konfiguriert ist, ohne dass eine einheitliche Mindestanforderung an alle Dienste gestellt wird.

6.2.3. Szenario 3: Nutzung durch weniger technikaffine Personen (Zero-Setup-Ansatz)

Das dritte Szenario prüft, ob das System auch für Nutzer zugänglich ist, die keine technischen Vorkenntnisse mitbringen.

Ausgangssituation: Monika, 58 Jahre alt, nutzt Smartphone und E-Mail, aber kein Online-Banking und hat wenig Erfahrung mit Web-Applikationen.

Fokus: Kann Monika ohne Einarbeitung die grundlegende Funktion des Systems nutzen, also eine Adressänderung anlegen und die Level-1-Integration (E-Mail-Vorlage) verwenden?

Walkthrough: Der Walkthrough basiert auf dem Prototyp mit Level-1-Integration und wird durch die Beobachtungen aus dem Problemidentifikationstest (zwei Testpersonen, 60–65 Jahre) gestützt.

Registrierung (E-Mail und Passwort) und Adresseingabe (sechs strukturierte Felder) verlaufen ohne Hürden. Erste Hürde: Das Label-Feld („Bezeichnung der Adresse“) ist nicht intuitiv. Es ist unklar, dass hier ein frei wählbarer Name wie „Neue Wohnung“ einzutragen ist. Zweite Hürde: Ohne Onboarding-Hinweis ist unklar, welcher Einstiegspunkt – Umzug oder Dienste zuerst – der intendierte ist, obwohl beide Wege zum Ziel führen. Das Dashboard zeigt nach der Registrierung eine dreischrittige Anleitung (vgl. Abbildung A.3 im Anhang), die den intendierten Ablauf skizziert – dieser Hinweis wurde von den Testpersonen jedoch nicht als ausreichend orientierend wahrgenommen. Der Dienstkatalog ist durchsuchbar; Monika findet Stromanbieter und Krankenkasse per Namenssuche. Die E-Mail-Vorlage – vorausgefüllte Nachricht zum Kopieren ins eigene E-Mail-Programm – verursacht keinen Lernaufwand, weil das Prinzip einer vertrauten Handlung entspricht. Statusverfolgung nach manuellem Versand erfolgt über das Dashboard; der Fortschritt aller Dienste ist auf einen Blick erkennbar.

Heuristische Einordnung der Reibungspunkte. Die im Walkthrough beobachteten Hürden lassen sich drei Heuristiken nach Nielsen (1994a) zuordnen: *Match between System and the Real World* (H2), *User Control and Freedom* (H3) sowie *Recognition rather than Recall* (H6); Tabelle 6.3 fasst die Befunde mit Schweregrad zusammen (Skala 0 = kein Problem, 4 = Katastrophe (Nielsen, 1994c)).

Bewertung: Das Bewertungskriterium für dieses Szenario ist die Benutzerfreundlichkeitsanforderung aus Abschnitt 3.4: Zero-Setup-Nutzung ohne Einarbeitung, minimale Schritte, verständliche Rückmeldungen. Die heuristische Einordnung zeigt: Die identifizierten Reibungspunkte liegen im Schweregrad 1–3; es gibt keine funktionalen Blockaden. Drei der vier Befunde sind durch gezielte Textanpassungen behebbar, ohne die Systemarchitektur zu berühren. Die Level-1-Integration selbst verursacht keinen

Lernaufwand. Sie greift auf das vertraute Konzept „E-Mail schreiben“ zurück und führt kein neues Interaktionsmuster ein. Das System setzt weder API-Kenntnisse noch Browser-Extension-Nutzung voraus. Die Benutzerfreundlichkeitsanforderung ist damit für den Level-1-Pfad als erfüllt zu werten – unter dem expliziten Vorbehalt, dass diese Einschätzung auf einem Problemidentifikationstest ohne systematische Messung basiert und durch eine kontrollierte Studie oder vollständige heuristische Evaluation zu validieren wäre (vgl. Kapitel 7.2).

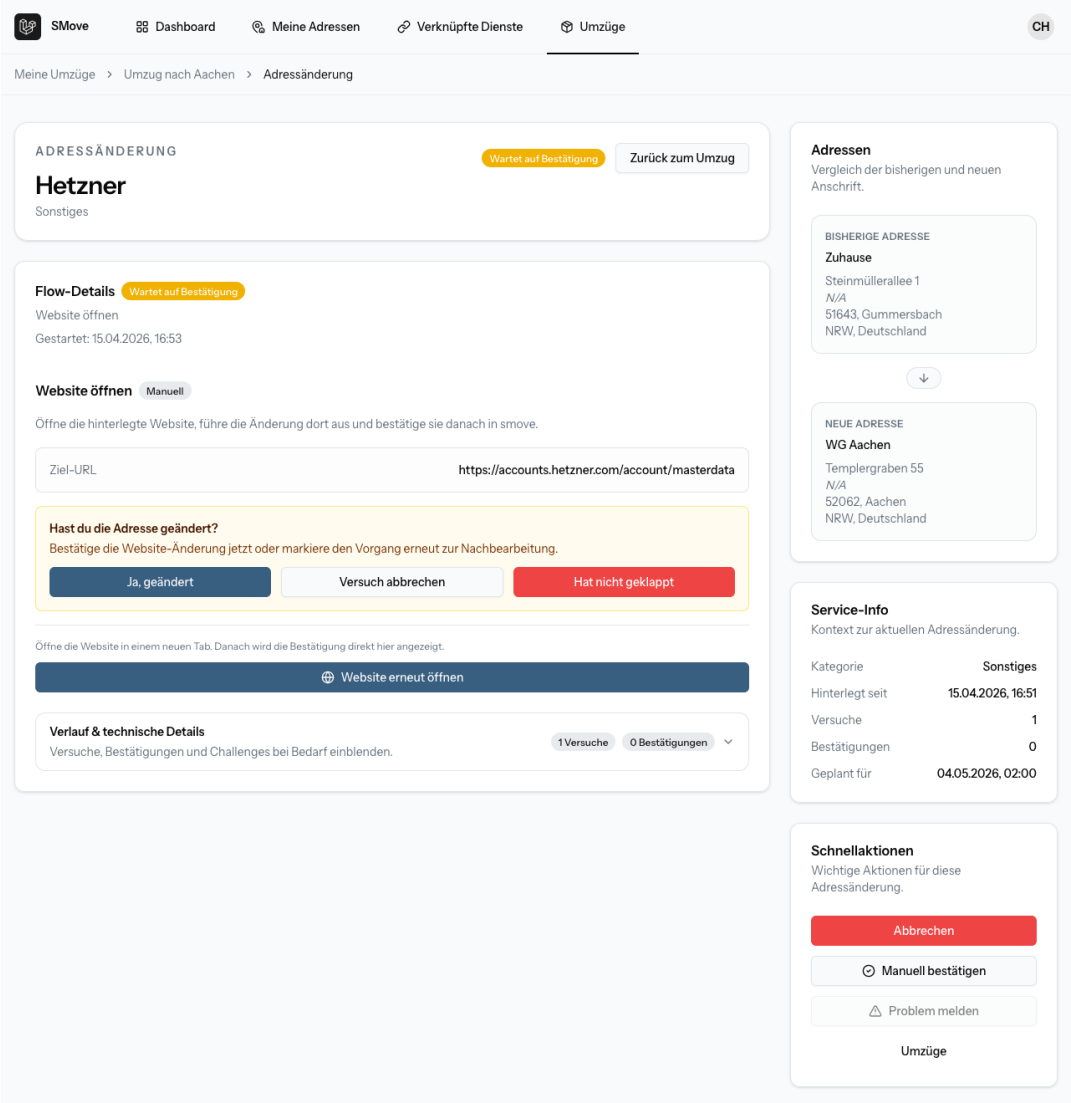


Abbildung 6.2.: Nach Öffnen der Webseite des Cloud-Anbieters Hetzner: Bestätigung ausstehend, 1 Versuch, Nutzer wird zur manuellen Rückmeldung aufgefordert

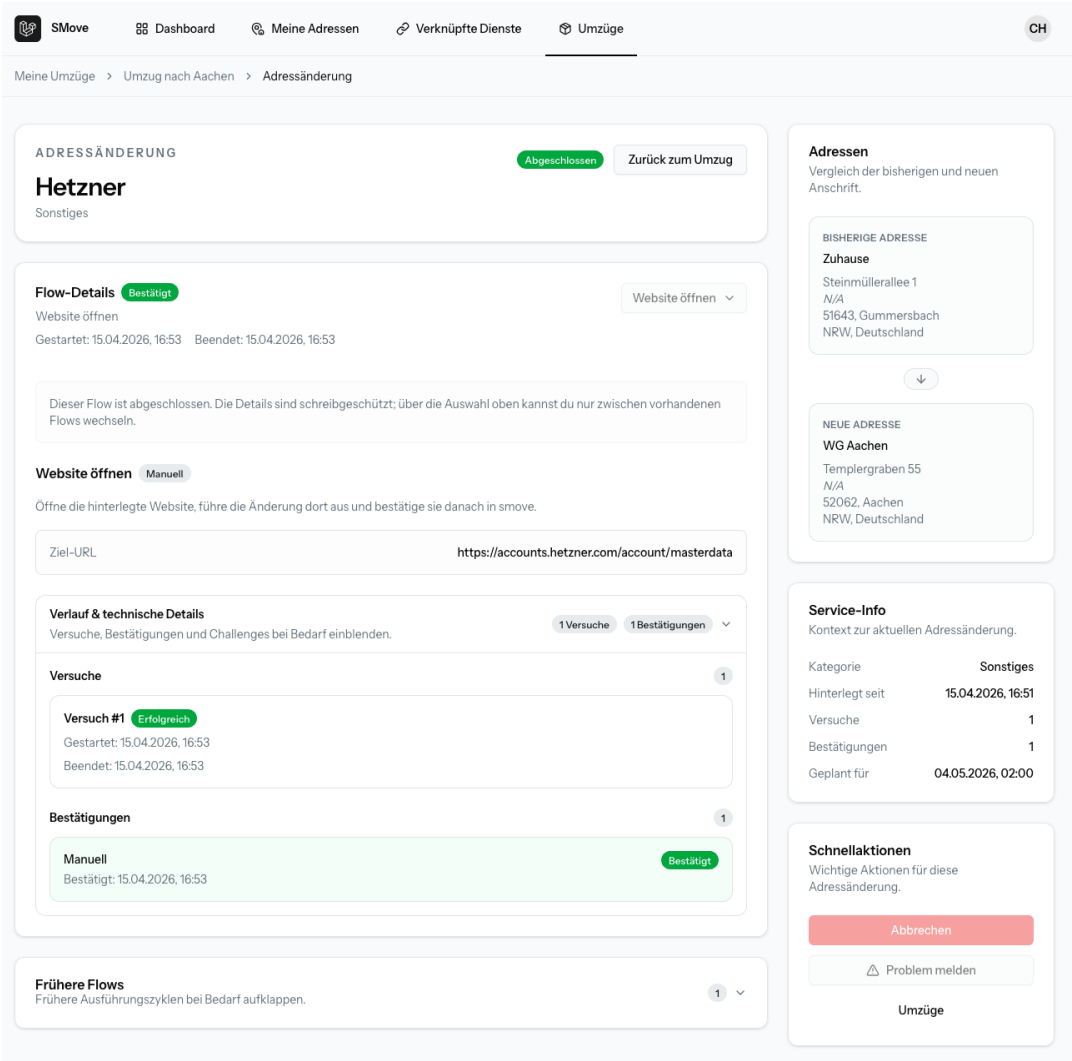


Abbildung 6.3.: Abgeschlossene Adressänderung: Status *Abgeschlossen*, Versuch erfolgreich, manuelle Bestätigung protokolliert

Tabelle 6.3.: Heuristische Einordnung der Usability-Befunde (Szenario 3)

Beobachtung	**Heuristik**	**Einordnung**	**Schweregrad**
Label-Feld („Bezeichnung der Adresse“) nicht intuitiv	H6: Recognition rather than Recall	Nutzer müssen erraten, was einzutragen ist; ein Platzhaltertext („z. B. Neue Wohnung“) würde den kognitiven Aufwand eliminieren	2
Reihenfolge Umzug vor Diensten unklar	H6: Recognition rather than Recall	Nutzer müssen sich die korrekte Reihenfolge merken, anstatt sie aus dem Interface ablesen zu können; geführtes Onboarding macht den nächsten Schritt sichtbar	2
Strategieauswahl nicht intuitiv	H2: Match between System and the Real World	Begriffe wie „Strategie“ oder „automatischer Versand“ entsprechen nicht dem mentalen Modell eines normalen Nutzers; nutzernahe Bezeichnungen wären angemessener	3
Unsicherheit über Einstiegspunkt	H3: User Control and Freedom	Kein Blocker (beide Wege führen zum Ziel), aber fehlende Orientierung darüber, welcher Weg der intendierte ist; nach Onboarding-Erweiterung von Testpersonen nicht mehr als Hauptbürde genannt	1

6.3. Diskussion der Evaluationsergebnisse

Die Evaluation auf zwei Ebenen – technische Korrektheit und praktische Nützlichkeit – zeigt, dass das entwickelte Artefakt die in Kapitel 3.3 formulierten Anforderungen im Wesentlichen erfüllt.

Die automatisierten Tests belegen mit 488 bestandenen Testfällen und einer Zeilenabdeckung von 84,17 %, dass das System technisch stabil ist und die definierten Kernfunktionen zuverlässig ausführt. Besonders die hohe Abdeckung der Models (93,27 %) und Actions (90,10 %) – also jener Schichten, in denen die domänenspezifische Logik konzentriert ist – gibt Vertrauen in die Korrektheit der Statusübergänge, der Autorisierungsprüfungen und der Retry-Mechanismen.

Die szenarienbasierte Evaluation zeigt darüber hinaus, dass der Multi-Channel-Ansatz in verschiedenen Nutzungskontexten einen Mehrwert gegenüber dem Status quo bietet. In Szenario 1 liegt dieser Mehrwert primär in der Transparenz und Vollständigkeit; in Szenario 2 kommt der Effekt der akkumulierten Dienste-Basis hinzu, der den Nutzen mit jedem weiteren Umzug steigert.

Die drei Teilfragen aus Abschnitt 1.2 lassen sich auf Basis der Evaluationsergebnisse wie folgt beantworten:

Teilfrage 1 – Multi-Channel Web-Integration: Die Kombination aus E-Mail-Vorlagen, Browser-Extension und API-Adapter bildet ein vollständiges Spektrum webbasierter Integrationsmuster ab; jede Stufe erbringt eigenständig Nutzen (bestätigt für Level 1 in Szenario 1, Level 2 in Szenario 2).

Teilfrage 2 – Browser-Extensions als Brückentechnologie: Die Extension schließt die Lücke zwischen manuellen Prozessen und API-Integration. Dienstspezifisches CSS-Selektor-Mapping greift vorrangig. Fehlt es, erkennt ein generischer Fallback über `autocomplete`-Attribute und Feldbezeichnungs-Heuristik die Formularfelder ohne Zusatzkonfiguration. Grenze ist das Versagen der Heuristik bei obfuskierten oder gänzlich bezeichnerfreien Feldern.

Teilfrage 3 – Inkrementelle Adoption: Szenario 3 und der informelle Nutzungstest zeigen, dass das System durch strukturierte Führung, vollständige Dienste-Liste und transparente Statusverfolgung, auch ohne Browser-Extension und API-Konfiguration unmittelbaren Mehrwert liefert. Der Nutzen skaliert mit der vorhandenen Konfiguration; eine kritische Masse ist nicht Voraussetzung.

Gleichzeitig werden durch die Evaluation Grenzen sichtbar: Die URL-Erkennung der Browser-Extension setzt ein hinterlegtes URL-Muster voraus. Der generische Fallback deckt viele Formulare ab, greift aber nur zuverlässig, wenn aussagekräftige `autocomplete`-Attribute oder Feldbezeichnungen vorhanden sind. Das System löst das

Problem der *Durchführung* von Adressänderungen, nicht das vorgelagerte Problem der *vollständigen Bestandsaufnahme*. Ansätze dazu sowie weiterführende Forschungsfragen werden in den Kapiteln 7.2 und 7.3 diskutiert.

Rückbezug auf die Forschungsfrage: Die in Abschnitt 1.2 formulierte Forschungsfrage – wie lässt sich eine Multi-Channel-Integrationsarchitektur gestalten, die unabhängig vom Digitalisierungsgrad externer Dienste funktioniert – wird durch die Evaluation *beantwortet*. Level 1 (E-Mail) und Level 2 (Browser-Extension) liefern unmittelbaren Nutzen ohne jede Partner-Kooperation. Level 2 funktioniert dabei auch ohne dienstspezifische Konfiguration, da der generische Fallback über `autocomplete`-Attribute und Feldbezeichnungs-Heuristik eine Vielzahl realer Formulare abdeckt. Lediglich Level 3 (API) setzt Kooperation voraus – ist aber als optionale Erweiterungsstufe konzipiert, nicht als Voraussetzung für Grundnutzen. Partnerunabhängigkeit ist damit strukturell verankert: Kooperation *verbessert* das System, ist aber auf keiner Stufe *notwendig* für sofortigen Mehrwert.

7. Diskussion und Ausblick

7.1. Generalisierbarkeit und architektonische Implikationen

7.1.1. Übertragbarkeit auf andere Datentypen

Die entwickelte Architektur ist konzeptionell nicht auf Postadressen beschränkt. Das zentrale Abstraktionsprinzip – ein Nutzer verwaltet einen strukturierten Datensatz und möchte Änderungen koordiniert an externe Dienste propagieren – ist auf andere personenbezogene Stammdaten übertragbar. Telefonnummern, E-Mail-Adressen, Bankverbindungen oder Notfallkontakte folgen demselben Muster.

Die Übertragbarkeit ergibt sich direkt aus dem Systemaufbau. Das Konzept der *ServiceDataRequirements* ist nicht adressspezifisch. Es beschreibt, welche Felder ein Dienst für eine Änderungsoperation benötigt. Das Strategy-Pattern der `BaseExecutionStrategy` abstrahiert den Übermittlungskanal vollständig vom Dateninhalt. Neue Datentypen ließen sich durch Erweiterung des Datenmodells und neue Strategieklassen integrieren, ohne die bestehende Architektur zu verändern. Einschränkend ist anzumerken, dass verschiedene Datentypen unterschiedliche Sensitivitäts- und Compliance-Anforderungen mit sich bringen; die Erweiterbarkeit ist daher eine strukturelle Eigenschaft des Designs, keine unmittelbare Produktionszusage.

7.1.2. Architektonische Implikationen für ähnliche Systeme

Die Arbeit bestätigt drei Designprinzipien, die über den konkreten Anwendungsfall hinaus relevant sind.

Partnerunabhängigkeit durch abgestufte Kanalstrategie: Das strukturelle Adoptionsproblem föderierter Systeme – kein Nutzen ohne Partner, keine Partner ohne Nutzer – lässt sich durch eine abgestufte Strategie durchbrechen, bei der die unterste Stufe ohne externe Kooperation auskommt. Die Beobachtung aus den Evaluationsszenarien, dass Level 1 den größten praktischen Mehrwert lieferte, stützt dieses Prinzip. Hohpe und Woolf betonen, dass Integrationslösungen mit bestehenden Systemen zusammenarbeiten müssen, ohne diese verändern zu können. Der Mehrwert muss also

unabhängig von der vollständigen Teilnahme aller Partner entstehen (Hohpe & Woolf, 2003).

Zentraler Vertrauens-Provider als Entwurfsmuster: Die Erfahrung aus der Implementierung zeigt, dass die zentrale Architektur nicht nur ein pragmatisches Zugeständnis ist, sondern aktiv Mehrwert schafft. Die stellvertretende Authentifizierung gegenüber externen Diensten ist erst durch Zentralisierung möglich und wäre in einem rein dezentralen Modell nicht realisierbar. Hohpe und Woolf beschreiben das *Message Broker*-Muster als Mechanismus, der Sender und Empfänger entkoppelt und die Routing-Verantwortung zentralisiert (Hohpe & Woolf, 2003, S. 286); SMove überträgt dieses Prinzip auf den Authentifizierungskontext.

Graceful Degradation als Qualitätsmerkmal: Der explizite Status `needs_attention` hat sich in der Evaluation als entscheidend erwiesen: Ein System, das Handlungsbedarf sichtbar macht statt stumm zu versagen, erhält die Handlungsfähigkeit des Nutzers auch bei partiellen Fehlern. Tanenbaum und Van Steen beschreiben *partial failure* als charakteristisches Merkmal verteilter Systeme und formulieren als zentrales Designziel, dass ein System beim Ausfall einer Komponente weiterhin in akzeptabler Weise operieren können soll (Tanenbaum & Van Steen, 2007, S. 321). Dieses Prinzip ist auf jeden Integrationskontext übertragbar, in dem externe Systeme heterogen und nicht vollständig kontrollierbar sind.

7.1.3. Datensouveränität ohne SSI

Die drei genannten Prinzipien lassen sich in einen breiteren konzeptuellen Rahmen einordnen. SSI-Projekte wie Sovrin und Trinsic haben Datensouveränität über dezentrale Infrastruktur angestrebt, sind aber am Adoptionsdilemma gescheitert (vgl. Kapitel 2.3). SMove verfolgt eine andere Strategie: Datensouveränität wird nicht durch Dezentralisierung realisiert, sondern durch *explizite Kontrolle im zentralisierten Modell*. Der Nutzer entscheidet für jeden Dienst individuell, welche Adressfelder weitergegeben werden; der Audit-Log macht jede Übermittlung nachvollziehbar. Diese Annäherung verzichtet bewusst auf die technische Reinheit des SSI-Modells, erreicht aber die wesentlichen Nutzerziele – Transparenz, Kontrolle, Selektivität – ohne die Adoptionsvoraussetzungen, die SSI-Systeme bislang blockiert haben.

7.2. Limitierungen und Herausforderungen

Der Prototyp erfüllt die gesetzten Forschungsziele, weist aber Grenzen auf, die für eine produktive Weiterentwicklung relevant sind.

Eingeschränkte empirische Absicherung der Usability-Aussagen: Die bedeutendste methodische Grenze dieser Arbeit liegt in der Evaluation der Benutzerfreundlichkeit. Die szenarienbasierte Evaluation wurde durch einen informellen zweistufigen Nutzungstest mit insgesamt drei Testpersonen (23 und 60–65 Jahre) ergänzt. Diese Datenbasis erlaubt keine generalisierbaren Aussagen über Fehlerrate, Bearbeitungszeit oder Akzeptanz im Breiteneinsatz. Die heuristische Einordnung der identifizierten Reibungspunkte (Tabelle 6.3) kompensiert diese Lücke teilweise. Sie macht Befunde methodisch handhabbar, ersetzt aber keine externe Validierung durch unabhängige Evaluatoren oder eine kontrollierte Nutzerstudie. Alle Usability-Aussagen dieser Arbeit – insbesondere die Einschätzung, dass identifizierte Hürden im Schweregrad 1–2 liegen und das System auch für weniger technikaffine Nutzer zugänglich ist – sind als vorläufig einzustufen und bedürfen einer Überprüfung mit robusteren Methoden.

Initiale Befüllung der Dienste-Landschaft: Der Mehrwert von SMove wächst mit der Vollständigkeit der hinterlegten Dienste. Gleichzeitig ist aber diese initiale Vollständigkeit die größte Hürde bei der ersten Nutzung. Ein Nutzer, der SMove erstmals einrichtet, muss sich aktiv daran erinnern, bei welchen Diensten seine Adresse gespeichert ist – strukturell dasselbe Problem, das beim manuellen Umzug zum Vergessen einzelner Dienste führt. SMove verwaltet effizient, was der Nutzer bereits kennt; Ansätze zur systematischen Erkennung der individuellen Dienste-Landschaft werden in Kapitel 7.3 diskutiert.

Pflege der Service Registry: Der zweistufige Ansatz – dienstspezifisches Mapping vorrangig, generischer Fallback über `autocomplete`-Attribute und mehrsprachige Feldbezeichnungen – reduziert den Pflegeaufwand erheblich. Dennoch bleibt die Skalierung der URL-Musterpflege eine Herausforderung, die eine community-driven Registry adressieren könnte (vgl. Kapitel 7.3). Die dienstspezifischen Mappings wurden exemplarisch für wenige Testdienste konfiguriert; eine produktionstaugliche Abdeckung würde eine kontinuierlich gewartete Registry mit Hunderten von Einträgen erfordern.

Adoption ohne Netzwerkeffekte: Anders als SSI-Systeme ist SMove nicht auf die Kooperation externer Dienste angewiesen. Dennoch entsteht ein subtiler Netzwerkeffekt auf der Ebene der Service Registry: Je mehr Dienste konfiguriert sind, desto nützlicher ist das System für neue Nutzer.

Formular-Heterogenität und Erkennungsrate: Der generische Fallback der Formularerkennung (vgl. Abschnitt 5.2.3) funktioniert zuverlässig, solange Formularfelder aussagekräftige Bezeichner tragen. Verwenden Formulare generische oder obfuskierte Feldbezeichnungen, ist ein manuelles CSS-Selektor-Mapping erforderlich. Für ausgewählte reale Dienste existieren JavaScript-Tests, die die Erkennungsrate exemplarisch absichern. Eine systematische Erhebung über eine repräsentative Breite an Diensten bleibt einer Folgestudie vorbehalten.

Fehlende akademische Vorarbeiten: Eine systematische Literaturrecherche zu multi-channel personal data propagation oder föderiertem Stammdatenmanagement im Consumer-Kontext ergibt keinen akademischen Bestand, der direkt vergleichbar wäre. Dieser negative Befund ist selbst ein Ergebnis: Er unterstreicht die Relevanz des gewählten Forschungsansatzes und begründet, warum die Arbeit primär auf Standards, Praxisberichte und verwandte EAI-Literatur zurückgreift.

Kein Produktionsbetrieb: SMove ist ein Forschungsprototyp. Aspekte wie Skalierbarkeit unter Last, Infrastrukturbetrieb, Monitoring und fehlertolerante Queue-Verarbeitung wurden konzeptionell berücksichtigt und könnten in einer Folgeentwicklung produktionsreif umgesetzt werden.

7.3. Offene Forschungsfragen

Die Evaluation und Implementierung von SMove legen mehrere Forschungsfragen offen, die über den Rahmen dieser Arbeit hinausgehen.

Vollständige Bestandsaufnahme der Dienste-Landschaft. Das zentrale ungelöste Problem ist nicht die Durchführung von Adressänderungen, sondern die vorgelagerte Frage: Wie erfährt ein Nutzer zuverlässig, bei welchen Diensten seine Adresse gespeichert ist? Zwei konzeptionelle Ansätze sind denkbar, die jedoch beide weiterführende Forschung erfordern:

Erstens ein *Push-Modell nach Vorbild der Deutschen Post*: Unternehmen könnten analog zum bestehenden Nachsendeauftrag Adressänderungen beim Nutzer anfragen, aber der Nutzer behält die vollständige Kontrolle darüber, welche neue Adresse er freigibt und an wen. Dieses Modell setzt jedoch die aktive Kooperation der Dienste voraus und führt damit dasselbe Adoptionsdilemma wieder ein, an dem bestehende SSI-Ansätze gescheitert sind (vgl. Kapitel 2.3.2).

Zweitens eine *passive Erkennung über die Browser-Extension*: Die Extension könnte im normalen Browser-Alltag des Nutzers Webseiten erkennen, bei denen ein Konto mit gespeicherter Adresse vermutet wird – etwa anhand von Login-Formularen oder Profil-Seiten bekannter Dienste –, und den Nutzer fragen, ob dieser Dienst verknüpft werden soll. Dieser Ansatz funktioniert ohne Kooperation externer Dienste und wäre ein natürlicher Erweiterungspfad der bestehenden Extension-Architektur. Offen bleiben dabei Fragen der Präzision (false positives), der Nutzererfahrung bei häufigen Prompts sowie des Datenschutzes beim Browser-seitigen Tracking besuchter Seiten.

Community-driven Service Registry. Die manuelle Pflege von Dienst-Konfigurationen durch einen zentralen Betreiber skaliert langfristig nicht. Eine community-driven

Registry, in der Nutzer selbst Konfigurationen einreichen und pflegen, ist ein naheliegender Erweiterungspfad, aber offene Fragen zur Qualitätssicherung, Duplikatsvermeidung und Vertrauensstruktur aufwirft, die Anleihen aus kollaborativen Wissensdatenbanken oder Open-Source-Prozessen nehmen könnten.

Quantitative Nutzerstudie. Die szenarienbasierte Evaluation dieser Arbeit ist argumentativer Natur. Eine kontrollierte Nutzerstudie mit einer repräsentativen Stichprobe – idealerweise mit Personen, die sich tatsächlich in einem Umzug befinden – würde belastbarere Aussagen über Zeitersparnis, Fehlerquoten und Nutzerzufriedenheit ermöglichen.

8. Zusammenfassung

Diese Arbeit hat eine Multi-Channel-Integrationsarchitektur für föderiertes Adressmanagement konzipiert, prototypisch implementiert und evaluiert. Ausgangspunkt war ein praktisches Problem mit strukturellen Ursachen: Millionen Umziehende müssen ihre Adresse manuell bei dutzenden Diensten aktualisieren. Es ist ein repetitiver, fehleranfälliger Prozess ohne zentrale Koordination. Bisherige SSI-Ansätze scheiterten nicht an technischer Unreife, sondern am Adoptionsdilemma (vgl. Kapitel 2.3); SMove begegnet diesem durch Partnerunabhängigkeit als zentrales Designprinzip.

Die dreistufige Kanalstrategie – E-Mail-Vorlagen (Level 1), Browser-Extension (Level 2) und API-Integration (Level 3) – beantwortet die Teilfragen aus Abschnitt 1.2 wie in Kapitel 6 ausgeführt. Der entscheidende Effekt liegt dabei weniger in der Zeitersparnis als in der *strukturellen Vollständigkeit*. Das System stellt sicher, dass kein registrierter Dienst vergessen wird und macht Handlungsbedarf durch den Status `needs_attention` explizit sichtbar. Drei architektonische Designprinzipien erweisen sich als auf ähnliche Stammdaten-Verwaltungsszenarien übertragbar: *Partnerunabhängigkeit durch abgestufte Kanalstrategie*, *zentraler Vertrauens-Provider* und *Graceful Degradation*.

Ausblick

Die entwickelte Architektur ist konzeptionell nicht auf Postadressen beschränkt. Das Abstraktionsprinzip ist auf andere personenbezogene Stammdaten übertragbar. Offene Forschungsfragen betreffen die automatische Bestandsaufnahme der individuellen Dienste-Landschaft, die community-getriebene Pflege der Service Registry sowie eine kontrollierte Nutzerstudie zur empirischen Absicherung der Usability-Erkenntnisse (vgl. Kapitel 7.3).

Literaturverzeichnis

Aaron Parecki, aman-agrwl, rhertogh & Athanasios Kapsalis. (n. d.). PKCE for OAuth 2.0. Verfügbar 25. Februar 2026 unter https://oauth.net/2/pkce/

Bank-Verlag GmbH. (n. d.). PSD2 : Deutsche Bundesbank. Verfügbar 25. April 2026 unter https://resources.bank-verlag.de/psd2/deutsche-bundesbank

browser-use/browser-use [original-date: 2024-10-31T16:00:56Z]. (2026, April). Verfügbar 13. April 2026 unter https://github.com/browser-use/browser-use

Christopher Allen. (2016, April). The Path to Self-Sovereign Identity. Verfügbar 19. Februar 2026 unter https://www.lifewithalacrity.com/article/the-path-to-self-soverereign-identity/

Curran, S. (2025, Februar). Sovrin Foundation MainNet Ledger Shutdown Likely on or before March 31, 2025. Verfügbar 19. Februar 2026 unter https://sovrin.org/sovrin-foundation-mainnet-ledger-shutdown-likely-on-or-before-march-31-2025/

Curran, S. (2026, Januar). The Sovrin Foundation Has Been Dissolved — but Sovrin MainNet Remains. Verfügbar 19. Februar 2026 unter https://sovrin.org/the-sovrin-foundation-has-been-dissolved-but-sovrin-mainnet-remains/

Deutsche Post Adress GmbH & Co. KG. (2024, Juni). *Die Umzugstudie 2024 - So zieht Deutschland um* (Studie). Deutsche Post Adress. Gütersloh. Verfügbar 29. Oktober 2025 unter https://www.postadress.de/umzugsstudie.pdf

Deutsche Post AG. (n. d.). Post an neue Adresse nachsenden | Deutsche Post | Nachsendeservice. Verfügbar 13. Februar 2026 unter https://www.deutschepost.de/de/n/nachsendeservice.html

Deutsche Post Direkt. (2025a, April). Adress-Studie 2025: Untersuchung zur Qualität von Adressdaten in Deutschland. Verfügbar 13. April 2026 unter https://www.deutschepost.de/int-static/informieren/downloads/ext/dp/deutsche-post-direkt/dp-ddp-adress-studie-2025-de.pdf

Edwards, F. (2026, Januar). 2025 in Review: cheqd's Year of Building Trust, Identity, and Verifiable AI [Section: cheqd News]. Verfügbar 1. April 2026 unter https://cheqd.io/blog/2025-in-review-cheqds-year-of-building-trust-identity-and-verifiable-ai/

Erweiterungen / Manifest V3. (n. d.). Verfügbar 1. April 2026 unter https://developer.chrome.com/docs/extensions/develop/migrate/what-is-mv3?hl=de

Europäisches Parlament. (2016, April). Verordnung (EU) 2016/679 des Europäischen Parlaments und des Rates vom 27. April 2016 zum Schutz natürlicher Personen bei der Verarbeitung personenbezogener Daten, zum freien Datenverkehr und zur Aufhebung der Richtlinie 95/46/EG (Datenschutz-Grundverordnung) (Text von Bedeutung für den EWR) [Legislative Body: EP, CONSIL]. Verfügbar 25. Februar 2026 unter http://data.europa.eu/eli/reg/2016/679/oj

Europäisches Parlament. (2024, April). Regulation (EU) 2024/1183 of the European Parliament and of the Council of 11 April 2024 amending Regulation (EU) No 910/2014 as regards establishing the European Digital Identity Framework [Legislative Body: CONSIL, EP]. Verfügbar 19. Februar 2026 unter http://data.europa.eu/eli/reg/2024/1183/oj

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995, Januar). *Design patterns: elements of reusable object-oriented software | Guide books | ACM Digital Library*. Verfügbar 1. April 2026 unter https://dl.acm.org/doi/book/10.5555/186897

H98: Using HTML autocomplete attributes | WAI | W3C. (n. d.). Verfügbar 12. April 2026 unter https://www.w3.org/WAI/WCAG21/Techniques/html/H98

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Q.*, *28*(1), 75–105. https://doi.org/10.2307/25148625

Hohpe, G., & Woolf, B. (2003, November). *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley.

Holländer, C., Kermani, H. M., Ley, S., & Pürenli, S. (2025, Oktober). Requirements Engineering Tool - Adressänderungen. Verfügbar 1. April 2026 unter https://re-tool-582487.gitlab.io/

Homeday.de. (n. d.). Umzugstipps: Checkliste für den reibungslosen Umzug | Homeday. Verfügbar 13. Februar 2026 unter https://www.homeday.de/de/immobilienwissen/umzugstipps/

Hovi - Identity Infrastructure For The Internet. (n. d.). Verfügbar 1. April 2026 unter https://www.hovi.id/

HTML attribute: autocomplete - HTML | MDN. (2026, März). Verfügbar 12. April 2026 unter https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Attributes/autocomplete

Hughes, R. (2024a, Oktober). Why Verifiable Credentials Aren't Widely Adopted & Why Trinsic Pivoted. Verfügbar 19. Februar 2026 unter https://rileyparkerhughes.medium.com/why-verifiable-credentials-arent-widely-adopted-why-trinsic-pivoted-aee946379e3b

ImmobilienScout24. (n. d.). Zeitplan & Checkliste für den Umzug: Was du beachten solltest. Verfügbar 13. Februar 2026 unter https://www.umzug.de/checkliste.html

Inertia.js - The Modern Monolith. (n. d.). Verfügbar 1. April 2026 unter https://inertiajs.com/

Kubach, M., Schunck, C. H., Sellung, R., & Roßnagel, H. (2020). Self-sovereign and Decentralized identity as the future of identity management? https://doi.org/10.24406/publica-r-409252

Laravel Community. (n. d. b). Eloquent: Mutators & Casting - Laravel 12.x - The PHP Framework For Web Artisans. Verfügbar 25. Februar 2026 unter https://laravel.com/docs/12.x/eloquent-mutators

Laravel Community. (n. d. d). Laravel Passport - Laravel 12.x - The PHP Framework For Web Artisans. Verfügbar 25. Februar 2026 unter https://laravel.com/docs/12.x/passport

Lisa Gutknecht. (n. d.). Checkliste Adressänderung: Richtig ummelden nach dem Umzug. Verfügbar 13. Februar 2026 unter https://wohnglueck.de/artikel/richtig-ummelden-checkliste-35462

Lodderstedt, T., Bradley, J., Labunets, A., & Fett, D. (2025, Januar). *Best Current Practice for OAuth 2.0 Security* (Request for Comments Nr. RFC 9700) (Num Pages: 46). Internet Engineering Task Force. https://doi.org/10.17487/RFC9700

Nielsen, J. (1994a, April). 10 Usability Heuristics for User Interface Design. Verfügbar 26. April 2026 unter https://www.nngroup.com/articles/ten-usability-heuristics/

Nielsen, J. (1994c, November). *Usability Engineering*. Morgan Kaufmann. https://dl.acm.org/doi/10.5555/2821575

OpenAI. (2025, Oktober). Wir stellen vor: ChatGPT Atlas, der Browser mit integrierter ChatGPT-Funktion. Verfügbar 13. April 2026 unter https://openai.com/de-DE/index/introducing-chatgpt-atlas/

Peffers, K., Tuunanen, T., Rothenberger, M., & Chatterjee, S. (2007). A Design Science Research Methodology for Information Systems Research. *J. Manage. Inf. Syst.*, *24*(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302

Sakimura, N., Bradley, J., & Agarwal, N. (2015, September). *Proof Key for Code Exchange by OAuth Public Clients* (Request for Comments Nr. RFC 7636) (Num Pages: 20). Internet Engineering Task Force. https://doi.org/10.17487/RFC7636

Satybaldy, A., Ferdous, M. S., & Nowostawski, M. (2024). A Taxonomy of Challenges for Self-Sovereign Identity Systems. *IEEE Access*, *12*, 16151–16177. https://doi.org/10.1109/ACCESS.2024.3357940

Shopify. (n. d.). customerAddressUpdate - Storefront API. Verfügbar 24. April 2026 unter https://shopify.dev/docs/api/storefront/latest/mutations/customerAddressUpdate

Skyvern-AI/skyvern [original-date: 2024-02-28T15:45:19Z]. (2026, April). Verfügbar 13. April 2026 unter https://github.com/Skyvern-AI/skyvern

Sovrin.org. (n. d.). Sovrin Home. Verfügbar 11. Oktober 2025 unter https://sovrin.org/

Tanenbaum, A. S., & Van Steen, M. (2007). *Distributed Systems: Principles and Paradigms* (2. Aufl.). Pearson Prentice Hall.

Umzugsmitteilung | umziehen.de. (2025, Dezember). Verfügbar 1. April 2026 unter https://umziehen.de/umzugsmitteilung

Wallets | Hovi Documentation. (n. d.). Verfügbar 1. April 2026 unter https://hovi.id/concepts/wallets

Woo. (n. d.). Customers – WooCommerce REST API Documentation - WP REST API v3. Verfügbar 24. April 2026 unter https://woocommerce.github.io/woocommerce-rest-api-docs/#customers

Weiterführende Literatur

ADDRESSFACTORY | Deutsche Post | Deutsche Post Direkt. (n. d.). Verfügbar 13. April 2026 unter https://www.deutschepost.de/de/d/deutsche-post-direkt/addressfactory.html

Adressqualität: Jede sechste Kundenadresse ist unzustellbar. (2018, Oktober). Verfügbar 13. April 2026 unter https://www.ibusiness.de/aktuell/db/210113steiger_2.html

Alina Kazlova. (2024, Februar). Header vs Sidebar: A simple guide to better navigation design — Header vs Sidebar: A simple guide to better navigation design. Verfügbar 19. Januar 2026 unter https://saltnbold.com/blog/post/header-vs-sidebar-a-simple-guide-to-better-navigation-design

Bitwarden. (n. d.). Whitepaper zur Sicherheit bei Bitwarden. Verfügbar 25. Februar 2026 unter https://bitwarden.com/de-de/help/bitwarden-security-whitepaper/

Build Trust Registries | Product Docs. (2025, Juli). Verfügbar 1. April 2026 unter https://docs.cheqd.io/product/studio/trust-registries

Deutsche Post Adress GmbH & Co. KG. (2018, Mai). Umzug: Adresse ändern & Versicherungen melden | umziehen.de. Verfügbar 13. Februar 2026 unter https://umziehen.de/an-ab-ummelden/adressaenderung-beim-umzug-12151

Deutsche Post Direkt. (2025b, April). Adressaktualisierung mit ADDRESSFACTORY: Sales Information. Verfügbar 13. April 2026 unter https://www.deutschepost.de/dam/jcr:fdbe58fd-1c80-498a-8180-38b1e7ca30b8/dp-ddp-si-adressaktualisierung.pdf

Ed Sullivan & Scott DeVaney. (2025, März). So geht Mozilla an Manifest V3 heran: Was ist anders und warum ist das für Nutzer*innen von Erweiterungen wichtig? | The Mozilla Blog. Verfügbar 1. April 2026 unter https://blog.mozilla.org/de/firefox-de/mozilla-manifest-v3/

Gregor, S., & Hevner, A. R. (2013). Positioning and Presenting Design Science Research for Maximum Impact1. *MIS Quarterly*, *37*(2), 337–355. https://doi.org/10.25300/MISQ/2013/37.2.01

Grobecker, C. (2024, November). Wanderungsbewegungen | Sozialbericht 2024. Verfügbar 13. Februar 2026 unter https://www.bpb.de/kurz-knapp/zahlen-und-fakten/sozialbericht-2024/553022/wanderungsbewegungen/

How does Sovrin use Blockchain? (n. d.). Verfügbar 1. April 2026 unter https://sovrin.org/faq/how-does-sovrin-use-blockchain/

HTML Standard. (n. d.). Verfügbar 12. April 2026 unter https://html.spec.whatwg.org/multipage/form-control-infrastructure.html#autofilling-form-controls:-the-autocomplete-attribute

Hughes, R. (2024b, Dezember). Trinsic Divests SSI Assets to Dentity, Completing Pivot to Identity Acceptance - Trinsic. Verfügbar 19. Februar 2026 unter https://trinsic.id/trinsic-decentralized-identity-assets-acquired-by-dentity-to-accelerate-digital-id-adoption/

Hughes, R. (2025). (15) Posten | LinkedIn. Verfügbar 19. Februar 2026 unter https://www.linkedin.com/posts/rileyparkerhughes_an-end-of-an-era-sovrin-foundation-announced-activity-7254133733178449921-TUgb/

Huttenloher, R. (2018, Dezember). Schlechte Daten sind schlecht für das Geschäft. Verfügbar 13. April 2026 unter https://line-of.biz/customer-relationship-management/schlechte-daten-sind-schlecht-fuer-das-geschaeft/

Inhaltsscripts. (n. d.). Verfügbar 1. April 2026 unter https://developer.chrome.com/docs/extensions/develop/concepts/content-scripts?hl=de

Laatikainen, G., Kolehmainen, T., & Abrahamsson, P. (2021). Self-Sovereign Identity Ecosystems: Benefits and Challenges. https://aisel.aisnet.org/scis2021/10/

Laravel Community. (n. d. a). Authorization - Laravel 12.x - The PHP Framework For Web Artisans. Verfügbar 25. Februar 2026 unter https://laravel.com/docs/12.x/authorization

Laravel Community. (n. d. c). Encryption - Laravel 12.x - The PHP Framework For Web Artisans. Verfügbar 25. Februar 2026 unter https://laravel.com/docs/12.x/encryption

Mike Schwartz & Riley Hughes. (2024, Dezember). Episode 75: Trinsic's pivot from SSI to identity acceptance - Gluu. Verfügbar 1. April 2026 unter https://gluu.org/office-episodes/episode-75-trinsics-pivot-from-ssi-to-identity-acceptance/

Movinga. (n. d.). Adressänderung beim Umzug - Checkliste. Verfügbar 13. Februar 2026 unter https://movinga.com/de/de/hub/adressaenderung/

Next-gen Web Extension Framework – WXT. (n. d.). Verfügbar 1. April 2026 unter https://wxt.dev

Nielsen, J. (1994b). Enhancing the explanatory power of usability heuristics. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 152–158. https://doi.org/10.1145/191666.191729

OpenAI. (2023, März). *GPT-4 Technical Report* (Techn. Ber.). OpenAI. Verfügbar 21. April 2026 unter https://cdn.openai.com/papers/gpt-4.pdf

The Payment & Trust Infrastructure for Credentials. (2026, März). Verfügbar 1. April 2026 unter https://cheqd.io/

Postbank. (n. d.). Checkliste Umzug mitteilen.

Press Release - Trinsic. (n. d.). Verfügbar 1. April 2026 unter https://trinsic.id/category/press-release/

Redreamality. (2024, März). Browser Extension Development Guide. Verfügbar 1. April 2026 unter https://redreamality.com/blog/browser-extension-development/

Röhl, K.-H., & Engels, B. (2025, April). *Behörden-Digimeter 2025* (Techn. Ber.). Institut der deutschen Wirtschaft (IW). Verfügbar 1. April 2026 unter https://www.iwkoeln.de/studien/klaus-heiner-roehl-barbara-engels-behoerden-digimeter-2025.html
Im Auftrag der Initiative Neue Soziale Marktwirtschaft (INSM).

Sketch.media. (2025, Juli). Website Navigation - Übersicht, Vor- &Nachteile & Beispiele. Verfügbar 19. Januar 2026 unter https://sketch.media/projekte/blog/ratgeber/851-website-navigation-menue-uebersicht-beispiele-vorteile-nachteile.html

Strategy. (n. d.). Verfügbar 1. April 2026 unter https://refactoring.guru/design-patterns/strategy

Strategy-Pattern. (n. d.). Verfügbar 1. April 2026 unter https://www.hsbi.de/elearning/data/FH-Bielefeld/lm_data/lm_1359639/pattern/strategy.html

Umzug-Adressermittlung senkt Postretouren & Kosten. (n. d.). Verfügbar 13. April 2026 unter https://addressware.com/umzug-adressermittlung

Web Extensions. (n. d.). Verfügbar 1. April 2026 unter https://w3c.github.io/webextensions/specification/

A. Anhang

A.1. Repositories und Demo-Deployment

Alle im Rahmen dieser Arbeit entstandenen Artefakte sind in der GitLab-Gruppe https://gitlab.com/christian98-uni/masterarbeit öffentlich einsehbar. Tabelle A.1 listet die einzelnen Repositories mit ihren direkten URLs auf.

Tabelle A.1.: Repositories der Implementierung

Komponente	**URL**
Backend / Web-App (SMove)	https://gitlab.com/christian98-uni/masterarbeit/smove
Browser-Extension	https://gitlab.com/christian98-uni/masterarbeit/smove-browser-extension

Ein öffentlich erreichbares Demo-Deployment ist unter folgender Adresse verfügbar:

https://smove.pyramine.de

Die Browser-Extension ist in den jeweiligen Browser-Stores verfügbar:

Chrome Web Store:
https://chromewebstore.google.com/detail/smove/ggcbamfjlapphmmfiggmidcopepibial

Firefox Add-ons:
https://addons.mozilla.org/en-US/firefox/addon/smove/

A.2. Evaluierte Dienste – Realitätscheck

Tabelle A.2 zeigt die im Rahmen des Realitätschecks (vgl. Abschnitt 3.2) untersuchten Dienste samt der jeweils vorgefundenen Änderungskanäle für Stammdaten.

Tabelle A.2.: Evaluierte Dienste und gefundene Adressänderungskanäle

Dienst	**Kategorie**	**Gefundene Kanäle**
Deutsche Bank	Bank	Website, Persönlich
Trade Republic	Bank / Broker	Website
Volksbank Köln-Bonn	Bank	Website, E-Mail, Persönlich
Deutsche Rentenversicherung	Behörde	Website, Postformular
Rundfunkbeitrag	Behörde	Website
TH Köln CaMS	Hochschule	E-Mail, Website
HUK	Versicherung	Website, E-Mail
Techniker Krankenkasse	Versicherung	Website
Amazon	Online-Handel	Website
Netflix	Streaming	Website
Overleaf (Stripe)	Abonnementdienst	Website
Payback	Kundenprogramm	Website
Contabo	Hosting	Website
Hetzner	Hosting	Website
Shopify	E-Commerce-Plattform	Website, API
WooCommerce	E-Commerce-Plattform	API, Website

A.3. Screenshots

A.3.1. Webanwendung

Die folgenden Screenshots illustrieren ausgewählte Schritte des Nutzer-Walkthroughs aus Kapitel 6.

SMove | Dashboard | Meine Adressen | Verknüpfte Dienste | Umzüge | CH

Meine Umzüge > Neuen Umzug erstellen

Neuen Umzug erstellen

Nach der Erstellung werden automatisch alle verknüpften Dienste der bisherigen Adresse übernommen.

Bisherige Adresse

Die neue Adresse wird direkt mit dem Umzug gespeichert.

Gespeicherte Adresse verwenden | Neue Adresse anlegen

Du hast noch keine gespeicherten Adressen. Lege hier direkt eine neue an.

Anzeigename: Zuhause

Straße | Hausnummer

Adresszusatz

Postleitzahl | Stadt

Bundesland | Land

Neue Adresse

Die neue Adresse wird direkt mit dem Umzug gespeichert.

Gespeicherte Adresse verwenden | Neue Adresse anlegen

Du hast noch keine gespeicherten Adressen. Lege hier direkt eine neue an.

Anzeigename: Zuhause

Straße | Hausnummer

Adresszusatz

Postleitzahl | Stadt

Bundesland | Land

Umzugsdatum *: TT . MM . JJJJ

Titel: z. B. Umzug nach München

Notizen: Weitere Informationen zum Umzug...

Abbrechen | Umzug erstellen

Abbildung A.1.: Formular zum Erstellen eines Umzugs mit Feldern für bisherige und neue Adresse sowie Umzugsdatum

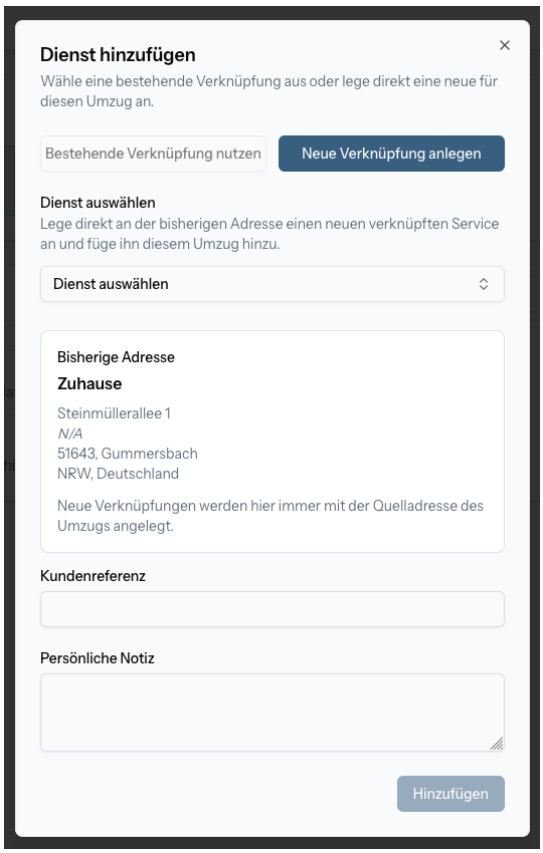


Abbildung A.2.: Dienst einem Umzug hinzufügen

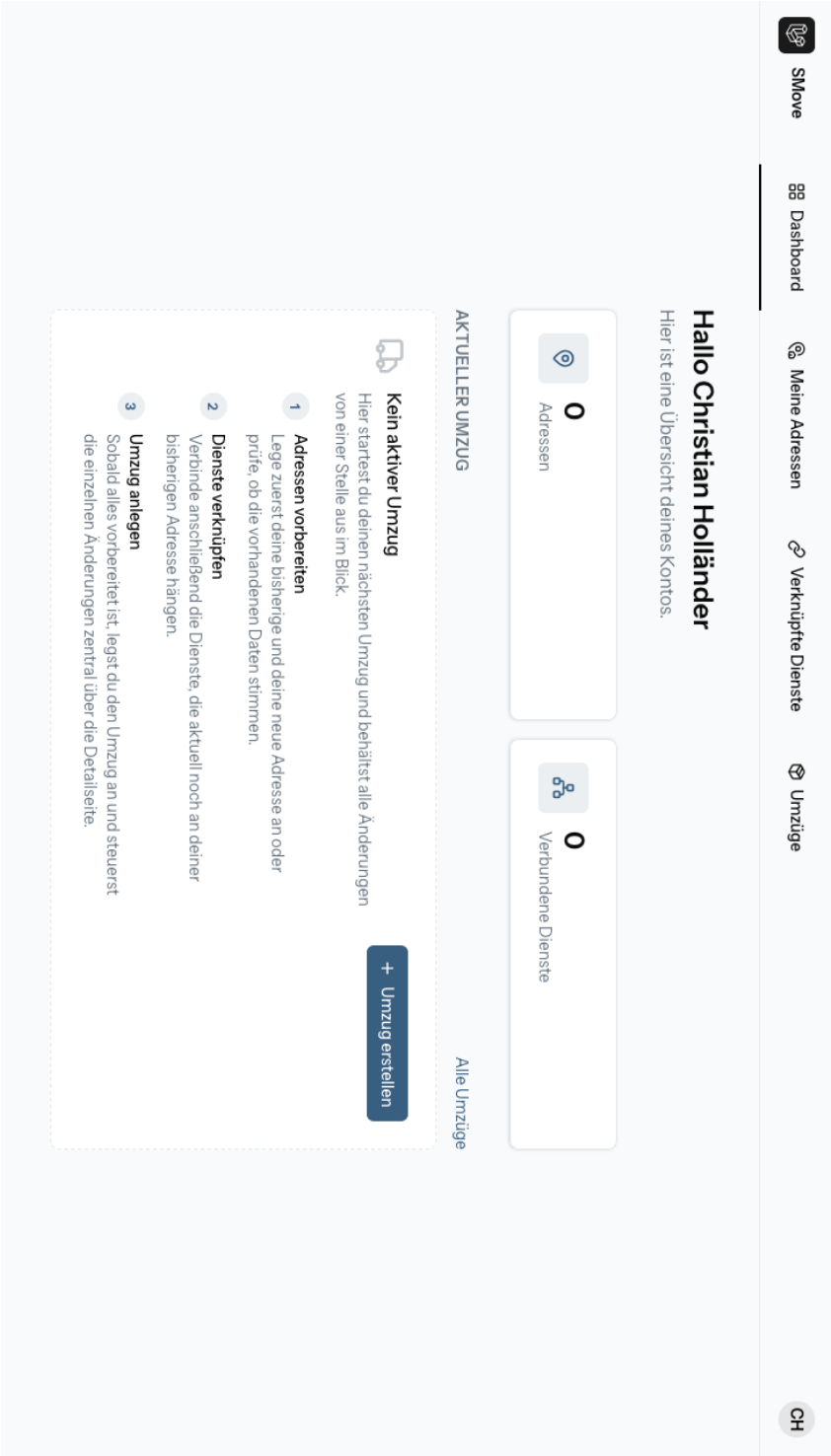


Abbildung A.3.: Dashboard nach Registrierung: Leerer Startzustand mit dreischrittiger Onboarding-Anleitung

A.3.2. Browser-Extension

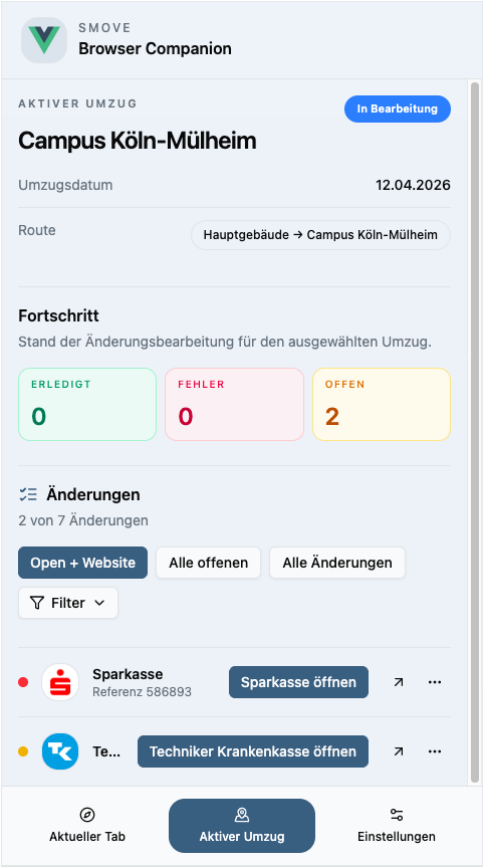


Abbildung A.4.: Extension-Popup: Tab „Aktiver Umzug“ mit Fortschrittsübersicht und offenen Änderungen

A.4. OAuth-2.0-Flow der Browser-Extension

Die Authentifizierung der Extension folgt dem OAuth-2.0-Flow mit PKCE (vgl. Abschnitt 5.2.3). Der vollständige Ablauf:

1. Der Background Service Worker initiiert den In-Browser-Redirect zum SMove-Consent-Screen.
2. Nach erfolgreicher Nutzerfreigabe tauscht der Service Worker den Authorization Code gegen Access- und Refresh-Token aus.

3. Beide Token werden in `chrome.storage.local` gespeichert – auf die Extension selbst beschränkt.

4. Vor jedem API-Aufruf wird die Token-Gültigkeit geprüft und erneuert gegebenenfalls den Access Token transparent über den Refresh-Token-Grant.

Der Popup-Kontext kann diesen Flow nicht übernehmen, weil Manifest V3 ihn automatisch schließt, sobald der Nutzer zur OAuth-Autorisierungsseite wechselt. Ein dort gestarteter Flow verliert seinen Callback-Kontext.

A.5. Build-vs.-Buy-Entscheidungen

Tabelle A.3 fasst die wesentlichen Eigenentwicklungs- gegenüber Zukauf-Entscheidungen im Technologie-Stack zusammen (vgl. Abschnitt 5.1).

Tabelle A.3.: Build-vs.-Buy-Entscheidungen im Technologie-Stack

Anforderung	**Entscheidung**	**Begründung**
Adressfeldverschlüsselung	Buy (Encrypted Casts)	Transparente Integration, keine eigene Krypto-Implementierung
OAuth 2.0 + PKCE	Buy (Laravel Passport)	Standardkonformer Authorization-Server inklusive Consent-UI
Policy-basierte Autorisierung	Buy (Laravel Policies)	Direkte Framework-Integration, geringe Fehleranfälligkeit
Dienstspezifische Feldmappings	Build	Keine generische Lösung existiert; domänenspezifisches Wissen nötig
Audit-Log-System	Build (Laravel Events)	Event-System als Basis, domänenspezifische Log-Struktur

A.6. Ausgewählte Code-Listings

A.6.1. BaseExecutionStrategy

Die folgende vereinfachte Darstellung zeigt die relevanten Teile der abstrakten Basisklasse `BaseExecutionStrategy`. Sie enthält die einzige abstrakte Methode (`isAvailable`), die Ausführungslogik mit optionalem Queue-Dispatch für Hintergrundstrategien sowie die Lifecycle-Methoden für Flow-Verwaltung.

```
abstract class BaseExecutionStrategy implements BasicStrategy
{
    // Muss von jeder Strategie implementiert werden.
    // Prueft, ob die Strategie fuer den gegebenen Dienst
    // verfuegbar ist (z.B. ob ServiceEmailConfig vorhanden).
    abstract public function isAvailable(Service $service): bool;

    // Startet einen neuen Flow fuer den AddressChange.
    public function start(AddressChange $addressChange):
    AddressChangeFlow
    {
        return $addressChange->startStrategyFlow(
            $this->executionStrategy()
        );
    }

    // Fuehrt die Strategie aus. Hintergrundstrategien
    // (BackgroundExecutable) werden als Queue-Job dispatcht;
    // interaktive Strategien laufen synchron.
    public function execute(AddressChangeFlow $flow): ?
    AddressChangeAttempt
    {
        $attempt = $this->startAttempt($flow);

        if ($this instanceof Channel\BackgroundExecutable) {
            ProcessAddressChangeAttemptJob::dispatch($attempt);
        }

        return $attempt;
    }

    // Flow-Lifecycle: Abbrechen und als fehlgeschlagen markieren.
    public function cancelFlow(AddressChangeFlow $flow): void
    {
        $flow->cancel(AddressChangeFlowStatusReason::CancelledByUser);
    }

    public function failFlow(AddressChangeFlow $flow): void
    {
```

```
38          $flow->fail(
39              AddressChangeFlowStatusReason::UserReportedUnsuccessful
40          );
41      }
42  }
```

Listing A.1: BaseExecutionStrategy – relevante Auszüge

A.6.2. ExecutionStrategy-Enum

Das `ExecutionStrategy`-Enum bildet jeden Enum-Wert auf eine konkrete Strategieklasse ab. Die `resolve()`-Methode instanziiert die Klasse über den Laravel Service Container. Die `isSupportedFor()`-Methode prüft zur Laufzeit, ob die Strategie für einen bestimmten Dienst verfügbar ist. Die `prioritized()`-Methode definiert die Anzeigereihenfolge im UI: API-Integrationen werden vor Website- und E-Mail-Strategien angezeigt.

```
1   enum ExecutionStrategy: string
2   {
3       case EmailAutoSend     = 'email_auto_send';
4       case EmailTemplateCopy = 'email_template_copy';
5       case WebsiteOpen       = 'website_open';
6       case ApiCustomApi      = 'api_custom_api';
7       case ApiWebhook        = 'api_webhook';
8       case ApiPull           = 'api_pull';
9
10      // Bevorzugte Anzeigereihenfolge im \gls{ui} (\gls{api} > Website
        > E-Mail).
11      public static function prioritized(): array
12      {
13          return [
14              self::ApiCustomApi, self::ApiWebhook, self::ApiPull,
15              self::WebsiteOpen,
16              self::EmailTemplateCopy, self::EmailAutoSend,
17          ];
18      }
19
20      // Instanziiert die konkrete Strategieklasse via Service Container
        .
21      public function resolve(): BaseExecutionStrategy
22      {
23          return app($this->strategyClass());
24      }
25
26      // Prueft, ob die Strategie fuer den Dienst verfuegbar ist.
27      public function isSupportedFor(Service $service): bool
28      {
```

```
29          return $this->resolve()->isAvailable($service);
30      }
31
32      // Ordnet Enum-Werte konkrete Klassen zu.
33      public function strategyClass(): string
34      {
35          return match ($this) {
36              self::EmailAutoSend    => AutoSendEmailStrategy::class,
37              self::EmailTemplateCopy => TemplateCopyManualEmailStrategy
    ::class,
38              self::WebsiteOpen      => OpenWebsiteStrategy::class,
39              self::ApiCustomApi     => ApiCustomApiStrategy::class,
40              self::ApiWebhook       => ApiWebhookStrategy::class,
41              self::ApiPull          => ApiPullStrategy::class,
42          };
43      }
44  }
```

Listing A.2: ExecutionStrategy-Enum – Auszug der zentralen Methoden

A.6.3. ServiceWebsiteConfig – Struktur der Feldmappings

Eine `ServiceWebsiteConfig` verknüpft einen Dienst mit seiner Adressänderungsseite und enthält optionale dienstspezifische Feldmappings. Das JSON-Feld *form_field_-mappings unterstützt zwei Mapping-Typen:

- **simple**: Ein SMove-Adressfeld wird direkt auf einen CSS-Selektor abgebildet.
- **composite**: Mehrere Adressfelder werden über ein Template in ein einzelnes Formularfeld kombiniert (z. B. `{postcode} {city}` für Dienste mit einem kombinierten Stadtfeld).

```
1  {
2    "url": "https://www.example.de/mein-konto/adresse",
3    "domain": "example.de",
4    "url_match_pattern": "https://www.example.de/mein-konto/adresse*",
5    "form_field_mappings": {
6      "mappings": [
7        {
8          "type": "simple",
9          "smove_field": "street_name",
10         "selector": "#street"
11       },
12       {
13         "type": "simple",
14         "smove_field": "house_number",
15         "selector": "#house-nr"
```

```
16        },
17        {
18          "type": "composite",
19          "selector": "#city-line",
20          "template": "{postcode} {city}"
21        },
22        {
23          "type": "simple",
24          "smove_field": "country",
25          "selector": "select[name='country']"
26        }
27      ]
28    }
29  }
```

Listing A.3: Beispielhafte ServiceWebsiteConfig mit Feldmappings

Ist `form_field_mappings` leer oder nicht gesetzt, greift die generische Felderkennungslogik des Content Scripts (vgl. Listing A.6.4).

A.6.4. Generische Felderkennung im Content Script

Die Funktion `inferFillTarget` implementiert den zweistufigen Erkennungsansatz der Browser-Extension. Stufe 1 wertet das `autocomplete`-Attribut des HTML-Felds aus; Stufe 2 greift auf Regex-Muster zurück, die gegen aggregierte Feldmetadaten (`name`, `id`, `aria-label`, `placeholder`, `title`, `<label>`-Text) geprüft werden.

```
1  export function inferFillTarget(
2      field: FillableFieldElement,
3      metadata: string,
4  ): FillTarget | null {
5      // Stufe 1: \gls{html} autocomplete-Attribut (\gls{whatwg}-
       Standard)
6      const autocompleteValue = getAttributeValue(field, 'autocomplete')
       ;
7      switch (autocompleteValue) {
8          case 'address-level1':  return DataFieldEnum.State;
9          case 'address-level2':  return DataFieldEnum.City;
10         case 'address-line1':
11         case 'street-address':  return combinedStreetAddressTarget;
12         case 'address-line2':   return DataFieldEnum.Extra;
13         case 'country':
14         case 'country-name':    return DataFieldEnum.Country;
15         case 'postal-code':     return DataFieldEnum.Postcode;
16         default: break;
17     }
18
```

```
19      if (!metadata) return null;
20
21      // Stufe 2: Regex-Matching gegen Feldmetadaten (DE + EN)
22      if (matchesPattern(metadata, /\b(country|land)\b/))
23          return DataFieldEnum.Country;
24
25      if (matchesPattern(metadata, /\b(postal|postcode|post code|zip|zip
         code|plz)\b/))
26          return DataFieldEnum.Postcode;
27
28      if (matchesPattern(metadata, /\b(state|province|region|bundesland)
        \b/))
29          return DataFieldEnum.State;
30
31      if (matchesPattern(metadata, /\b(city|town|ort|locality)\b/))
32          return DataFieldEnum.City;
33
34      if (matchesPattern(metadata,
35          /\b(house number|street number|hausnummer|house_number|nr\.)\b
        /))
36          return DataFieldEnum.HouseNumber;
37
38      if (matchesPattern(metadata,
39          /\b(address line 2|address2|apartment|extra|adresszusatz|c\/o)
        \b/))
40          return DataFieldEnum.Extra;
41
42      if (matchesPattern(metadata,
43          /\b(address line 1|address1|street address|anschrift)\b/))
44          return combinedStreetAddressTarget;
45
46      if (matchesPattern(metadata,
47          /\b(street name|street|strasse|strae|road|thoroughfare)\b/))
48          return DataFieldEnum.StreetName;
49
50      return null;
51  }
```

Listing A.4: inferFillTarget – zweistufige Felderkennung (TypeScript)

A.6.5. ProcessAddressChangeAttemptJob – Retry-Mechanismus

Der `ProcessAddressChangeAttemptJob` implementiert den in Abschnitt 5.4 beschriebenen Retry-Mechanismus mit exponentiellem Backoff. Das `ShouldBeUnique`-Interface stellt sicher, dass für denselben Versuch nie zwei parallele Job-Instanzen laufen. `tryClaimAttempt()` setzt den Status atomar auf `Processing`. Eine veraltete Claim-Sperre (über den konfigurierten `retry_after`-Schwellenwert hinaus) wird dabei über-

schrieben. Im Fehlerfall (`failed()`) wird bei weniger als drei Versuchen ein neuer Versuch erstellt und mit exponentiell wachsender Verzögerung (5 min × Versuchsnummer) eingereiht.

```
class ProcessAddressChangeAttemptJob implements ShouldBeUnique, ShouldQueue
{
    public int $uniqueFor = 3600;

    public function uniqueId(): string
    {
        return (string) $this->addressChangeAttempt->id;
    }

    // Setzt Attempt atomar auf "Processing". Gibt false zurueck,
    // wenn der Attempt bereits von einem anderen Job beansprucht
    // wird (und der Lock noch nicht veraltet ist).
    private function tryClaimAttempt(AddressChangeAttempt $attempt): bool
    {
        $staleThreshold = now()->subSeconds($this->retryAfterSeconds());

        return AddressChangeAttempt::query()
            ->whereKey($attempt->id)
            ->where(function ($q) use ($staleThreshold) {
                $q->where('status', AddressChangeAttemptStatus::Started)
                    ->orWhere(function ($q) use ($staleThreshold) {
                        $q->where('status', AddressChangeAttemptStatus::Processing)
                            ->where('processing_started_at', '<=', $staleThreshold);
                    });
            })
            ->update([
                'status' => AddressChangeAttemptStatus::Processing,
                'processing_started_at' => now(),
            ]) === 1;
    }

    // Wird von Laravel aufgerufen, wenn der Job fehlschlaegt.
    // Unter drei Versuchen wird ein neuer Attempt mit exponentiellem
    // Backoff dispatcht; danach wird der Flow als fehlgeschlagen markiert.
    public function failed(?Throwable $exception): void
    {
        // ... Attempt-Status auf Failed setzen ...

        if (! $this->shouldRetry()) {
```

```
            $flow->update([
                'status' => AddressChangeFlowStatus::Failed,
                'status_reason' => AddressChangeFlowStatusReason::
    BackgroundJobFailed,
            ]);
            return;
        }

        $newAttempt = $flow->createAttempt();
        $attemptsCount = $flow->attempts()->count();

        // Exponentielle Verzoegerung: 5 min x Versuchsnummer
        self::dispatch($newAttempt)->delay(now()->addMinutes(5 *
    $attemptsCount));
    }

    public function shouldRetry(): bool
    {
        return $this->addressChangeAttempt->flow->attempts()->count()
    < 3;
    }
}
```

Listing A.5: ProcessAddressChangeAttemptJob – Claim- und Retry-Logik (vereinfacht)

A.6.6. AddressChangeFlowObserver – Status-Propagierung

Der `AddressChangeFlowObserver` propagiert Statusänderungen eines `AddressChangeFlow` automatisch auf den übergeordneten `AddressChange` und dessen `Relocation`. `refreshStatus()` auf `AddressChange`-Ebene leitet den AC-Status aus dem aktuell aktiven Flow ab; dasselbe gilt für den gecachten Relocation-Status. Die Methode `shouldRefreshAddressChange()` stellt sicher, dass Statusaktualisierungen auch durch abgeschlossene (finale) Flows ausgelöst werden, nicht nur durch den aktuell aktiven.

```
class AddressChangeFlowObserver
{
    // Wird bei created, updated und deleted ausgeloest.
    private function refreshAddressChange(
        AddressChangeFlow $flow
    ): void {
        $addressChange = $flow->addressChange()->first();

        if ($addressChange === null) return;

        // Status des AddressChange aus dem aktiven Flow ableiten,
        // wenn der geaenderte Flow der aktive ist ODER in einem
```

```
13          // finalen Status landet.
14          if ($this->shouldRefreshAddressChange($flow, $addressChange->
    current_flow_id)) {
15              $addressChange->refreshStatus();
16          }
17
18          // Gecachten Relocation-Status ebenfalls aktualisieren.
19          $addressChange->relocation()->first()?->refreshStatus();
20      }
21
22      private function shouldRefreshAddressChange(
23          AddressChangeFlow $flow,
24          ?int $currentFlowId,
25      ): bool {
26          return $currentFlowId === $flow->id
27              || $flow->status->isFinal();
28      }
29  }
```

Listing A.6: AddressChangeFlowObserver – Weiterleitung von Statusänderungen

A.7. Datenbankschema – Entity-Relationship-Diagramme

Die folgenden Diagramme zeigen das relationale Datenbankschema von SMove. Abbildung A.5 gibt einen vollständigen Überblick; die Abbildungen A.6–A.8 gliedern das Schema thematisch auf. Abbildung A.9 vertieft den Ausführungsteil des Schemas.

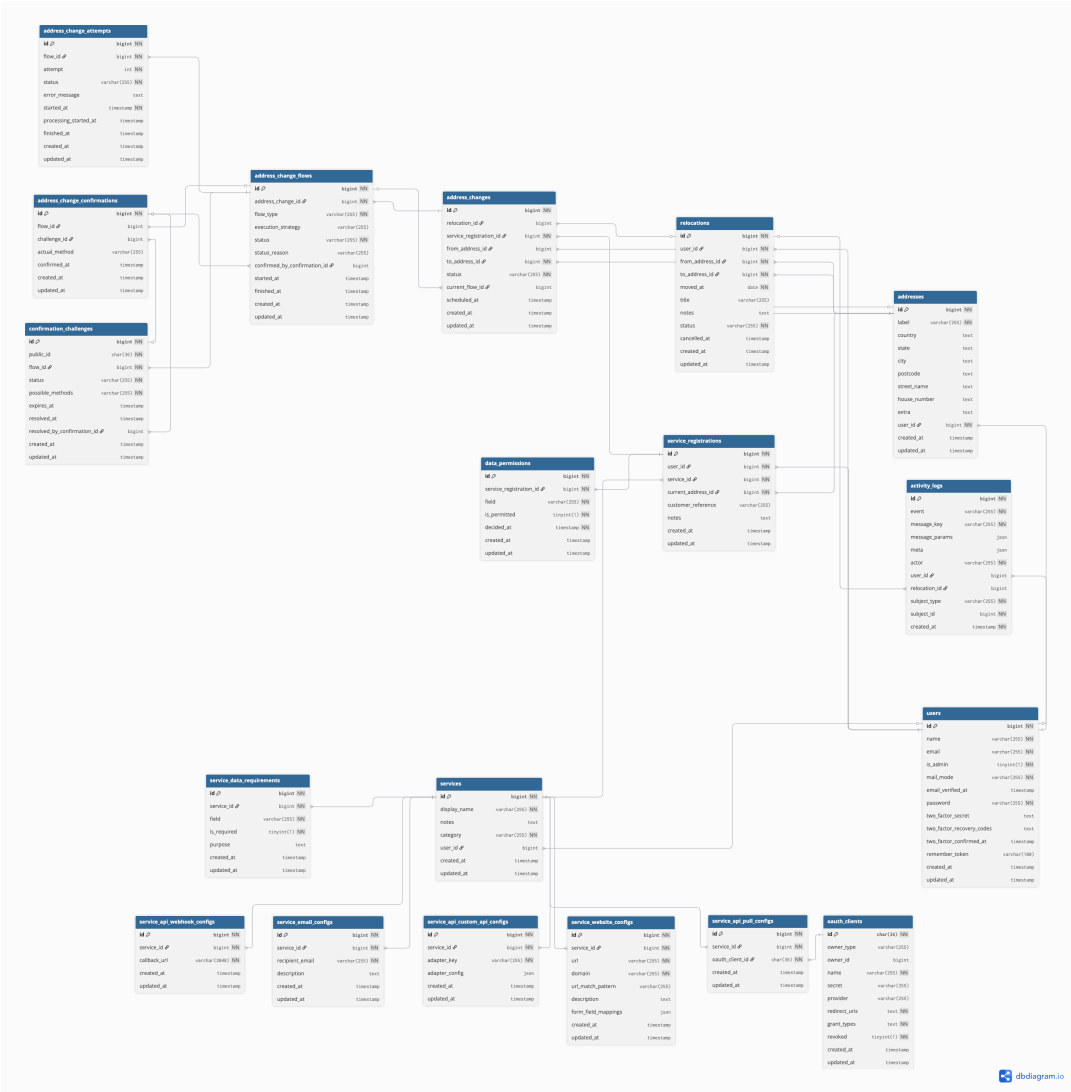


Abbildung A.5.: Vollständiges ER-Diagramm (Übersicht)

Abbildung A.6 zeigt den nutzerzentrierten Kern des Schemas: die Tabellen `users` und `addresses` sowie deren Verknüpfung über `service_registrations`, die eine Adresse einem externen Dienst zuordnet. Der `audit_log` protokolliert alle datenschutzrelevanten Zugriffe und Änderungen an personenbezogenen Daten.

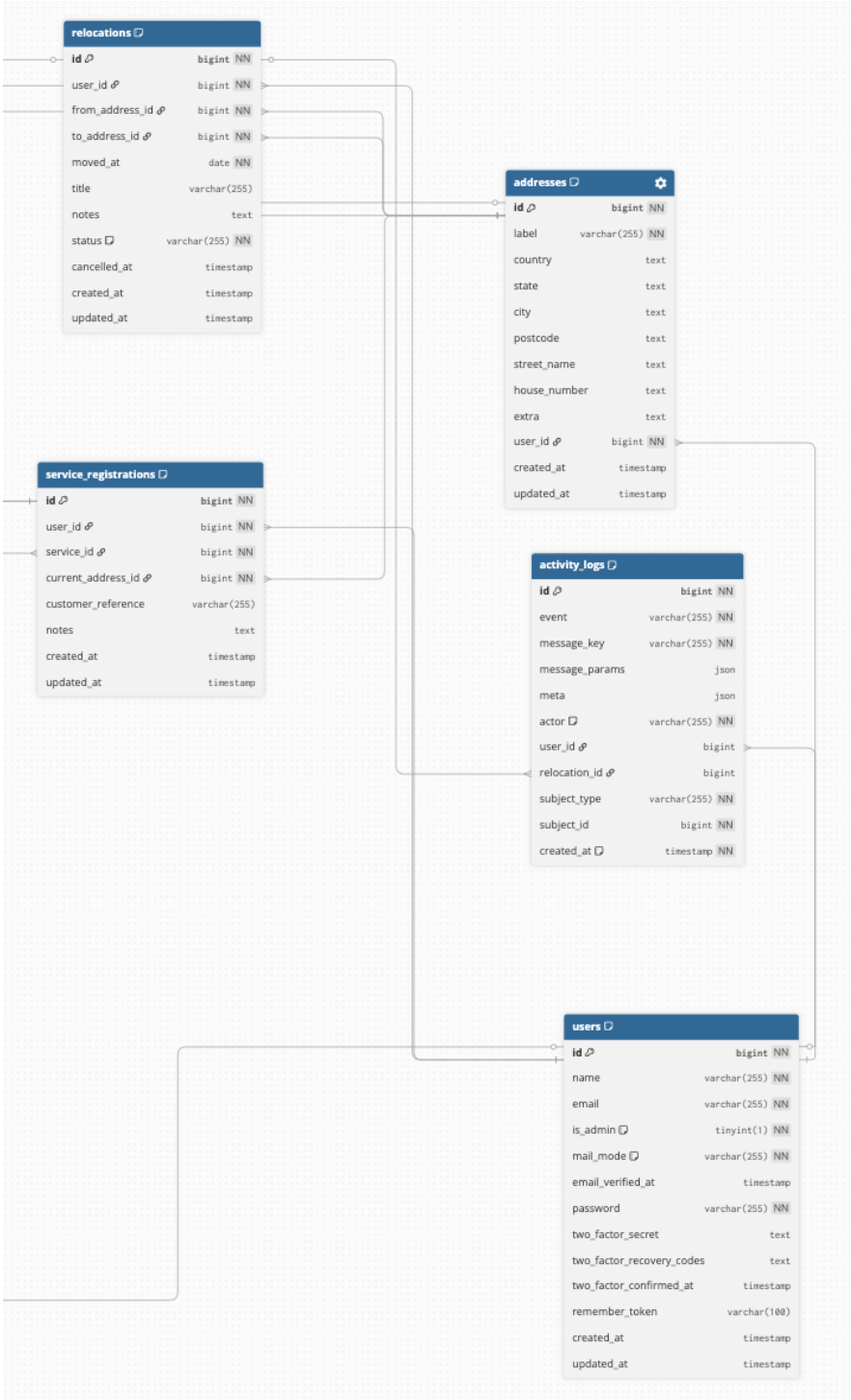


Abbildung A.6.: ER-Diagramm: Nutzer, Adressen, Registrierungen und Audit-Log

Abbildung A.7 zeigt die Service Registry: Die zentrale Tabelle `services` wird durch typisierte Kanalkonfigurationen erweitert (`service_email_configs`, `service_api_configs`, `service_extension_configs`), die jeweils die für den jeweiligen Integrationskanal notwendigen Parameter hinterlegen. Diese Struktur ermöglicht es, pro Dienst mehrere Kanäle parallel zu konfigurieren.

Abbildung A.8 zeigt den Umzugsteil des Schemas: Ein `relocation`-Eintrag bündelt mehrere `address_changes`, von denen jeder eine Statusmaschine durchläuft. Jedem `address_change` ist ein `flow` zugeordnet, der die eigentliche Ausführung koordiniert und die Verbindung zur gewählten Strategie herstellt.

Abbildung A.9 vertieft die Ausführungsebene: `attempts` dokumentieren einzelne Ausführungsversuche eines Flows inklusive Fehlerdetails; `confirmation_challenges` erzeugen einmalig gültige Tokens für die externe Bestätigung; `confirmations` protokollieren den Zeitpunkt und die Art der Bestätigung.

Abbildung A.7.: ER-Diagramm: Service Registry mit typisierten Kanalkonfigurationen

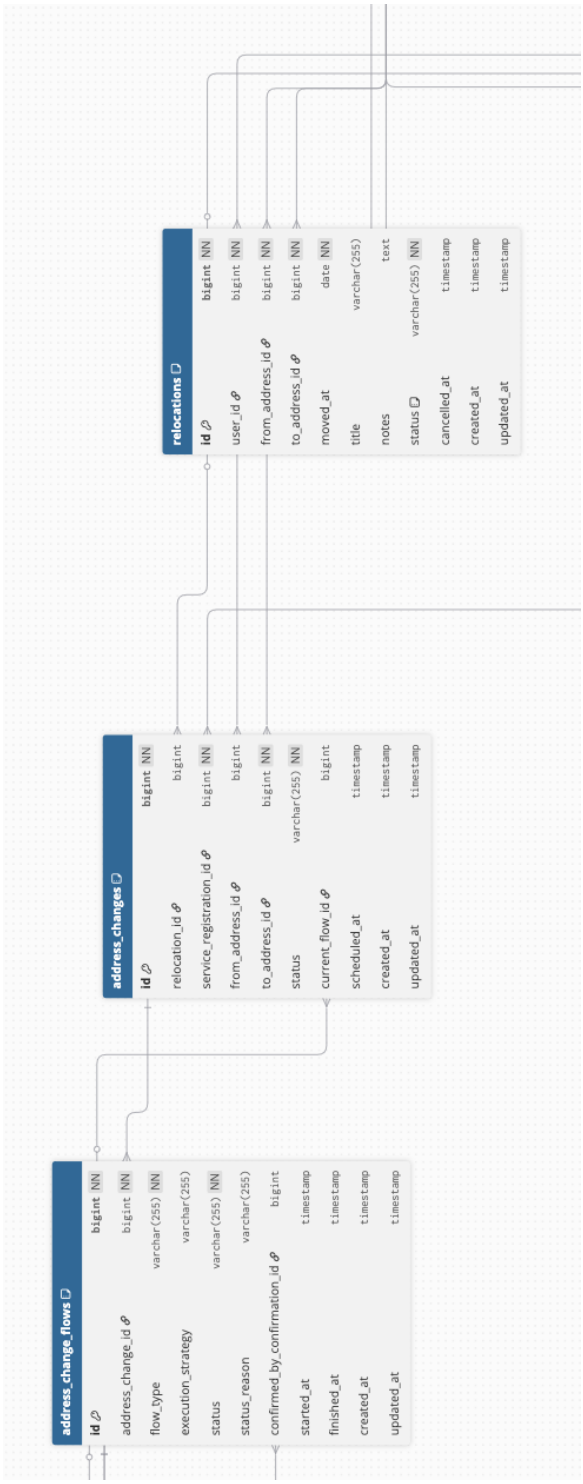


Abbildung A.8.: ER-Diagramm: Relocation, AddressChange und Flow

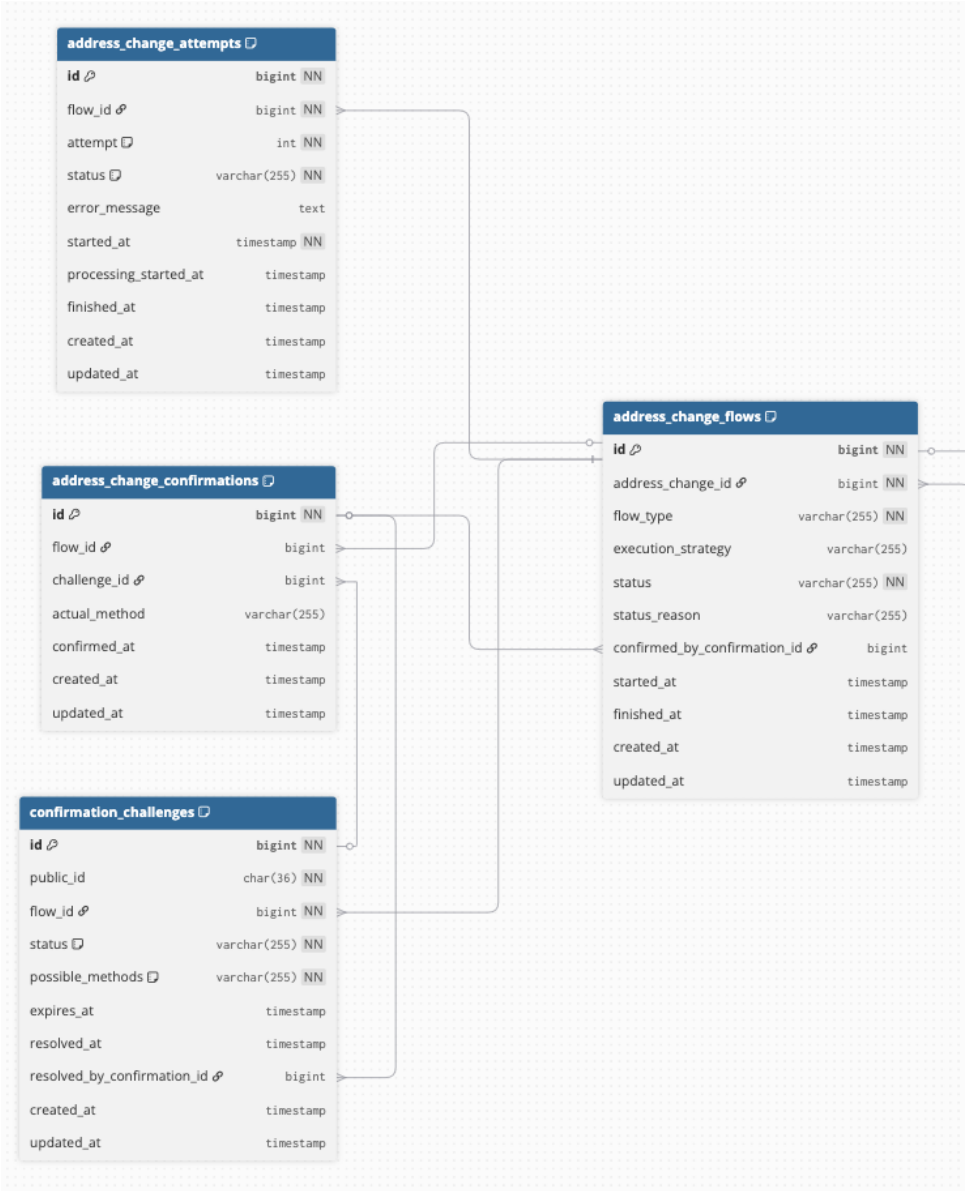


Abbildung A.9.: ER-Diagramm: Flow-Ausführung – Attempts, Confirmations und Challenges

Einsatz KI-gestützter Werkzeuge

Im Rahmen dieser Masterarbeit wurden KI-gestützte Sprachmodelle unterstützend eingesetzt. Dieser Abschnitt dokumentiert transparent, welche Werkzeuge genutzt wurden, für welche Aufgaben und – wo sinnvoll – mit welchen konkreten Eingaben.

Verwendete Werkzeuge

- **Claude (Anthropic)** – Modelle: Claude Sonnet 4.5 / Sonnet 4.6 sowie Claude Opus 4, genutzt über die *Claude Code*-CLI (Agentic Coding Assistant)
- **Perplexity AI** – KI-gestützte Suchmaschine, genutzt für Literatur- und Faktenrecherche
- **OpenAI Codex** – Codegenerierungsmodell, genutzt über die *Codex*-CLI für Implementierungsaufgaben im Prototyp

Einsatzbereiche und repräsentative Prompts

1. Ideendiskussion und Konzeptverfeinerung

LLMs wurden als Gesprächspartner genutzt, um Ideen, Ansätze und Designentscheidungen zu diskutieren und zu schärfen, wie etwa bei der Wahl der Integrationsarchitektur, der Modellierung der Statusmaschine oder der Abgrenzung der Integrationslevels. Das Werkzeug lieferte dabei Gegenargumente, alternative Perspektiven und Rückfragen, die den Verfasser dazu veranlassten, Annahmen explizit zu begründen oder Konzepte weiterzuentwickeln. Die daraus resultierenden Entscheidungen wurden vom Verfasser getroffen und verantwortet.

2. Codeanalyse und Abgleich mit dem Thesis-Entwurf

Das Werkzeug wurde eingesetzt, um den aktuellen Stand der Prototyp-Implementierung systematisch mit dem Thesis-Entwurf zu vergleichen und konkrete Abweichungen zu identifizieren.

Repräsentativer Prompt:

> *"Analysiere das Projekt und den Code, um Informationen für meine Masterarbeit zu extrahieren. Schreibe Notizen und fasse zusammen. Suche nach Architekturentscheidungen. Vergleiche auch mit dem aktuellen Stand der Thesis und schlage Änderungen vor, die den aktuellen Code- und Projektstand widerspiegeln."*

Repräsentativer Prompt (Folge-Iteration):

> *"Ich habe deine Änderungen überprüft und Kommentare hinzugefügt. Außerdem hat sich die Codebasis weiterentwickelt. Bitte überprüfe das Projekt erneut und schlage Änderungen vor, damit die Thesis dem Code entspricht."*

3. Strukturierung und sprachliche Ausarbeitung

Für einzelne Abschnitte, die konzeptionell skizziert, aber noch nicht ausformuliert waren (z. B. Kap. 3.5 Trade-offs, Kap. 5.4 Herausforderungen), wurde das Werkzeug genutzt, um Rohfassungen auf Basis der vorliegenden Stichpunkte und des Projektkontexts zu erstellen. Alle erzeugten Texte wurden anschließend vom Verfasser inhaltlich geprüft und überarbeitet.

4. Literatur- und Faktenrecherche (Perplexity AI)

Perplexity AI wurde als KI-gestützte Suchmaschine für die Recherche nach Quellen, Projekthintergründen und Fakten eingesetzt – insbesondere zu den analysierten SSI-Projekten (Sovrin, cheqd, Trinsic, Hovi) sowie zu technischen Standards (OAuth 2.0, Manifest V3). Die gefundenen Quellen wurden vom Verfasser eigenständig bewertet und über Zotero in das Literaturverzeichnis aufgenommen.

5. Prototyp-Implementierung (OpenAI Codex)

OpenAI Codex wurde über die *Codex*-CLI für Implementierungsaufgaben im Laravel-Prototyp eingesetzt, etwa zur Generierung von Boilerplate-Code, Migrations-Gerüsten und Test-Stubs. Alle generierten Code-Fragmente wurden vom Verfasser geprüft, angepasst und in das Gesamtsystem integriert.

6. Quellenrecherche und Zitations-Management (Claude)

Claude wurde genutzt, um vorhandene BibTeX-Einträge bestehenden Zitations-Platzhaltern (`\todo{...}`) zuzuordnen und die entsprechenden `\autocite`-Befehle einzusetzen.

Abgrenzung

Sämtliche fachlichen Inhalte, wissenschaftlichen Argumente, Designentscheidungen und Schlussfolgerungen wurden vom Verfasser selbst erarbeitet und verantwortet. Die genannten Werkzeuge wurden ausschließlich als Hilfsmittel für Recherche, Analyse, Textbearbeitung und technische Umsetzung eingesetzt – vergleichbar mit der Nutzung einer Suchmaschine, eines Lektors oder einer Programmierhilfe. Alle KI-generierten Ausgaben wurden kritisch geprüft, inhaltlich bewertet und wo nötig überarbeitet. Alle KI-generierten Ausgaben wurden geprüft; die inhaltliche Verantwortung für alle in dieser Arbeit getroffenen Aussagen und Entscheidungen verbleibt vollständig beim Verfasser.

Erklärung

Ich versichere, die von mir vorgelegte Arbeit selbstständig verfasst zu haben. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder nicht veröffentlichten Arbeiten anderer oder der Verfasserin/des Verfassers selbst entnommen sind, habe ich als entnommen kenntlich gemacht. Sämtliche Quellen und Hilfsmittel, die ich für die Arbeit benutzt habe, sind angegeben. Die Arbeit hat mit gleichem Inhalt bzw. in wesentlichen Teilen noch keiner anderen Prüfungsbehörde vorgelegen.

Ort, Datum	Unterschrift